

Параллельный Python

Модуль 1 - Модуль 1. Введение Python против времени

Урок 1 - Что такое параллелизм и зачем он нужен

Шаг 01:

Миссия 1

Программист Алекс решил ускорить обработку большого списка данных. Он слышал о параллелизме, но не уверен, что это такое. Что лучше всего описывает идею параллелизма?

Подсказка: Вспомни, как повара на кухне делят задачи: один режет, другой жарит, третий подаёт.

Шаг 02:

Миссия 2

Инженер Лена запустила тяжёлый анализ логов. Пока она ждала, браузер и музыка работали без проблем. Что это показывает о современном компьютере?

Подсказка: Даже если один процесс загружен, другие продолжают жить своей жизнью.

Шаг 03:

Миссия 3

Миша запускает функцию, которая скачивает 10 файлов по очереди. Он хочет ускорить процесс. Что ему стоит попробовать?

Подсказка: Когда ты не ждёшь окончания одного дела, чтобы начать другое — это уже намёк на нужное направление.

Шаг 04:

Миссия 4

Когда Петя запустил программу на 8-ядерном процессоре, она использовала только одно ядро. Почему?

Подсказка: Python не всегда может использовать все ядра, если не применён правильный подход.

Шаг 05:

Миссия 5

Разработчик Дима решил использовать многопроцессность для обработки больших CSV-файлов. Что при этом произойдёт?

Подсказка: Каждый процесс живёт своей жизнью, у него своя память и ресурсы.

Шаг 06:

Миссия 6

Настя изучает разные подходы к параллельности. Сопоставь понятие с его описанием.

Подсказка: Потоки делят память, процессы — нет.

🔗 Шаг 07:

Миссия 7

👤 Игорь тестирует разные библиотеки. Сопоставь библиотеку с её основной областью применения.

Подсказка: Вспомни, какая библиотека — для процессов, а какая — для потоков.

🔗 Шаг 08:

Миссия 8

Тимофей пишет программу, которая должна считать данные и обрабатывать их одновременно. Сопоставь понятие с примером из жизни.

Подсказка: Представь ресторан: повар, официант, клиент.

🔗 Шаг 09:

Миссия 9

👤 Разработчик Никита изучает, как распределяется нагрузка. Сопоставь термин с описанием.

Подсказка: Балансировка — это как делить работу между коллегами.

🔗 Шаг 10:

Миссия 10

Алина хочет понять, какие задачи требуют синхронизации. Сопоставь ситуацию и причину синхронизации.

Подсказка: Когда два потока пишут в один файл — возможен хаос.

🔗 Шаг 11:

Миссия 11

📁 У программиста Кости программа скачивает файлы параллельно. Расположи этапы выполнения в правильном порядке.

Подсказка: Сначала создаём задачи, потом запускаем, ждём завершения и подводим итоги.

🔗 Шаг 12:

Миссия 12

👤 Разработчик Артём пишет асинхронный код. Расставь шаги выполнения `async`-функций в правильной последовательности.

Подсказка: Сначала определяем корутину, затем запускаем через `event loop`.

🔗 Шаг 13:

Миссия 13

👤 Катя хочет разделить обработку данных между процессами. Расположи шаги создания пула процессов.

Подсказка: Сначала создаётся пул, потом передаются задачи, затем — закрытие и ожидание.

🔗 Шаг 14:

Миссия 14

👤 Влад строит систему мониторинга серверов. Ему нужно упорядочить этапы конкурентного выполнения.

Подсказка: Сначала ставим задачи, потом следим за их статусами и собираем результаты.

Урок 2 - CPU-bound и IO-bound задачи

Шаг 01:

Миссия 1

Алекс запускает программу, которая выполняет сложные математические вычисления и замечает, что CPU постоянно загружен на 100%. К какому типу задач относится его программа?

Подсказка: Если процессор занят вычислениями, а не ждёт данных — ответ очевиден.

Шаг 02:

Миссия 2

Разработчица Лена пишет программу, которая скачивает тысячи изображений с сайта.

Процессор при этом почти не загружен, но программа работает долго. Какой это тип задачи?

Подсказка: Здесь важнее скорость сети, чем вычисления.

Шаг 03:

Миссия 3

Петя пишет программу, которая читает гигабайты данных с жёсткого диска и анализирует их. Какое место может стать узким горлышком?

Подсказка: Представь, что ты читаешь книгу: глаза успевают, но страницы переворачиваются медленно.

Шаг 04:

Миссия 4

Миша решил ускорить CPU-bound задачу, используя потоки (threading). Но ускорения не заметил. Почему?

Подсказка: Вспомни про ограничение, которое мешает потокам реально работать одновременно в Python.

Шаг 05:

Миссия 5

Аня запускает CPU-bound задачу через multiprocessing. Почему она видит значительный прирост скорости?

Подсказка: Подумай, что каждый процесс получает свой Python-интерпретатор и ядро процессора.

Шаг 06:

Миссия 6

Кирилл пишет программу, которая одновременно скачивает и обрабатывает файлы. Какой подход даст ему наибольшую выгоду?

Подсказка: Если задача включает и ожидание, и вычисления — комбинируй подходы.

Шаг 07:

Миссия 7

 Игорь хочет выбрать подход к оптимизации. Сопоставь тип задачи и лучший метод.

Подсказка: Потоки не ускоряют CPU-bound задачи, но помогают при I/O.

🔮 Шаг 08:

Миссия 8

☞ София объясняет стажёру разницу между CPU и I/O. Сопоставь понятие с описанием.

Подсказка: CPU делает вычисления, I/O ждёт внешние ресурсы.

🔮 Шаг 09:

Миссия 9

Настя пишет асинхронный загрузчик файлов. Расположи шаги выполнения I/O-bound задачи в правильном порядке.

Подсказка: Сначала планируем, потом запускаем, потом ждём результаты.

🔮 Шаг 10:

Миссия 10

Артём распределяет CPU-bound задачу по ядрам. Расположи этапы multiprocessing в правильном порядке.

Подсказка: Создаём пул → передаём задачи → ждём завершения → собираем результат.

🔮 Шаг 11:

Миссия 11

🔗 Лена оптимизирует веб-сканер. Расположи действия для смешанной CPU/I/O задачи.

Подсказка: Сначала скачиваем данные, потом анализируем.

🔮 Шаг 12:

Миссия 12

🔗 Ваня хочет протестировать производительность CPU-bound программы. Расположи действия в логическом порядке.

Подсказка: План → измерение → анализ → оптимизация.

📖 Урок 3 - Почему Python «медленный» и при чём здесь GIL

🔮 Шаг 01:

Миссия 1

Программист Оля запускает простой цикл `for i in range(10**7): x = i*i`. Она замечает, что это занимает значительное время. Что это показывает о Python?

Подсказка: Даже простые вычисления в чистом Python могут быть не быстрыми.

🔮 Шаг 02:

Миссия 2

Разработчик Пётр запускает несколько потоков, каждый выполняет вычисления. Он замечает, что ускорения почти нет. Почему?

Подсказка: Вспомни механизм, который мешает потокам выполнять Python-код одновременно.

🔮 Шаг 03:

Миссия 3

Вика читает статью: «GIL — главный тормоз Python». Что такое GIL?

Подсказка: Это внутренний механизм CPython, связанный с потоками.

🔗 Шаг 04:

Миссия 4

Лена запускает многопроцессный код для вычислений. Почему это даёт прирост производительности?

Подсказка: Каждый процесс имеет свой интерпретатор и не подчинён GIL другого.

🔗 Шаг 05:

Миссия 5

Марина читает чужой код с множеством потоков, но не видит улучшения. Что может быть причиной?

Подсказка: Проверь, не является ли задача «CPU-bound». Тогда потоки почти не помогут.

🔗 Шаг 06:

Миссия 6

Артём хочет разобраться: сопоставь термин и описание.

Подсказка: Подумай, где появляется «lock», где — отдельные процессы и т. д.

Термины:

Ответ:

- GIL
- Интерпретатор CPython
- Поток (thread)
- Процесс (process)

Описания:

1. Блокировка, разрешающая только одному потоку исполнять байткод
2. Запускает байткод Python и управляет памятью
3. Лёгкий поток внутри процесса, разделяет память
4. Изолированная единица выполнения с отдельной памятью

🔗 Шаг 07:

Миссия 7

🔍 Мария читает статью и хочет упорядочить: сопоставь концепцию и её следствие.

Подсказка: GIL влияет на потоки, но не на процессы.

Концепции:

Ответ:

- Наличие GIL
- Многопроцессность
- Многопоточность
- Параллелизм на уровне ядер

Следствия:

1. Только один поток исполняет Python-байткод одновременно
2. Процессы могут выполняться параллельно на разных ядрах
3. Потоки конкурируют из-за GIL

4. Выделение задач на несколько ядер ускоряет вычисления

Шаг 08:

Миссия 8

💡 Сергей хочет понять, что замедляет код. Сопоставь сценарий и узкое место.

Подсказка: Обрати внимание, где операции ввода-вывода, где вычисления.

Сценарии:

Ответ:

- Сложное численное моделирование
- Скачивание файлов из сети
- Сжатие большого архива
- Чтение логов с диска

Узкие места:

1. CPU (вычисления)
2. I/O (сеть)
3. CPU
4. I/O (диск)

Шаг 09:

Миссия 9

⚙️ Даша хочет выбрать способ ускорить код. Сопоставь проблему и предложение.

Подсказка: Поток не решит проблему GIL.

Проблемы:

Ответ:

- CPU-bound код
- I/O-bound код
- Смешанная работа (I/O + вычисления)
- Потребность в однопоточном порядке

Решения:

1. Запуск через multiprocessing или C-расширения
2. Использование threading или asyncio
3. Комбинация multiprocessing + asyncio/поток
4. Оставить последовательное исполнение с профилированием

Шаг 10:

Миссия 10

🔍 Женя анализирует производительность. Сопоставь инструмент и что он показывает.

Подсказка: Профайлер покажет, где код «висит» на вычислениях.

Инструменты:

Ответ:

- cProfile
- time.time()
- tracemalloc

- line_profiler

Показывают:

1. Время исполнения функций и их вызовов
2. Простая замерка времени участка кода
3. Память, выделенная в разных точках
4. Время исполнения на уровне строк

Шаг 11:

Миссия 11

Максим хочет распараллелить тяжёлую функцию. Расположи шаги использования multiprocessing.

Подсказка: Пул → задачи → закрыть → ждать → собрать.

Шаг 12:

Миссия 12

 Света оптимизирует смешанную задачу I/O + вычисления. Расположи логику по этапам.

Подсказка: Сначала выполняется ввод-вывод, затем вычисления на результатах.

Шаг 13:

Миссия 13

 Павел строит тест производительности многопроцессного кода. Расположи шаги в логическом порядке.

Подсказка: Запуск → сбор данных → сравнение → итерации.

Шаг 14:

Миссия 14

 Финальная миссия: Ирина хочет объединить все шаги для ускорения Python-кода. Расположи этапы от планирования до проверки оптимизации.

Подсказка: Сначала анализ, затем эксперимент, затем выбор подхода и проверка.

Урок 4 - Практика измеряем время выполнения

Шаг 01:

Миссия 1

Программист Антон решил узнать, сколько времени выполняется его функция сортировки. Какой модуль Python ему пригодится?

Подсказка: Этот модуль — старый друг всех, кто замеряет секунды исполнения кода.

Шаг 02:

Миссия 2

© Кира запустила таймер: она записала `start = time.time()` , потом `end = time.time()` . Как вычислить длительность выполнения?

Подсказка: Подумайте, что произойдёт, если вычесть момент начала из момента окончания.

🔧 Шаг 03:

Миссия 3

⚙ Тимур заметил, что при использовании `time.time()` на Windows точность оставляет желать лучшего. Какую альтернативу стоит применить для более точных измерений?

Подсказка: Этот метод из того же модуля, но с приоритетом на высокую точность.

🔧 Шаг 04:

Миссия 4

☕ Аня решила сделать перерыв и добавила в код `time.sleep(5)`. Что произойдёт?

Подсказка: Эта функция не ускоряет, а наоборот делает паузу.

🔧 Шаг 05:

Миссия 5

📝 Никита написал бенчмарк и заметил, что на каждом запуске время отличается. Почему?

Подсказка: На результат влияют фоновые процессы, кеширование и нагрузка на систему.

🔧 Шаг 06:

Миссия 6

🔗 У Даши две функции: одна быстрая, другая — медленная. Что будет корректным способом их сравнить по времени?

Подсказка: Сравнивай в одинаковых условиях и несколько запусков подряд.

🔧 Шаг 07:

Миссия 7

Сопоставьте типы измерения времени с их описанием.

Подсказка: Одни функции возвращают секунды, другие — наносекунды.

🔧 Шаг 08:

Миссия 8

⚙ Вася решил выбрать подходящий способ измерения времени. Сопоставьте задачу и подходящий инструмент.

Подсказка: Если нужна высокая точность — берите `perf_counter()`.

🔧 Шаг 09:

Миссия 9

🕒 Сопоставьте термин и его значение.

Подсказка: Важно отличать время процессора от реального времени.

🔧 Шаг 10:

Миссия 10

⚡ Сопоставьте код и его результат по времени выполнения.

Подсказка: `time.sleep()` всегда добавляет задержку.

🔮 Шаг 11:

Миссия 11

🕒 Сопоставьте способ и тип задачи, где он используется.

Подсказка: Для микротестов — одно, для асинхронности — другое.

🔮 Шаг 12:

Миссия 12

🗑️ Расположите шаги измерения времени выполнения функции в правильном порядке.

Подсказка: Сначала зафиксируй начало, потом конец.

🔮 Шаг 13:

Миссия 13

Расставьте этапы оптимизации замера по возрастанию сложности.

Подсказка: Начни с простых таймеров, а потом переходи к бенчмаркингу.

🔮 Шаг 14:

Миссия 14

🔧 Расположите шаги подготовки мини-бенчмарка.

Подсказка: Перед измерением убедись, что код готов и стабильный.

🔮 Шаг 15:

Миссия 15

🔗 Упорядочите действия разработчика, который хочет измерить производительность функции с помощью timeit.

Подсказка: Сначала импорт, потом замер, потом анализ.

🔮 Шаг 16:

Миссия 16

📌 Расставьте шаги для измерения времени выполнения блока кода при помощи контекстного менеджера.

Подсказка: Контекст — это структура с __enter__ и __exit__.

📖 Урок 5 - Квест найди узкое место в коде

🔮 Шаг 01:

Миссия 1

Программист Миша заметил, что его программа стала работать медленнее. Какой первый шаг, чтобы найти узкое место в коде?

Подсказка: Сначала важно понять, какая часть программы тормозит сильнее других.

🔮 Шаг 02:

Миссия 2

🔧 Ольга решила использовать модуль cProfile . Что он позволяет сделать?

Подсказка: Этот инструмент помогает увидеть, какие функции занимают больше всего времени при выполнении.

🔗 Шаг 03:

Миссия 3

🔗 Сергей запустил `cProfile.run('main()')` . Что он получит на выходе?

Подсказка: Этот инструмент выводит таблицу с временем выполнения и количеством вызовов функций.

🔗 Шаг 04:

Миссия 4

📌 Даша использует `time.perf_counter()` внутри цикла. Почему важно запускать тесты несколько раз?

Подсказка: Один замер может быть неточным из-за фоновых процессов и кеша.

🔗 Шаг 05:

Миссия 5

📊 После профилирования Андрей видит, что функция `parse_data()` вызывается 10000 раз и занимает 85% времени. Что это означает?

Подсказка: Посмотрите на долю времени — она покажет, где узкое место.

🔗 Шаг 06:

Миссия 6

🔗 Лена решила оптимизировать код, но не измерила время до и после изменений. Почему это ошибка?

Подсказка: Без данных до и после невозможно понять, стала ли программа быстрее.

🔗 Шаг 07:

Миссия 7

Сопоставьте инструмент профилирования с его назначением.

Подсказка: Один собирает статистику по функциям, другой — визуализирует результат.

🔗 Шаг 08:

Миссия 8

📊 Сопоставьте метрику профилирования с её значением.

Подсказка: Обратите внимание, что `cumtime` — суммарное время.

🔗 Шаг 09:

Миссия 9

📌 Сопоставьте тип проблемы производительности с её примером.

Подсказка: Иногда виноваты не алгоритмы, а ввод-вывод.

🔗 Шаг 10:

Миссия 10

⚡ Сопоставьте ситуацию с оптимальным подходом к анализу.

Подсказка: Для узких мест в циклах — профилирование, для общих — агрегированные метрики.

Шаг 11:

Миссия 11

 Сопоставьте действия разработчика с их целью.

Подсказка: Иногда цель — не ускорить всё, а найти конкретную боль.

Шаг 12:

Миссия 12

 Расположите шаги поиска узкого места в правильном порядке.

Подсказка: Сначала нужно измерить, потом анализировать, потом исправлять.

Шаг 13:

Миссия 13

 Расположите этапы оптимизации кода по возрастанию сложности.

Подсказка: Начни с простого — алгоритмов, потом переходи к потокам и асинхронности.

Шаг 14:

Миссия 14

 Расставьте шаги анализа профиля с помощью pstats.

Подсказка: Сначала нужно загрузить файл, потом фильтровать и сортировать данные.

Шаг 15:

Миссия 15

 Расположите шаги подготовки визуального профиля с помощью snakeviz.

Подсказка: Визуализация требует сохранённого профиля и запуска snakeviz.

Шаг 16:

Миссия 16

 Расставьте шаги рациональной оптимизации программы.

Подсказка: Не оптимизируй вслепую — сначала измерь, потом действуй.

Модуль 2 - Модуль 2. Потоки одна память — много рук

Урок 1 - Основы модуля threading

Шаг 01:

Миссия 1

Вы — новичок, запускаете первый поток в Python с threading.Thread. Какой метод нужно вызвать, чтобы поток начал работу?

Подсказка: У потока есть специальный метод, не run(), который иницирует начало выполнения.

Шаг 02:

Миссия 2

 Вы пишете функцию, и хотите, чтобы один поток ждал окончания другого. Какой метод вы используете?

Подсказка: Это блокирующий метод, поток, вызвавший его, приостанавливается до завершения другого.

🔧 Шаг 03:

Миссия 3

🔍 Какой атрибут у потока показывает, запущен он или нет?

Подсказка: Имя этого свойства напоминает “alive”.

🔧 Шаг 04:

Миссия 4

🔍 Вы хотите прервать поток (не прямо, но корректно). Какую практику вы примените?

Подсказка: Обычно это контроль флага, который поток сам проверяет и завершает.

🔧 Шаг 05:

Миссия 5

🔍 Передача аргументов в поток. Какой способ корректный при создании Thread?

Подсказка: В конструкторе Thread есть параметры args и kwargs.

🔧 Шаг 06:

Миссия 6

🔍 Сопоставьте термин и описание в контексте threading:

Термины:

Ответ:

- daemon thread
- main thread
- race condition
- GIL

Описание:

1. Главный поток, из которого создаются другие
2. Поток, который автоматически завершится при завершении программы
3. Состояние, когда потоки конфликтуют при доступе к общей памяти
4. Ограничение интерпретатора, не дающее параллелизм потокам в Python

Подсказка: Подумайте, какой поток “фоновой”, какой — главный, и что мешает настоящему параллелизму.

🔧 Шаг 07:

Миссия 7

🔍 Сопоставьте методы Thread и их действие:

Методы:

Ответ:

- start
- join
- is_alive
- run

Действие:

1. Возвращает True, если поток всё ещё работает
2. Запускает поток, вызывает run в новом контексте
3. Ожидает завершения потока
4. Функция, которую исполняет поток при запуске

Подсказка: Метод run — не запускает поток сам по себе, а описывает, что поток делает.

Шаг 08:

Миссия 8

Сопоставьте типичные ошибки при работе с потоками и их последствия:

Ошибка:

Ответ:

- Не использовать lock при модификации общей переменной
- Вызывать join на самом себе
- Запускать поток дважды (start дважды)
- Не обрабатывать исключения внутри потока

Последствие:

1. RuntimeError: threads can only be started once
2. Блокировка программы / deadlock
3. Непредсказуемые значения переменной / race condition проявится
4. Исключения приводят к Silent failure или прекращению потока неожиданно

Подсказка: Подумайте про каждую ошибку: что могло случиться в жизни?

Шаг 09:

Миссия 9

 Сопоставьте синхронизатор и ситуацию, когда его используют:

Синхронизаторы:

Ответ:

- Lock (mutex)
- RLock
- Event
- Semaphore

Ситуация:

1. Только один поток может захватить ресурс одновременно
2. Рекурсивное блокирование внутри одного потока
3. Ожидание сигнала от другого потока, прежде чем продолжить
4. Ограничить число потоков, одновременно использующих ресурс

Подсказка: Семафор ограничивает количество; Event — сигнал; RLock — блокировка, которую может повторно взять тот же поток.

Шаг 10:

Миссия 10

 Сопоставьте методы synchronization-объектов с их действием:

Методы:

Ответ:

- lock.acquire()
- lock.release()
- event.set()
- semaphore.acquire()

Действие:

1. Уменьшить счётчик семафора (захват слота)
2. Установить флаг события, разбудив ожидающие потоки
3. Заблокировать доступ к ресурсу
4. Освободить ресурс, сняв блокировку

Подсказка: acquire — захват, release — освобождение; event.set() сообщает, что событие произошло.

Шаг 11:

Миссия 11

🔑 Упорядочите шаги, как вы бы реализовали безопасный доступ к разделяемой переменной с mutex:

1. Чтение или запись переменной
2. Создать объект lock
3. Вызвать lock.acquire() перед операцией
4. Вызвать lock.release() после операции

Подсказка: Сначала нужно lock, потом операция, затем освобождение, но lock должен быть создан заранее.

Шаг 12:

Миссия 12

🔑 Упорядочите действия при использовании Event для синхронизации двух потоков A и B:

1. Поток A ждёт event.wait()
2. Поток B устанавливает event.set()
3. Поток A продолжает работу после события
4. Создать объект Event перед запуском потоков

Подсказка: Сначала событие создаётся, потом один из потоков ждёт, другой — устанавливает, затем первый продолжает.

Шаг 13:

Миссия 13

🔑 Упорядочите использование Semaphore для ограничения до двух потоков:

1. semaphore.release() после завершения работы внутри потока
2. Создать semaphore с value=2
3. semaphore.acquire() перед началом работы с ресурсом
4. Поток делает свою работу с ресурсом

Подсказка: Сначала инициализация, затем каждый поток захватывает семафор, использует ресурс и отпускает.

🔔 Шаг 14:

Миссия 14

Упорядочите этапы работы с RLock (рекурсивной блокировкой) внутри одного потока:

1. lock.acquire()
2. lock.acquire() повторно (внутри той же функции)
3. lock.release()
4. lock.release() второй раз

Подсказка: При рекурсивном lock тот же поток может брать блокировку несколько раз, но должен освобождать столько же раз.

🔔 Шаг 15:

Миссия 15

🌐 Ваша задача: упорядочите этапы запуска веб-серверов в разных потоках:

1. Прослушивание порта и приём соединений
2. Создание потоков при поступлении соединения
3. Запустить главный цикл сервера
4. Инициализация настроек сервера перед запуском потоков

Подсказка: Настройки идут до цикла, затем сервер слушает, на каждый клиент создаётся поток.

🔔 Шаг 16:

Миссия 16

🔗 Упорядочите шаги участия потоков в барьере:

1. Вызов barrier.wait() в каждом потоке
2. Инициализация barrier с заданным числом участников before threads start
3. После того как все потоки вызовут wait, все продолжают работу
4. Запуск потоков, которые будут участвовать в барьере

Подсказка: Барьер действует, когда все участники дошли до него; сначала задаётся барьер, запускаются потоки, все ждут, после все идут дальше.

📖 Урок 2 - Создание потоков вручную

🔔 Шаг 01:

Миссия 1

Алиса запускает функцию в новом потоке вручную через threading.Thread. Что нужно передать в конструктор, чтобы поток начал работу?

Подсказка: Передайте вызываемую функцию, а не её результат.

🔔 Шаг 02:

Миссия 2

✂ Боб хочет запустить метод экземпляра класса в потоке. Какой способ передачи аргумента target корректен?

Подсказка: Помните, что метод — это функция с первым аргументом self.

🔮 Шаг 03:

Миссия 3

📦 Катя хочет запустить поток в режиме daemon. Что это значит?

Подсказка: Подумайте, что произойдет с демоном-поток, если главный поток завершится.

🔮 Шаг 04:

Миссия 4

🌐 Елена хочет передать параметры в поток, включая ключевые аргументы. Как это сделать?

Подсказка: В `threading.Thread` используются два параметра `args` и `kwargs`.

🔮 Шаг 05:

Миссия 5

👂 Сопоставьте методы потока с их назначением:

Методы:

Ответ:

- `start()`
- `join()`
- `is_alive()`
- `setDaemon()`

Назначения:

1. Запустить исполнение потока
2. Ждать завершения потока
3. Проверить, работает ли поток
4. Установить режим демона

Подсказка: `start()` вызывает запуск, `join()` ждёт, `is_alive` — проверка, `setDaemon` — режим демона.

🔮 Шаг 06:

Миссия 6

🔗 Сопоставьте ситуацию с возможным последствием неправильного использования потоков:

Ситуации:

Ответ:

- Два потока одновременно пишут в одну переменную
- Один поток читает, другой записывает без блокировки
- Поток не завершён перед выходом из программы
- Приоритет демона включён неверно

Последствия:

1. Гонка данных / некорректные данные
2. Непредсказуемое поведение программы или чтение «мусора»
3. Поток оборвётся прерыванием программы
4. Демон-поток продолжает после главного или прерывается слишком рано

Подсказка: Подумайте о потокобезопасности и моменте завершения.

🔧 Шаг 07:

Миссия 7

📖 Сопоставьте атрибут класса Thread с его функцией:

Атрибуты:

Ответ:

- daemon
- name
- ident
- _target

Функции:

1. Имя потока для отладки
2. Флаг демона / пользовательский поток
3. Идентификатор потока (уникальный)
4. Ссылка на целевую функцию

Подсказка: ident генерируется системой, name задаёт человек, daemon — булево, _target — внутреннее хранилище.

🔧 Шаг 08:

Миссия 8

📖 Упорядочьте шаги при создании и запуске потока вручную:

1. Передать target и args/kwargs
2. Выполнять код в потоке
3. Создать объект Thread
4. Вызвать start()

Подсказка: Сначала создаём объект, затем настраиваем, и в конце запускаем.

🔧 Шаг 09:

Миссия 9

🔗 Упорядочите фазы обработки задач в пуле потоков (не вручную):

1. Добавление задач в очередь
2. Поток извлекает задачу
3. Выполнение задачи
4. Возврат результата / завершение задачи

Подсказка: Задачи кладут в очередь, потоки берут, выполняют, возвращают.

🔧 Шаг 10:

Миссия 10

📖 Упорядочите этапы завершения и очистки потоков:

1. Вызвать join()
2. Проверить, что поток ещё жив
3. Ожидание завершения
4. Проверка результата выполнения (если возвращает) или состояния

Подсказка: join сначала, затем проверяем состояние и результат.

🔗 Шаг 11:

Миссия 11

🔗 Иван запускает несколько потоков, передавая методы с аргументами. Он хочет убедиться, что переданные объекты не изменятся до момента запуска. Какой паттерн стоит использовать?

Подсказка: Используйте замыкание или лямбду, чтобы зафиксировать параметры.

🔗 Шаг 12:

Миссия 12

Ольга хочет передать отправку задачи в новом потоке и сразу дождаться результата (результат функции). Какой подход правильный?

Подсказка: Стандартный Thread не возвращает результат — нужен другой механизм.

🔗 Шаг 13:

Миссия 13

🔗 Ваш коллега настроил поток, но не получил ошибок, хотя функция не выполняется корректно. Сопоставьте возможные причины и признаки:

Причины:

Ответ:

- target вызвана вместо передачи
- args неправильно указаны
- поток завершился слишком рано (daemon не нужен)
- исключение внутри потока

Признаки:

1. Функция не запускается вовсе
2. Аргументы не передаются, TypeError
3. Поток прерывается вместе с программой
4. Наблюдается SilentFail, не видно исключения

Подсказка: target() — вызов сразу в основном; args неверны — ошибка; исключения в потоке часто «глушатся».

🔗 Шаг 14:

Миссия 14

🔗 Упорядочите стадии запуска и завершения потока с учётом ошибок:

1. Создать Thread
2. Настроить target, args, daemon
3. Вызвать start()
4. Внутри target выполнить код, возможно с try/except
5. Вызвать join()
6. Обработать возвращённое или сохранённое состояние

Подсказка: Сначала конфигурация, затем запуск, затем ожидание и анализ результата.

Шаг 15:

Миссия 15

★ Финальная миссия: упорядочите этапы, если вы вручную создаёте N потоков и затем собираете все результаты:

1. Генерация списка задач / аргументов
2. Создание списка объектов Thread
3. Настройка target и аргументов
4. Запуск всех потоков
5. Вызов join для каждого потока
6. Сбор результатов из потоков / общих структур

Подсказка: Сначала создайте и настройте, потом запустите, потом ждите, потом собирайте.

Урок 3 - Демон-потоки и завершение программы

Шаг 01:

Миссия 1

Анна запустила демон-поток, который периодически обновляет лог-файл. Как сделать так, чтобы при завершении программы демон-поток не мешал?

Подсказка: Демон-поток не должен блокировать завершение программы.

Шаг 02:

Миссия 2

Борис хочет, чтобы главный поток дождался завершения всех рабочих потоков, но демон-потоки игнорировались. Что он должен сделать?

Подсказка: Только не-демон-потоки влияют на finish процесса.

Шаг 03:

Миссия 3

Катя запускает фоновый поток, который слушает порт, но при закрытии GUI-приложения поток продолжает висеть. Что надо сделать?

Подсказка: Поток может быть демоном или должен проверять флаг завершения.

Шаг 04:

Миссия 4

Павел хочет, чтобы фоновый поток автоматически завершался, когда программа закрывается. Что он устанавливает?

Подсказка: Демон-поток не препятствует выходу из программы.

Шаг 05:

Миссия 5

Катя создала поток-демон, но она боится, что важная операция не выполнится при завершении. Что лучше делать?

Подсказка: Демон-поток не гарантирует завершение задач — если нужно, используйте явное ожидание.

🔗 Шаг 06:

Миссия 6

🔍 Сопоставьте термины с описаниями, относящимися к демон-потокам и их завершению:

Подсказка: Некоторые особенности связаны с поведением ОС при завершении процесса.

Термины:

Ответ:

- Daemon thread
- Non-daemon thread
- thread.join()
- sys.exit()

Описания:

1. Ожидает завершения указанного потока
2. Поток, который не препятствует завершению процесса
3. Поток, который удерживает процесс живым до своего конца
4. Вызывает выход из текущего процесса

🔗 Шаг 07:

Миссия 7

🔍 Разработчик запустил поток в даемон-режиме, но часть задач не выполнена. Какие причины могут быть?

Подсказка: Демон-поток может быть прерван досрочно.

Причины:

Ответ:

- Поток завершился слишком рано
- Главный поток вышел до выполнения задачи
- Не вызван join на задачах
- Демон-флаг сброшен

Последствия:

1. Задача не выполнена полностью
2. Процесс завершился неожиданно
3. Задачи остались в очереди без исполнения
4. Поток превратился в обычный

🔗 Шаг 08:

Миссия 8

🔍 В сценарии фонового логирования используются термины. Сопоставьте:

Подсказка: Подумайте, какие потоки контролируют завершение процесса, а какие — нет.

Термины:

Ответ:

- background logger (демон)
- cleanup worker (не демон)
- main thread
- termination signal flag

Описания:

1. **Флаг, который сообщает потокам** — пора завершаться
2. Поток, отвечающий за финальную очистку после работы
3. Демон-поток записи логов в файл
4. Главный управляющий поток приложения

🔮 Шаг 09:

Миссия 10

📋 Упорядочьте шаги, которые нужно выполнить, чтобы корректно закрыть приложение с демонами и рабочими потоками:

1. установить флаг завершения для рабочих потоков
2. вызвать `join()` для рабочих потоков
3. запустить демон-потоки
4. осуществить shutdown логики и дождаться `cleanup` переносимое
5. запустить основные рабочие потоки

Подсказка: Демон-потоки должны быть запущены раньше завершения, но они не ждут `join`.

🔮 Шаг 10:

Миссия 11

📋 Упорядочьте сценарии завершения демонов и рабочих:

1. Главный поток ждёт `join` рабочих потоков
2. Начать демон-потоки (логирование, мониторинг)
3. Сигнал остановки распространяется среди потоков
4. Рабочие потоки завершают обработку текущих задач
5. Процесс закрывается, демоны автоматически выгружаются

Подсказка: Демон-потоки не ждут `join`, а просто выкидываются.

🔮 Шаг 11:

Миссия 12

📋 Упорядочите, когда лучше запускать разные типы потоков и завершения:

1. Запустить главный поток-контроллер
2. Запустить вспомогательные `non-daemon` потоки (очистка, завершающие задачи)
3. Запустить демон-потоки мониторинга
4. По сигналу установить флаги завершения
5. Дождаться `join()` `non-daemon` потоков
6. Завершить программу, демоны погибают автоматически

Подсказка: Демоны не ждут `join`, но работают параллельно с `non-daemon`.

🔮 Шаг 12:

Миссия 13

📋 Последовательность при обработке задач в фоновом режиме:

1. Получить задачу из очереди
2. Обработать её

3. Отправить в лог (демон-логгер)
4. Проверить флаг завершения
5. Если флаг, прекратить цикл

Подсказка: Нужно регулярно проверять желание завершиться.

Шаг 13:

Миссия 14

 Упорядочите этапы graceful shutdown (аккуратного завершения):

1. отключение приёма новых задач
2. отправка сигнала остановки (termination flag)
завершение текущих задач рабочими потоками
3. join() на non-daemon потоках
4. освобождение ресурсов (файлы, соединения)
5. процесс завершает работу (демоны убиваются автоматически)

Подсказка: Сначала запрещаем вход новой работы, потом ждём, потом закрываем ресурсы.

Шаг 14:

Миссия 15

 Упорядочите шаги в жизненном цикле приложения с demon-логгером и рабочими задачами:

1. инициализация данных и структур
2. запуск демон-логгера
3. запуск рабочих задач
4. приём сигнала завершения (Ctrl+C и т.п.)
5. остановка рабочих задач (через флаги)
6. join рабочих задач
7. закрытие логера, завершение программы

Подсказка: Демон-логгер запускается перед задачами, но завешается сам.

Шаг 15:

Миссия 16

 Упорядочите важные этапы при реализации завершения фоновых потоков:

1. установить признак остановки (flag) для потоков
2. демон-потоки продолжают работу, пока главный не завершился
3. в рабочих потоках проверять флаг в цикле
4. после сигнала завершения вызвать join на non-daemon потоках
5. освобождение ресурсов и выход из процесса

Подсказка: Демоны не ждут join и не должны препятствовать выходу.

Шаг 16:

Миссия 17

 Самый полный сценарий: упорядочите шаги приложения с демонами и завершающими потоками:

1. инициализация и конфигурация
2. запуск демон-фонов (мониторинг, логирование)

3. запуск non-daemon рабочих потоков
4. главный поток ожидает внешнего сигнала завершения
5. устанавливается флаг остановки для рабочих потоков
6. рабочие потоки завершают текущие задания
7. `join()` рабочих потоков
8. освобождение ресурсов и `clean up`
9. процесс завершается, демоны исчезают автоматически

Подсказка: Демоны умирают «на сдачу» после завершения процессов.

Урок 4 - Очереди `queue.Queue`

Шаг 01:

Миссия 1

Команда пишет чат-бота, и один из модулей должен обрабатывать входящие сообщения в порядке их поступления с помощью `queue.Queue`. Что из ниже перечисленного — правильное свойство этой очереди?

Подсказка: очередь — `first in, first out`.

Шаг 02:

Миссия 2

В сервере загружаются задачи от пользователей и ставятся в очередь `queue.Queue`. Какой метод используется, чтобы положить задачу в очередь?

Подсказка: вспомните методы `queue.Queue`: `put`, `get`.

Шаг 03:

Миссия 3

При обработке сообщений бот хочет взять задачу из очереди. Какой метод используется для извлечения элемента?

Подсказка: метод назван как «`get`».

Шаг 04:

Миссия 4

В многопоточной программе один поток может ждать, пока очередь не станет непустой. Какой метод блокирует поток до появления элемента?

Подсказка: метод `get` по умолчанию блокирует.

Шаг 05:

Миссия 5

В чат-боте стоит ограничение: очередь не должна расти безгранично. Какой параметр `queue.Queue` помогает задать ограничение переполнения?

Подсказка: конструктор принимает `maxsize`.

Шаг 06:

Миссия 6

Если очередь заполнена (достигла `maxsize`), и мы пытаемся вызвать `put()`, какое поведение по умолчанию?

Подсказка: put блокирует, если блок=True.

🔗 Шаг 07:

Миссия 7

✂ Свяжите методы queue.Queue с их назначением.

Методы:

Ответ:

- put(block=False)
- get(block=False)
- put(block=True)
- get(block=True)

Назначения:

1. Вернуть элемент, или выбросить исключение, если очередь пуста
2. Поместить элемент, или выбросить исключение, если очередь полна
3. Блокировать до тех пор, пока очередь не получит элемент
4. Блокировать до тех пор, пока не освободится место в очереди

🔗 Подсказка:

block=False означает неблокирующее поведение, True — ожидание.

🔗 Шаг 08:

Миссия 8

✂ В веб-приложении потоки используют очередь для логирования. Сопоставьте сценарии с возможным поведением методов.

Сценарии:

Ответ:

- Добавление записи, даже если очередь полна
- Попытка извлечь запись, но очередь пуста
- Ожидание, когда очередь пуста
- Ожидание, когда очередь полна

Поведение:

1. put(block=False) вызывает исключение queue.Full
2. get(block=False) вызывает исключение queue.Empty
3. **get(block=True)** — блокирует до появления элемента
4. **put(block=True)** — блокирует до освобождения места

🔗 Подсказка:

Для каждого сценария подумайте: хочется ли ждать или сразу получить ошибку?

🔗 Шаг 09:

Миссия 9

✂ Разработчики спорят: чем отличается очередь queue.Queue от простой списка. Сопоставьте характеристики:

Характеристики:

Ответ:

- Безопасность для потоков
- Управление блокировками
- Гарантированный порядок FIFO
- Автоматическое ожидание при переполнении / опустошении

Объяснения:

1. Использует внутренние блокировки, работа с потоками безопасна
2. Если очередь пуста или полна, методы могут блокировать
3. Элементы извлекаются в порядке добавления
4. Списку приходится самому реализовывать синхронизацию вручную

🔔 **Подсказка: list — не подходит для многопоточного доступа без своих блокировок.**

🔔 **Шаг 10:**

Миссия 10

✂ В стартапе реализуют очереди задач: сопоставьте события с методами:

События:

Ответ:

- Новая задача отправлена на исполнение сразу
- Новая задача отклоняется, если очередь полна
- Рабочий ждать, если нет задач
- Приложение ждать, если очередь заполнена

Методы поведения:

1. put(block=False)
2. reject (не использовать очередь)
3. get(block=True)
4. put(block=True)

🔔 **Подсказка: Подумайте: какое поведение соответствует «отказу»?**

🔔 **Шаг 11:**

Миссия 11

✂ При отладке системы логов программисты составляют таблицу: сопоставьте понятия с описаниями.

Понятия:

Ответ:

- queue.Empty
- queue.Full
- block=True
- block=False

Описания:

1. Исключение, сигнализирующее, что попытка взятия из пустой очереди неблокирующим методом
2. Исключение, сигнализирующее, что попытка положить в полную очередь неблокирующим методом

3. Позволяет методу ждать до готовности
4. Не ждёт и сразу возвращает ошибку, если нельзя выполнить операцию

🔗 **Подсказка:** `queue.Empty` и `queue.Full` — стандартные исключения модуля `queue`.

🔗 Шаг 12:

Миссия 12

✂ В микросервисе очередь используется для балансировки. Сопоставьте преимущества и обстоятельства:

Преимущества / обстоятельства:

Ответ:

- Асинхронная обработка
- Контроль загрузки
- Гибкость масштабирования
- Изоляция ошибок

Объяснения:

1. Задачи отправляются в очередь и обрабатываются позже
2. Если очередь заполнена, новые задачи не допускаются сразу
3. Новые воркеры могут читаться из одной очереди
4. Ошибка одного задания не ломает весь поток обработки

🔗 **Подсказка:** Подумайте: как очередь помогает распределить нагрузку и локализовать сбои.

🔗 Шаг 13:

Миссия 13

✂ Представьте систему обработки запросов: расположите этапы работы с очередью в правильном порядке:

1. Рабочий поток извлекает задачу
2. Пользователь отправляет запрос
3. Обработанная задача возвращается клиенту
4. Запрос помещается в очередь
5. Результат сформирован

🔗 **Подсказка:**

Сначала запрос приходит, затем ставится в очередь, потом извлекается на исполнение и возвращается клиенту.

🔗 Шаг 14:

Миссия 14

✂ В сценарии веб-скрейпинга запросы разбираются и ставятся в очередь, модули обработки извлекают, обрабатывают и записывают результат. Расположите стадии:

1. Обработать данные
2. Сохранить в базу
3. Получить задачу из очереди
4. Добавить URL в очередь
5. Парсинг HTML

Подсказка:

Сначала очередь наполняется URL, затем извлекаются, парсятся, обрабатываются и сохраняются.

Шаг 15:

Миссия 15

 В системе доставки сообщений: клиент → очередь → обработчик → отправка ответа. Расположите шаги:

1. Обработчик формирует ответ
2. Клиент отправляет сообщение
3. Ответ передаётся клиенту
4. Сообщение кладётся в очередь
5. Обработчик извлекает сообщение

Подсказка:

Всё происходит в строгом FIFO-порядке: сообщение сначала кладётся, затем извлекается, обрабатывается и отправляется обратно.

Шаг 16:

Миссия 16

 В архитектуре микрослуг очередь используется между сервисами А и В:

1. Сервис В обрабатывает задачу
2. Сервис А генерирует задачу
3. Сервис А кладёт задачу в очередь
4. Сервис В забирает задачу
5. Результат возвращается или логируется

Подсказка:

Последовательность: генерировать → ставить в очередь → забирать → обрабатывать → возвращать.

Шаг 17:

Миссия 17

 Представьте производство: заказы входят, ставятся в очередь, рабочие цеха обрабатывают, отгрузка. Расположите этапы:

1. Собрать готовый товар
2. Добавить заказ в очередь
3. Получить заказ из очереди
4. Произвести изделие
5. Отгрузка клиенту

Подсказка:

Сначала заказ поступает, затем ставится в очередь, дальше рабочий цех его извлекает, производит и отгружает.

Урок 5 - Race condition и блокировки (Lock, RLock)

Шаг 01:

Миссия 1

Ваша команда параллельно считает продажи в общем счётчике без блокировок — что может случиться?

Подсказка: Две нитки могут одновременно прочесть старое значение.

Шаг 02:

Миссия 2

✂ В коде используют `threading.Lock()` вокруг инкремента, но всё равно наблюдается неправильный результат. Почему?

Подсказка: Возможно, блокировка берётся не вокруг всего критического участка.

Шаг 03:

Миссия 3

🗄 Что делает `RLock` отличного от обычного `Lock`?

Подсказка: RLock позволяет одному потоку захватить блокировку рекурсивно.

Шаг 04:

Миссия 4

🔗 Вы используете `with lock:` вокруг части кода. Что важно, чтобы избежать deadlock?

Подсказка: Не захватывать два lock в разном порядке.

Шаг 05:

Миссия 5

🔍 Сопоставьте термин и его объяснение:

Термины:

Ответ:

- Race condition
- Critical section
- Lock
- Deadlock

Определения:

1. Ситуация, когда два потока ждут освобождения друг друга
2. Участок кода, доступ к которому необходимо сериализовать
3. Условие, когда результат зависит от порядка выполнения
4. Механизм синхронизации, обеспечивающий эксклюзивный доступ

Подсказка: Прочитайте определения и подумайте, что делает Lock и что такое гонка.

Шаг 06:

Миссия 6

🗄 Сопоставьте методы Lock/RLock и их поведение:

Методы:

Ответ:

- `acquire()`
- `release()`
- `acquire(blocking=False)`
- `acquire()` для `RLock`, когда уже захвачен

Поведение:

1. Освободить `lock`
2. Попытаться взять без ожидания и вернуть `False`, если неудача
3. **Уже внутри** — просто увеличить счётчик захватов
4. Ожидать, пока `lock` освободится, и захватить его

Подсказка: У `RLock` может быть счётчик захватов, а non-blocking режим возвращает булево.

 Шаг 07:

Миссия 7

 Сопоставьте ситуацию и нужный инструмент синхронизации:

Ситуации:

Ответ:

- Нужен простой эксклюзивный доступ к счётчику
- Нужно, чтобы один поток мог рекурсивно входить в блокировку
- Популярный ресурс читается часто, пишется редко
- Требуется проверить, доступен ли ресурс (без ожидания)

Инструменты:

1. `RLock`
2. `Lock`
3. `acquire(blocking=False)` + `Lock`
4. Read-Write Lock (условно, внешняя библиотека)

Подсказка: Подумайте, когда нужен reentrant захват и когда нужна попытка без блокировки.

 Шаг 08:

Миссия 8

 Сопоставьте общее название ошибки и её симптом:

Ошибка:

Ответ:

- Lost update
- Wrong interleaving
- Deadlock
- Priority inversion

Симптомы:

1. Оба потока ждут захвата, никто не освобождает
2. Обновление одного потока перезаписывает другое
3. Странные порядки операций, неожиданные результаты
4. Низкий приоритет потока держит `lock`, блокируя важный

Подсказка: Подумайте, что означает «lost update» и что бывает при deadlock.

Шаг 09:

Миссия 9

Расположите шаги защиты разделяемого ресурса с помощью Lock:

1. Записать новое значение
2. Освободить lock
3. Захватить lock
4. Считать текущее значение

Подсказка: Сначала ты должен взять блокировку, потом читать, потом писать, потом отпустить.

Шаг 10:

Миссия 10

 В каком порядке нужно вызывать захват двух lock, чтобы избежать deadlock?

1. free оба lock
2. acquire lock B
3. acquire lock A
4. acquire lock A затем lock B

Подсказка: Важно фиксировать единую последовательность захвата для всех потоков.

Шаг 11:

Миссия 11

 Расставьте по порядку события при deadlock:

1. Поток 2 ждёт lock, который захвачен Поток 1
2. Поток 1 ждёт lock, захваченный Поток 2
3. Оба потока захватили по одному lock
4. Никто не освобождает

Подсказка: Во-первых, оба захватывают один lock, потом ждут второй.

Шаг 12:

Миссия 12

 Расположите этапы использования RLock в рекурсивной функции:

1. Освободить lock (одно из захватов)
2. Захватить lock при входе в рекурсию
3. Выполнить тело функции
4. Вернуться из рекурсии и освободить

Подсказка: Каждая рекурсивная ступень должна захватывать и затем освобождать lock.

Урок 6 - Практика многопоточная загрузка файлов

Шаг 01:

Миссия 1

Разработчица Алиса запускает многопоточный загрузчик файлов. Какая цель стоит в основе разделения задач на потоки?

Подсказка: Подумайте, зачем мы хотим, чтобы разные потоки работали параллельно.

Шаг 02:

Миссия 2

Боб читает список URL-файлов и распределяет их по потокам. Какой способ распределения лучше всего подойдёт, если файлы имеют сильно разные размеры?

Подсказка: Если один поток получает слишком крупный файл — он тормозит остальных.

Шаг 03:

Миссия 3

Девушка-разработчик Катя хочет предотвратить ситуацию, когда два потока одновременно пытаются записать в один файл. Что она должна использовать?

Подсказка: В операциях с общей памятью нужен механизм блокировки.

Шаг 04:

Миссия 4

 При выполнении загрузок из сети возникла ситуация: одна загрузка блокируется, но остальные налаживаются. Какой прием можно применить, чтобы избежать «вечной» блокировки одного потока?

Подсказка: Подумайте об ограничении времени ожидания блокировки.

Шаг 05:

Миссия 5

Когда Алексей профилирует многопоточный загрузчик, что особенно важно измерить?

Подсказка: Подумайте о том, как эффективность меняется при увеличении числа потоков.

Шаг 06:

Миссия 6

 Сопоставьте сценарий с соответствующим риском многопоточной загрузки:

Сценарии:

Ответ:

- Поток записывает файл, пока другой читает то же имя файла
- Один поток ждёт блокировку слишком долго
- Потоки не получают равную нагрузку из-за разного размера файлов
- Слишком много потоков конкурируют за сеть и падает общая производительность

Риски:

1. Состояние гонки или повреждённые данные
2. Голодание потока / дедлок с ожиданием слишком долго
3. Низкая эффективность из-за неравномерной загрузки
4. Переполнение контекста, перегрузка ресурса

Подсказка:

Каждый сценарий — это либо конфликт доступа, либо дисбаланс, либо слишком много нагрузок.

🔗 Шаг 07:

Миссия 7

📖 Сопоставьте термин с правильным описанием в контексте многопоточных загрузок:

Термины:

Ответ:

- Lock
- Semaphore
- Deadlock
- Throughput

Определения:

1. Максимальное количество работ, выполняемых в единицу времени
2. Состояние, когда потоки навсегда ждут друг друга
3. Объект, дающий ограниченное число разрешений
4. Механизм, исключающий одновременный доступ к ресурсу

🔗 **Подсказка:**

Lock — обычно простая блокировка одного ресурса, Semaphore — множество разрешений.

🔗 Шаг 08:

Миссия 8

📖 Сопоставьте показатель с его типичным эффектом при наращивании числа потоков:

Показатели:

Ответ:

- Скорость загрузки (throughput)
- Процент простоя потоков
- Накладные расходы на контекстные переключения
- Утилизация сети

Типичные эффекты:

1. Растёт до некоторого значения, затем выравнивается
2. Может сильно возрасти при избыточном числе потоков
3. Уменьшается, если потоки простаивают
4. Достигает насыщения сети при большом числе запросов

🔗 **Подсказка: Подумайте, какие факторы начинают мешать после некоторого числа потоков.**

🔗 Шаг 09:

Миссия 9

📖 Сопоставьте подход к распределению задач с его преимуществом:

Подходы:

Ответ:

- Статическое распределение URL
- Динамическое распределение с очередью
- Пул фиксированного числа потоков

- Неограниченное создание потоков по требованию

Преимущества:

1. Простота реализации, но риск дисбаланса
2. **Гибкость** — потоки берут задачи по мере освобождения
3. Контроль над количеством одновременно работающих потоков
4. Максимальное параллелизм, но риск перегрузки

🔗 **Подсказка: Подумайте: что лучше — жёсткое разделение, управляемый пул или безграничные потоки?**

🔗 **Шаг 10:**

Миссия 10

🌐 Сопоставьте стратегию с её недостатком:

Стратегии:

Ответ:

- Каждый поток загружает весь файл частями
- Каждый поток загружает свой файл целиком
- Переиспользование потоков в пуле
- Кэширование уже загруженных частей файла

Недостатки:

1. Сложность координации частей файла
2. Возможны задержки при больших файлах в одном потоке
3. Могут простаивать потоки при нерегулярной нагрузке
4. Высокая память на хранение кэша

🔗 **Подсказка: Каждый подход даёт выгоду и приносит свою сложность — ищите соответствие.**

🔗 **Шаг 11:**

Миссия 11

📁 Расположите этапы многопоточной загрузки файла по порядку:

1. Инициализация потоков / пула
2. Добавление задач в очередь URL
3. Потоки берут задачи и загружают файлы
4. Обработка ошибок и повторные попытки
5. Запись загруженных данных в файловую систему
6. Ожидание завершения всех потоков

🔗 **Подсказка: Подумайте, что должно произойти до того, как потоки начнут работать, и что — после.**

🔗 **Шаг 12:**

Миссия 12

✂ Расположите фазы обработки скачанных кусочков файла:

1. Проверка контрольной суммы
2. Запись кусочка в общий файл

3. Запрос следующего кусочка
4. Обработка ошибок и повторная загрузка кусочка
5. Сборка окончательного файла из частей

💡 **Подсказка: Сначала проверка, потом запись, потом переход к следующему, и в конце сборка.**

💡 Шаг 13:

Миссия 13

📦 Расположите уровни оптимизации многопоточной загрузки по возрастанию сложности:

1. Увеличение числа потоков
2. Динамическое распределение задач
3. Использование семафоров ограничения
4. Кэширование уже загруженных частей
5. Распараллеливание внутри файла (частичное скачивание)

💡 **Подсказка: Начните с простого увеличения потоков, затем — более продвинутые стратегии.**

💡 Шаг 14:

Миссия 14

📦 Расположите задачи от высшего к низшему при проектировании загрузчика:

1. Обработка ошибок и повторные попытки при сбоях
2. Планирование и распределение задач потокам
3. Запись данных в диск
4. Проверка целостности скачанных данных
5. Загрузка данных по сети

💡 **Подсказка: Сначала идёт планирование, потом загрузка, проверка, запись, и обязательно обработка ошибок.**

💡 Шаг 15:

Миссия 15

🕒 Расположите шаги отладки производительности многопоточного загрузчика:

1. Сбор метрик (throughput, блокировки, простои)
2. Идентификация узких мест (lock contention, сеть, дисковый ввод-вывод)
3. Выбор стратегии оптимизации (семафоры, кэш, перераспределение задач)
4. Внедрение изменений и повторное профилирование
5. Сравнение результатов с базовой версией

💡 **Подсказка: Отладка — это цикл: измерить → найти узкое место → изменить → проверить.**

📖 Урок 7 - Квест оптимизируй потоковый код

💡 Шаг 01:

Миссия 1

Вы пишете многопоточный счётчик кликов на веб-сайте. Какое действие в потоке наиболее критично защитить от гонки?

Подсказка: Подумайте, какой ресурс одновременно изменяется разными потоками.

Шаг 02:

Миссия 2

Программист Катя создала поток, который должен обрабатывать задачи из очереди. Что нужно предпринять, чтобы избежать завешивания программы?

Подсказка: Подумайте, что делать, если поток остался без работы.

Шаг 03:

Миссия 3

Вася реализует потоковую загрузку файлов. Какой из подходов поможет ускорить загрузку при множестве мелких файлов?

Подсказка: Подумайте о накладных расходах на операцию ввода-вывода.

Шаг 04:

Миссия 4

Разработчик Олег добавил блокировку (lock) вокруг фрагмента работы, но скорость упала. Что наиболее вероятно неправильного?

Подсказка: Подумайте, что увеличивает время удержания блокировки.

Шаг 05:

Миссия 5

🔍 Сопоставьте тип гонки данных и её симптом:

Типы гонок:

Ответ:

- Lost update
- Read-write race
- Write-write race
- Atomicity violation

Симптомы:

1. Обновление затирает предыдущее изменение
2. Чтение старого значения в середине записи
3. Два потока пишут одновременно, итог непредсказуем
4. Несколько операций, которые должны быть атомарны, вмешиваются

Подсказка: Вспомните: lost update — обновление теряется, atomicity violation — части операции пересекаются.

Шаг 06:

Миссия 6

🔍 У вас есть модули синхронизации: lock, RLock, Condition, Semaphore. Сопоставьте с их назначением:

Модули:

Ответ:

- Lock

- RLock
- Condition
- Semaphore

Назначения:

1. Ожидание и уведомление потоков
2. Рекурсивная блокировка, может захватываться многократно одним потоком
3. Ограничить количество потоков, работающих одновременно
4. Простейший мьютекс

Подсказка: RLock — recursive lock, Condition — wait/notify, Semaphore — счётчик разрешений.

🔮 Шаг 07:

Миссия 7

📦 Представьте: очередь задач, буфер и потоки-потребители. Сопоставьте элемент с его ролью:

Элементы:

Ответ:

- Producer thread
- Consumer thread
- Task queue (buffer)
- Blocking queue operation

Роли:

1. Создаёт задачи и добавляет в очередь
2. Берёт задачу из очереди и обрабатывает её
3. Хранит задачи до обработки
4. Когда очередь пуста или полна, поток ждёт

Подсказка: Producer производит, Consumer потребляет, буфер хранит, blocking — ждет при переполнении/опустошении.

🔮 Шаг 08:

Миссия 8

🔗 Когда вы строите конвейер из нескольких стадий, часто используют паттерны. Сопоставьте паттерн с его описанием:

Паттерны:

Ответ:

- Fan-out / Fan-in
- Worker pool
- Pipeline stages
- Producer-consumer chain

Описание:

1. Выход нескольких задач из одного входа и объединение результатов
2. Набор «рабочих» потоков, которые берут задачи из очереди
3. **Стадии обработки по очереди: выход одной** — вход другой
4. Связь производитель → очередь → потребитель

Подсказка: Pipeline — линейная цепочка, worker pool — пул потоков, fan-out/in — развести и собрать.

🔗 Шаг 09:

Миссия 9

У вас есть методы синхронизации: barrier, event, lock, semaphore. Сопоставьте метод с примером использования:

Методы:

Ответ:

- Barrier
- Event
- Lock
- Semaphore

Примеры:

1. Синхронизация старта групп потоков одновременно
2. Ожидание сигнала от одного потока к другому
3. Гарантия, что критическая секция будет одна за раз
4. Ограничить доступ к ресурсу, допустим, лишь N потокам одновременно

Подсказка: barrier — точка синхронизации, event — сигнал, semaphore — счётчик доступа.

🔗 Шаг 10:

Миссия 10

Расположите шаги отладки гонки данных:

1. Написать тест, воспроизводящий гонку
2. Проанализировать стеки потоков
3. Добавить логирование до и после критической секции
4. Уточнить сценарии многократного повторения
5. Изолировать участок, вызывающий баг

Подсказка: Сначала надо воспроизвести, затем сузить, затем логировать и анализировать, и в конце смотреть стек.

🔗 Шаг 11:

Миссия 11

🔗 Машинное обучение: несколько потоков читают данные, затем один поток обновляет модель, потом шаг синхронизации. Расположите фазы:

1. Обновление модели
2. Чтение данных параллельно
3. Синхронизация перед новой итерацией
4. Распределение задач чтения по потокам

Подсказка: Чтение и распределение — до обновления, а синхронизация — после.

🔗 Шаг 12:

Миссия 12

✳ Потоки обрабатывают фрагменты массива, затем нужно собрать результаты. Расположите шаги:

1. Собрать результаты в общий массив
2. Запуск рабочих потоков
3. Деление массива на блоки
4. Ожидание завершения всех потоков

Подсказка: Сначала создаём блоки, запускаем потоки, ждём их завершения, затем собираем результаты.

🔗 Шаг 13:

Миссия 13

📖 Логика: считать файлы в параллельных потоках и затем записать итоговый отчёт. Расположите шаги:

1. Записать итоговый отчёт
2. Считывание файлов параллельно
3. Предварительная обработка каждого файла
4. Ожидание завершения всех чтений

Подсказка: Сначала чтение + обработка, затем синхронизировать, затем итог.

🔗 Шаг 14:

Миссия 14

🔗 У вас есть пулы потоков и задачи с зависимостями: сначала задача A, затем B и C, потом D. Расположите порядок запуска:

1. Запуск B и C
2. Запуск A
3. Запуск D
4. Ожидание зависимостей

Подсказка: Сначала A, потом B и C, потом ожидание, затем D.

🔗 Шаг 15:

Миссия 15

🎯 Финальная миссия: оптимизация. У вас есть профилировка: hot-spot в критической секции. Расположите шаги оптимизации:

1. Микрооптимизация внутри блокировки
2. Убрать как можно больше работы из блокировки
3. Замена блокировки на lock-free структуру
4. Измерить и сравнить производительность
5. Сохранить корректность кода

Подсказка: Сначала минимизируйте, потом экспериментируйте с альтернативами, но проверяйте результаты на каждом шаге.

Модуль 3 - Модуль 3. Многопроцессность сила нескольких ядер

Урок 1 - Модуль multiprocessing

Шаг 01:

Миссия 1

Иван запускает функцию через multiprocessing.Process, но она сразу завершает работу. Что могло быть причиной?

Подсказка: Важен порядок создания и старта процесса.

Шаг 02:

Миссия 2

Аня хочет передать аргументы в целевую функцию процесса. Как правильно это сделать?

Подсказка: Проверьте, какой параметр Process ожидает для аргументов.

Шаг 03:

Миссия 3

Борис хочет запустить пул процессов через multiprocessing.Pool, но получает ошибку при вызове map. Что мог пойти не так?

Подсказка: Убедитесь, что объект Pool создаётся под защитой конструкта проверки __main__.

Шаг 04:

Миссия 4

Сергей замечает, что память процесса растёт со временем. Что может быть причиной?

Подсказка: Проверьте, не остаются ли ссылки на большие объекты после работы процесса.

Шаг 05:

Миссия 5

 Сопоставьте метод multiprocessing с его описанием:

Методы:

Ответ:

- Process
- Pool.map
- apply_async
- Queue

Назначения:

1. Параллельное применение функции к элементам последовательности
2. Создание отдельного процесса с вызовом функции
3. Асинхронный запуск задачи с возможностью callback
4. **Межпроцессное общение** — передача данных между процессами

Подсказка: Подумайте, какой инструмент нужен для коммуникации между процессами.

🔧 Шаг 06:

Миссия 6

🔗 Сопоставьте объекты передачи данных multiprocessing с их назначениями:

Объекты:

Ответ:

- Pipe
- Manager().list()
- Value
- Array

Назначения:

1. Общий список между разными процессами
2. Однонаправленная или двунаправленная связь между двумя процессами
3. Поделиться одиночным скалярным значением (int, float)
4. Общий массив примитивов (int, c_double и т.д.) между процессами

Подсказка: Подумайте, где нужен список, где нужен массив, а где единичное значение.

🔧 Шаг 07:

Миссия 7

Сопоставьте стратегию управления пулом с её описанием:

Стратегии:

Ответ:

- maxtasksperchild
- initializer
- chunksize
- processes

Назначения:

1. Максимальное число задач, которое может выполнить один дочерний процесс перед перезапуском
2. Число процессов в пуле
3. Инициализирующая функция, выполняемая при старте каждого процесса
4. Размер блока задач, передаваемых одному процессу за раз

Подсказка: Подумайте, как уменьшить нагрузку на GC и уменьшить коммуникационные накладные расходы.

🔧 Шаг 08:

Миссия 8

У Петра есть очередь (Queue). Расположите шаги, чтобы использовать её между процессами:

1. Процесс-производитель кладёт данные в очередь (put)
2. Процесс-консьюмер извлекает их методом get()
3. Закрывает очередь (close) и дожидается завершения процессов
4. Создать очередь перед порождением процессов

Подсказка: Очередь должна быть создана перед стартом процессов.

🔧 Шаг 09:

Миссия 9

☞ Максимум хотите уменьшить накладные расходы: расположите действия для chunked map:

1. Разделить задачу на чанки (блоки данных)
2. Передать каждому процессу по одному чанку
3. Собрать результаты из всех процессов
4. Вернуть один плоский список результатов

Подсказка: Chunking уменьшает число коммуникаций между пулом и мастером.

🔧 Шаг 10:

Миссия 10

✂ Алгоритм работы с Manager().dict(): расположите этапы:

1. Создать Manager()
2. Взять dict() из менеджера
3. Запустить процессы, которые модифицируют dict
4. После join получить итоговый словарь

Подсказка: Объекты менеджера должны быть созданы до запуска процессов.

🔧 Шаг 11:

Миссия 11

✂ При завязывании процессов с Pool: расположите этапы их корректного завершения:

1. pool.close()
2. pool.join()
3. pool.terminate()
4. pool.map/apply_async ранее запущены задачи

Подсказка: terminate используется как аварийный выход.

🔧 Шаг 12:

Миссия 12

🔗 У вас есть Queue и процессы producer и consumer. Расположите все шаги от старта до остановки:

1. Создать очередь
2. Запустить процессы producer и consumer
3. producer кладёт в очередь, consumer извлекает
4. producer завершает работу и сигнализирует остановку (например, None)
5. consumer получает сигнал и завершает
6. join всех процессов

Подсказка: Нужен специальный маркер остановки.

🔧 Шаг 13:

Миссия 13

В распределённой задаче: расположите шаги обмена данными через Manager:

1. Создать Manager() до процессов
2. Получить совместные объекты (list, dict)
3. Запустить процессы, модифицирующие эти объекты
4. join процессов
5. Собрать итог из совместных объектов

Подсказка: Совместные объекты должны существовать до запуска процессов.

🔧 Шаг 14:

Миссия 14

🌐 При построении конвейера из трёх процессов ($A \rightarrow B \rightarrow C$): расположите этапы:

1. A пишет данные в Pipe/Queue
2. B читает, обрабатывает, пишет дальше
3. C читает и выдаёт результат
4. Все процессы join после завершения

Подсказка: Конвейер — это последовательная передача между процессами.

🔧 Шаг 15:

Миссия 15

🔧 Финальный квест: оптимизируйте параллельную задачу. Расположите этапы:

1. Замерить время последовательного выполнения
2. Разбить задачу на части
3. Запустить параллельно (Pool/Process)
4. Собрать и объединить результаты
5. Сравнить время и профилировать узкие места

Подсказка: Без замеров вы не узнаете, где узкое место.

📖 Урок 2 - Процессы и межпроцессное взаимодействие

🔧 Шаг 01:

Миссия 1

Алиса запускает скрипт, который создает два процесса для независимой обработки изображений. Какой метод из модуля multiprocessing она должна использовать, чтобы создать новый процесс?

Подсказка: подумайте, как в Python обычно создаётся процесс в multiprocessing.

🔧 Шаг 02:

Миссия 2

🔧 Боб хочет передать простое значение (целое число) в дочерний процесс. Какой из способов передачи подходит?

Подсказка: для простых типов данных подойдет передача через аргументы конструктора Process.

🔧 Шаг 03:

Миссия 3

✂ Катя хочет завершить дочерний процесс до продолжения главного кода (ждать его завершения). Какой метод ей нужен?

Подсказка: вспомните методы у объекта Process, связанные с ожиданием.

🔧 Шаг 04:

Миссия 4

📦 Денис хочет передать сложный объект (например, словарь) между процессами. Что ему больше подходит?

Подсказка: вспомните про специальные структуры обмена из multiprocessing.

🔧 Шаг 05:

Миссия 5

🔗 Илья запускает несколько процессов, но замечает, что они работают последовательно. Что он мог забыть сделать?

Подсказка: возможно, не был вызван метод, который разрешает параллельную работу.

🔧 Шаг 06:

Миссия 6

🗣 Сопоставьте термин и описание в контексте multiprocessing:

Термины:

Ответ:

- Shared memory
- Queue
- Pipe
- Manager

Описание:

1. Объект, управляющий общими данными между процессами через прокси.
2. Подход, при котором несколько процессов читают и записывают одну область памяти.
3. Однонаправленный или двунаправленный канал между двумя процессами.
4. FIFO-очередь объектов между процессами.

Подсказка: вспомните, что используют чаще всего для безопасного обмена данными.

🔧 Шаг 07:

Миссия 7

🔍 Сопоставьте проблему и её причина в многопроцессных приложениях:

Проблемы:

Ответ:

- Deadlock
- Race condition
- Starvation
- Data corruption

Причины:

1. Два процесса ожидают друг друга, заблокировав ресурсы.
2. Один процесс постоянно получает ресурсы, другие ждут.
3. Несинхронизированный доступ к разделяемой памяти.
4. Отсутствие защиты при одновременном доступе.

Подсказка: подумайте, какая ситуация приводит к «гонке» и какая — к блокировке.

🔮 Шаг 08:

Миссия 8

⚙ Сопоставьте функцию модуля multiprocessing и её назначение:

Функции:

Ответ:

- apply_async
- map
- close
- terminate

Назначения:

1. Остановить пул немедленно.
2. Применить функцию ко всем элементам итератора параллельно с возвратом списка.
3. Закрыть пул, запрещая добавление новых задач.
4. Запустить задачу асинхронно, возвращая объект результата.

Подсказка: подумайте, какие методы используются у Pool для управления задачами.

🔮 Шаг 09:

Миссия 9

🔍 Сопоставьте ситуацию и подход к синхронизации:

Ситуации:

Ответ:

- Изменение одного счётчика разными процессами
- Чтение и запись списка данных параллельно
- Необходимость блокировки только части данных
- Передача команд между процессами

Подходы:

1. Lock
2. RLock / условные блокировки
3. Queue
4. Shared memory + atomic операции / Value

Подсказка: подумайте, когда достаточно обычного Lock, а когда нужны очереди или память.

🔮 Шаг 10:

Миссия 10

Иван описал последовательность настроек пула процессов. Расположите шаги в правильном порядке:

1. Выполнить задачи в пуле
2. Закрыть пул для добавления новых задач
3. Создать объект Pool
4. Ждать завершения всех задач
5. **При необходимости** — завершить форсированно

Подсказка: вспомните, как обычно управляют пулом: создание → добавление задач → закрытие → ожидание → завершение.

Шаг 11:

Миссия 11

🌐 Соня описывает шаги безопасного обновления разделяемого состояния с использованием Lock. Расставьте шаги:

1. Выполнить критическую секцию (изменить данные)
2. Освободить lock
3. Запросить lock
4. Проверить и подготовить данные перед изменением
5. Освободить lock в finally-блоке

Подсказка: важно никогда не забывать освобождать lock даже при ошибке.

Шаг 12:

Миссия 12

🌐 Павел реализует передачу через Pipe между двумя процессами. Упорядочите шаги:

1. Закрыть лишние концы pipe в каждом процессе
2. Создать два конца pipe: parent_conn, child_conn
3. Отправить или получить сообщение через conn.send()/recv()
4. Передать child_conn дочернему процессу
5. В главном процессе использовать parent_conn

Подсказка: подумайте, как установить канал и разделить концы между процессами.

Шаг 13:

Миссия 13

🔗 Алексей хочет контролировать завершение дочерних задач и возможную отмену через pool. Упорядочите шаги:

1. pool.terminate(), если требуется форсированно остановить
2. pool.close()
3. pool.join()
4. отправить задачи (map/apply_async)
5. создать пул процессов

Подсказка: сначала создаём пул, потом выдаём задачи, затем закрываем и ждём, а при необходимости — terminate.

Шаг 14:

Миссия 14

 Олег хочет агрегировать результаты множества дочерних процессов через Queue. Упорядочите действия:

1. Читать все элементы из очереди в главном процессе
2. Процессы помещают результаты через `queue.put()`
3. Создать очередь `multiprocessing.Queue()`
4. Передать очередь дочерним процессам как аргумент
5. Ожидать завершения дочерних процессов

Подсказка: сначала создаём очередь, передаём её, процессы кладут в неё данные, ждём завершения, затем читаем.

Шаг 15:

Миссия 15

 Лена реализует стратегию, где главная программа создаёт процессы и ждёт их завершения, а затем обрабатывает общий результат. Упорядочите:

1. Объединить (`merge`) результаты дочерних процессов
2. Запустить дочерние процессы (`start`)
3. Создать общую структуру через `Manager / Queue / Pipe`
4. Присоединиться к дочерним (`join`)
5. Создать дочерние процессы (`Process` или через `Pool`)

Подсказка: следуйте жизненному циклу: подготовка — запуск — ожидание — сбор результата.

Шаг 16:

Миссия 16

Финальная миссия: Алиса создает сложную систему обработки: сборщик, парсер и аналитик работают в трёх процессах и обмениваются данными. Упорядочите общий жизненный цикл:

1. Собрать все результаты в финальном процессе
2. Передать объекты обмена (`Queues, Pipes, Manager`) дочерним процессам
3. Инициализировать дочерние процессы
4. Запустить дочерние процессы
5. Ожидать завершения дочерних через `join/очереди`
6. Завершить дочерние форсированно, если потребуется
7. Разорвать/закрыть объекты обмена

Подсказка: подумайте о полном цикле создания, запуска, синхронизации, завершения и очистки ресурсов.

Урок 3 - Очереди, каналы и Manager

Шаг 01:

Миссия 1

Ваша служба поддержки хочет распределять задачи между процессами при помощи очереди `Manager()`. Какой тип объекта является подходящим для передачи между процессами?

Подсказка: Менеджер создаёт объекты, которые безопасны для обмена между процессами.

🔮 Шаг 02:

Миссия 2

✂ Вы строите систему логирования: процессы пишут записи в очередь, родитель читает. Что лучше использовать?

Подсказка: В multiprocessing есть специализированная очередь, адаптированная для процессов.

🔮 Шаг 03:

Миссия 3

⚙ У вас есть Manager().dict() как разделяемый словарь между процессами. Что он гарантирует?

Подсказка: Он управляется серверным процессом, обеспечивая синхронизацию.

🔮 Шаг 04:

Миссия 4

🔗 Сопоставьте объект и его предназначение в контексте multiprocessing.Manager:

Объект:

Ответ:

- Manager().Queue()
- Manager().dict()
- Manager().list()
- Manager().Namespace()

Назначение:

1. Общий словарь между процессами
2. Общий список между процессами
3. Пространство имён с атрибутами
4. Очередь сообщений между процессами

Подсказка: Подумайте, какой контейнер служит для последовательностей, какой для ключей и значений, а какой позволяет обмениваться сообщениями.

🔮 Шаг 05:

Миссия 5

📦 Сопоставьте канал коммуникации и его свойства:

Канал:

Ответ:

- multiprocessing.Pipe()
- Manager().Queue()
- multiprocessing.Queue()

Свойство:

1. Двухсторонняя связь между двумя процессами
2. Может обслуживать несколько отправителей и получателей
3. Узкоспециализированная очередь для обмена данными между процессами

Подсказка: Pipe обычно связывает два конца, Queue — более универсальна.

🔗 Шаг 06:

Миссия 6

Борис хочет, чтобы несколько процессов писали в один список с безопасностью. Что лучше всего использовать?

Подсказка: Помните про Manager в multiprocessing.

🔗 Шаг 07:

Миссия 7

Катя хочет передавать сразу несколько сообщений туда и обратно между процессами. Что ей подойдёт?

Подсказка: Есть вариант с «каналом» (Pipe) в multiprocessing.

🔗 Шаг 08:

Миссия 8

Дмитрий хочет, чтобы каждый дочерний процесс имел доступ к общему счётчику. Какой объект использовать?

Подсказка: Смотрим в Manager и его примитивы.

🔗 Шаг 09:

Миссия 9

Саша хочет предотвратить переполнение очереди при отправке. Что он может сделать?

Подсказка: В Queue можно задать максимальный размер или проверить блокировку.

🔗 Шаг 10:

Миссия 10

Лена использует Pipe, но один конец не читает — что может быть проблемой?

Подсказка: Pipe — это канал с двумя концами: надо их правильно подключить.

🔗 Шаг 11:

Миссия 11

Сопоставьте объекты multiprocessing и их предназначение:

Объекты:

Ответ:

- Queue
- Pipe
- Manager().dict()
- Manager().Value

Назначения:

1. Общий словарь между процессами
2. Один-на-один канал обмена
3. Безопасная очередь сообщений
4. Общий атомарный примитив (число, флаг)

Подсказка: Подумайте, для чего нужен каждый механизм обмена и хранения.

🔗 Шаг 12:

Миссия 12

Сопоставьте термин и описание:

Термины:

Ответ:

- manager proxy object
- put()
- get()
- maxsize

Описание:

1. Ограничивает число элементов в Queue
2. Записать элемент в очередь
3. Возвратить элемент из очереди
4. Представитель объекта в Manager для доступа из процессов

Подсказка: Подумайте, как проху-объекты ведут себя и какие операции у очереди.

🔗 Шаг 13:

Миссия 13

Сопоставьте ситуацию с возможным решением:

Ситуации:

Ответ:

- Процессы блокируются на ожидании сообщения
- Очередь переполнена
- Не удаётся обновить общий счётчик
- Канал закрыт преждевременно

Решения:

1. Использовать блокирующий вызов с таймаутом
2. Увеличить maxsize или обрабатывать быстрее
3. Использовать Manager().Value или Lock
4. Проверить, что оба конца Pipe открыты

Подсказка: Подумайте, какая ошибка вызовет блокировку или переполнение, и какое решение логично.

🔗 Шаг 14:

Миссия 14

Сопоставьте методы очереди и их эффекты:

Методы:

Ответ:

- put_nowait()
- get_nowait()
- qsize()
- empty()

Эффекты:

1. Вернуть число элементов в очереди
2. **Поместить элемент, не ожидая, если нельзя** — бросить исключение
3. Проверить, пуста ли очередь
4. **Извлечь элемент без ожидания, если нет** — исключение

Подсказка: путайте методы с `_nowait`, которые не блокируются.

Шаг 15:

Миссия 15

Сопоставьте этапы работы Manager:

Этапы:

Ответ:

- Запуск Manager()
- Создание управляемого объекта
- Использование объекта в процессах
- Остановка Manager()

Описания:

1. Создать проху-объект (list, dict, Value и т.п.)
Запустить сервер-демон Manager Передать проху объект в дочерние процессы Завершить работу Manager и закрыть процессы

Подсказка: Подумайте, что происходит от старта до завершения.

Шаг 16:

Миссия 16

Сопоставьте тип ошибки и возможную причина:

Типы ошибок:

Ответ:

- BrokenPipeError
- Queue.Full
- Queue.Empty
- AttributeError (у proxy)

Причины:

1. Пытаемся читать из пустой очереди с `get_nowait()`
Канал закрылся преждевременно Очередь переполнена при `put_nowait()` Обращение к несуществующему атрибуту проху-объекта

Подсказка: Подумайте, какая операция вызывает какую ошибку.

Шаг 17:

Миссия 17

Расположите шаги, которые нужно предпринять, чтобы передать задачу в очередь и обработать её:

1. Дочерний процесс читает задачу из очереди

2. Основной процесс кладёт задачу в очередь через `put()`
3. Обработать задачу и вернуть результат
4. Основной процесс получает результат через очередь или `проху`

Подсказка: Подумайте — сначала кладём, потом читаем, потом обрабатываем, потом возвращаем результат.

Шаг 18:

Миссия 18

Устроить этапы инициализации `Manager` и разделяемых данных:

1. Запустить `Manager()`
2. Создать `проху`-объекты (`list`, `Value`, `dict` и др.)
3. Передать `проху`-объекты в дочерние процессы
4. Закрыть `Manager` после выполнения
5. Ждать завершения всех дочерних процессов

Подсказка: Следует сначала стартовать, затем создавать объекты, потом передавать, потом завершать.

Шаг 19:

Миссия 19

Упорядочите шаги при обмене через `Pipe`:

1. Процесс А вызывает `send()`
2. Процесс В вызывает `recv()`
3. Закрыть концы `Pipe` после обмена
4. Инициализировать `Pipe` до запуска процессов

Подсказка: Сначала создаём `Pipe`, потом отправка/приём, потом закрываем.

Урок 4 - Пул процессов (Pool)

Шаг 01:

Миссия 1

Алиса хочет распараллелить функцию, применяющуюся к каждому элементу списка. Что из ниже перечисленного отражает суть работы пула процессов (`Pool`)?

Подсказка: Пул распределяет задачи по ядрам автоматически.

Шаг 02:

Миссия 2

✂ Бобу надо выбрать метод от пула: `map` или `apply_async`. В каком случае подойдёт `map`?

Подсказка: `map` возвращает результаты в том же порядке.

Шаг 03:

Миссия 3

🔗 Когда Алиса использует `Pool` с числом процессов больше числа ядер, что произойдёт?

Подсказка: Изучите, что такое контекст переключения процессов.

💡 Шаг 04:

Миссия 4

🔗 Вспомните методы Pool: сопоставьте метод с его описанием.

Методы:

Ответ:

- map
- starmap
- apply
- apply_async

Описание:

1. Вызывает функцию с одним аргументом для каждого элемента итерации
2. Как map, но позволяет распаковывать кортежи аргументов
3. Синхронно вызывает функцию с аргументом
4. Асинхронно вызывает функцию и возвращает результат через объект AsyncResult

Подсказка: Обратите внимание на синхронность и асинхронность.

💡 Шаг 05:

Миссия 5

📖 Сопоставьте термин и его последствие при неправильном использовании пула процессов:

Термины:

Ответ:

- Deadlock
- Zombie process
- High context switch overhead
- Resource exhaustion

Последствия:

1. Избыточные накладные расходы переключения
2. Недостаток системных ресурсов, ошибки «Too many processes»
3. Процесс завершён, но PID остаётся до родителя
4. Взаимная блокировка процессов

Подсказка: Подумайте, какое последствие характерно для каждого термина.

💡 Шаг 06:

Миссия 6

Сопоставьте ситуации с тем, какой механизм Pool поможет:

Ситуации:

Ответ:

- Есть список чисел, и функция отдельно обрабатывает один аргумент
- Функция принимает два аргумента из списка кортежей
- Нужно запустить одну функцию один раз
- Нужно запустить функцию много раз, но собирать результаты по мере готовности

Механизмы:

1. map
2. starmap
3. apply
4. apply_async

Подсказка: Смотрите на количество аргументов и на синхронность.

Шаг 07:

Миссия 7

Мария хочет создать пул и запустить серию задач. Расположите шаги от начала к концу:

1. pool.close()
2. pool = Pool(processes=N)
3. pool.join()
4. подать задачи (map, apply и т.д.)
5. создать функцию задач

Подсказка: Сначала надо определиться с функцией, потом создать пул, передать задачи, закрыть и ждать.

Шаг 08:

Миссия 8

🔧 Нужно аккуратно завершить пул после работы. Упорядочите шаги:

1. pool.terminate()
2. pool = Pool()
3. отменить дальнейшие задачи
4. pool.join()

Подсказка: terminate прерывает сразу, join ждёт завершения.

Шаг 09:

Миссия 9

🔗 При использовании apply_async Алиса получает AsyncResult. Расположите последовательность действий:

1. результат.get()
2. pool.apply_async(...)
3. pool.close()
4. pool.join()

Подсказка: Сначала запускаем, потом закрываем пул, затем ждём, а достаём результат позже.

Шаг 10:

Миссия 10

🤖 Денис задумался: можно ли использовать Pool внутри уже запущенного процесса, который сам в пуле? Что произойдёт?

Подсказка: Некоторые реализации запрещают вложенные пулы.

🔧 Шаг 11:

Миссия 11

📖 Илона измерила ускорение: при 4 ядрах ускорение оказалось почти 3х. Что это говорит?

Подсказка: Линейное ускорение это идеал.

🔧 Шаг 12:

Миссия 12

🔍 Сопоставьте причины медленной работы параллельного кода через Pool с возможными узкими местами:

Причины:

Ответ:

- Большой объём передачи данных между процессами
- Функции блокируют GIL (если вызывают C-код)
- Небольшие задачи (много мелких вызовов)
- Неоптимальное число процессов

Узкие места:

1. издержки сериализации / десериализации
2. вызов внешних библиотек, удерживающих GIL
3. накладные расходы на переключения
4. недозагрузка/перегрузка CPU

Подсказка: Подумайте о том, что именно тормозит параллельность: коммуникации, GIL, scheduling, конфигурации.

🔧 Шаг 13:

Миссия 13

🧠 Представьте, что Боб измеряет время: сопоставьте тип задачи с тем, какое распределение задач по процессам будет эффективным:

Типы задач:

Ответ:

- Однородные независимые задачи
- Задачи разной длительности
- Очень маленькие задачи
- Некоторые задачи блокирующие

Рекомендации:

1. pool.map или imap без очередей
2. динамическое распределение (apply_async с очередью)
3. использовать более крупные батчи задач
4. ограничить число процессов или избегать Pool

Подсказка: Для задач разной длительности лучше динамика, для блокирующих — ограничение.

🔧 Шаг 14:

Миссия 14

♡ При реализации обработки файлов с пулом, Ирина строит логику: открыть файл, обработать, закрыть файл, сохранить результат. Упорядочите шаги с учётом многопроцессной обработки:

1. закрыть файл
2. pool = Pool()
3. открыть файл / чтение данных
4. применить вычисления через пул
5. сохранить результат
6. pool.close() / join()

Подсказка: Чтение и подготовка данных — прежде, чем подавать их в пул. Затем — закрытие пула и сохранение.

🔮 Шаг 15:

Миссия 15

🧑 Павел разделяет задачи в батчи: чтение, разбивка, запуск пула, сбор всех результатов, финальная агрегация. Упорядочите:

1. финальная агрегация результатов
2. старт пула
3. разбивка на батчи
4. сбор результатов из пулов
5. чтение исходных данных

Подсказка: Данные читаются, затем разбиваются, затем пул работает, затем собирают и агрегируют.

🔮 Шаг 16:

Миссия 16

🧑 В проекте Анна использует пул и хочет корректно завершить программу в случае исключения. Расположите шаги:

1. pool.terminate()
2. перехват исключения
3. pool.join()
4. создать пул и запустить задачи

Подсказка: При исключении нужно прервать пул, потом ждать завершения.

🔮 Шаг 17:

Миссия 17

🧑 У Лены две версии запуска: sync-map и async-apply. Её алгоритм:

1. инициализация пула
2. запуск задач
3. закрыть пул
4. дождаться завершения
5. обработка результатов (get или возврат)

Упорядочите шаги корректно:

Подсказка: Результаты обязательно обрабатываются после join или get.

Урок 5 - Практика параллельные вычисления

Шаг 01:

Миссия 1

Алекс запускает функцию «сложный расчёт» в однопоточном режиме, но работает это слишком медленно. Что логично попробовать первым?

Подсказка: Иногда просто распараллеливание уже помогает.

Шаг 02:

Миссия 2

 При создании пула процессов, что нужно передавать в target функции?

Подсказка: Функция запускается в новом процессе, её аргументы надо явно передать.

Шаг 03:

Миссия 3

 При каком условии многопроцессность полезна для Python-кода?

Подсказка: Подумайте о “узком месте” — ЦПУ или ввод-вывод?

Шаг 04:

Миссия 4

 Таня распараллеливает задачу: создала пул процессов, но всё равно видит, что используется только одно ядро. Что могло быть не так?

Подсказка: Возможно, функция внутри пула не запускается, а просто возвращается объект.

Шаг 05:

Миссия 5

При объединении результатов из дочерних процессов, что удобнее использовать?

Подсказка: Что может безопасно передать результаты между процессами?

Шаг 06:

Миссия 6

 Сопоставьте понятие с описанием в контексте мультипроцессинга:

Ответ:

- Процесс
 - Пул процессов
 - IPC (межпроцессное взаимодействие)
 - Жизненный цикл процесса
1. Набор рабочих процессов для распределения задач.
 2. Средства обмена данными между процессами.
 3. Запущенная копия интерпретатора с памятью.
 4. От старта до завершения работы процесса.

Подсказка: Попробуйте представить, как процессы создаются, обмениваются данными и завершаются.

🔗 Шаг 07:

Миссия 7

🔍 Сопоставьте термин и пример использования:

Ответ:

- `apply_async`
- `map`
- `starmap`
- `close + join`

1. Запустить все задачи, потом ждать завершения.
2. Применить функцию к каждому элементу списка.
3. **Аргументы** — кортежи, распаковывать в функцию.
4. Отправить задачи асинхронно, получать результат позже.

Подсказка: Подумайте, какое поведение даёт каждое — синхронное, асинхронное, с распаковкой.

🔗 Шаг 08:

Миссия 8

⚙ Сопоставьте проблему и способ её решения при многопроцессности:

Ответ:

- Гонка данных
- Мёртвые блокировки (deadlock)
- Избыточная сериализация
- Неправильный сбор обратных результатов

1. Использовать очереди или объекты с блокировками.
2. Не закрывать пул или забыть `join`.
3. Не использовать `shared` память, а передавать копии.
4. Внутри дочернего процесса забыть возврат результата.

Подсказка: Каждая проблема проявляется в конкретных ошибках: отсутствие синхронизации, забытый `join`, передача сложных объектов.

🔗 Шаг 09:

Миссия 9

📋 Упорядочьте этапы запуска многопроцессной задачи:

1. Ожидание завершения (`join`)
2. Создание пула процессов
3. Передача задач (`map` / `apply_async`)
4. Закрытие пула к работе (`close`)

Подсказка: Сначала создавать, затем передавать задачи, затем закрывать, и в конце ждать.

🔗 Шаг 10:

Миссия 10

🔧 Упорядочите шаги отладки параллельной программы:

1. Добавление логов / печать из процессов

2. Изолировать минимальный сценарий, воспроизводящий баг
3. Проверить правильность join / close
4. Тестировать с небольшим пулом (2–4 процесса)

Подсказка: Сперва минимальный пример, затем логирование, затем проверка этапов запуска/завершения.

🔗 Шаг 11:

Миссия 11

📦 Упорядочите компоненты большого параллельного приложения:

1. Модуль агрегации результатов
2. Пул процессов или воркеры
3. Функция-обработчик задачи
4. Модуль распределения задач (scheduler)

Подсказка: Сначала scheduler назначает, потом воркеры запускают функции, потом агрегатор собирает.

🔗 Шаг 12:

Миссия 12

Упорядочите сценарии от наименее до наиболее сложного в параллельной обработке:

1. Запуск одной функции много раз с разными параметрами
2. Агрегация результатов нескольких независимых задач
3. Взаимодействие между задачами (обмен данными)
4. Распределённые вычисления с зависимостями

Подсказка: Чем больше зависимости между задачами, тем сложнее распараллеливать.

🔗 Шаг 13:

Миссия 13

🌐 В облачной среде (на многоядерной машине) сколько процессов стоит запускать?

Подсказка: Не забывайте об оверхедах и гиперпоточности.

🔗 Шаг 14:

Миссия 14

✂ Иван хочет сохранить прогресс задач в файл, но разные процессы пытаются писать одновременно — что выбрать?

Подсказка: Нужно синхронизация записи.

🔗 Шаг 15:

Миссия 15

🔗 Сопоставьте тип IPC и подходящий пример:

Ответ:

- Queue
- Pipe
- Manager().list()

- SharedMemory / Value / Array

1. Передача простых сообщений между процессами.
2. Общий список в памяти, управляемый менеджером.
3. Прямой канал между двумя процессами.
4. Общая разделяемая память примитивов.

Подсказка: Queue и Pipe — каналы сообщений, Manager — управляемые объекты, SharedMemory — низкоуровневый.

Шаг 16:

Миссия 16

Сопоставьте стратегию и её эффект на производительность:

Ответ:

- Слишком много процессов
 - Идеальное число процессов
 - Недостаточно процессов
 - Неверное распределение задач
1. Оверхед, конкуренция за ресурсы, снижение скорости.
 2. Баланс между загрузкой и расходами на переключение.
 3. Не используется весь потенциал CPU.
 4. Некоторым процессам придётся ждать или простаивать.

Подсказка: Представьте график «производительность vs число процессов».

Шаг 17:

Миссия 17

🗖 Упорядочите полный жизненный цикл распределённой задачи в параллельной программе:

1. Обработка результата и агрегация
2. Запуск процесса / worker
3. Подготовка и распределение задач
4. Ожидание завершения (join)
5. Закрытие пула (close)

Подсказка: С чего начинается — распределение, потом запуск, закрытие, ожидание, сбор результатов.

Урок 6 - Квест создаём ускоренный обработчик изображений

Шаг 01:

Миссия 1

Ваша команда – разработчики-энтузиасты, и вы решили ускорить обработку фотографий, распараллелив работу. Какой подход даст выигрыш производительности на многоядерной машине?

Подсказка: Подумайте: если работа независима, её можно выполнять раздельно.

Шаг 02:

Миссия 2

Вы уже решили разбить изображение на квадраты и распределить их между процессами. Что делать первым шагом при проектировании?

Подсказка: С чего стоит начать любое разбиение задачи?

🔗 Шаг 03:

Миссия 3

Вы поделили изображение на N блоков и хотите запустить N процессов, но наблюдаете, что один процесс жрёт много времени, остальные – простаивают. Какую проблему вы, скорее всего, допустили?

Подсказка: Какой класс ошибок возникает при неравномерной нагрузке?

🔗 Шаг 04:

Миссия 4

Чтобы избежать узкого места при записи обработанных блоков, вы решили использовать очередь. Какой механизм IPC (межпроцессного взаимодействия) подойдёт?

Подсказка: В Python multiprocessing есть готовые очереди.

🔗 Шаг 05:

Миссия 5

Задача: сопоставьте термины с их ролями при параллельной обработке изображений.

Термины:

Ответ:

- Процесс-воркер
- Менеджер (главный процесс)
- Блок изображения
- IPC очередь

Роли:

1. Хранит и распределяет задания воркерам
2. Фрагмент исходного изображения, который обрабатывается отдельно
3. Поток/процесс, выполняющий обработку блока пикселей
4. Канал передачи данных между процессами

Подсказка: Вспомните классические архитектуры «менеджер — воркер».

🔗 Шаг 06:

Миссия 6

Сопоставьте проблемы с их следствиями в параллельной обработке изображений:

Проблемы:

Ответ:

- Синхронизация блоков
- Накладные расходы на IPC
- Конфликты доступа к данным
- Зависание процесса

Следствия:

1. Производительность падает из-за ожидания доступа

2. Процесс останавливается и не отвечает
3. Избыточная коммуникация и задержки
4. Аварийные состояния или ошибка при записи данных

Подсказка: Представьте каждую проблему и подумайте, что она может «привести».

Шаг 07:

Миссия 7

Сопоставьте этапы pipeline при обработке изображений с назначениями:

Этапы:

Ответ:

- Чтение блока
- Обработка блока (фильтры, коррекции)
- Отправка результата менеджеру
- Сборка финального изображения

Назначения:

1. Осуществляет конечную сборку из обработанных фрагментов
2. Применяет алгоритмы к блоку
3. Считывает пиксели из файла или памяти
4. Передаёт обратно обработанный блок

Подсказка: Представьте себе конвейер обработки — что делает каждый шаг?

Шаг 08:

Миссия 8

Сопоставьте стратегии распределения задач с их особенностями:

Стратегии:

Ответ:

- Статическое разбиение
- Динамическое планирование задач (work stealing)
- Порционная очередь заданий
- Случайное распределение

Особенности:

1. Процессы крадут работу друг у друга при простое
2. Задачи разбиваются заранее и не меняются в ходе выполнения
3. Задания хранятся в центральной очереди; воркеры берут по мере готовности
4. Может привести к неравномерной загрузке по случайности

Подсказка: Подумайте, какая стратегия наиболее гибкая при разной длительности блоков.

Шаг 09:

Миссия 9

Расположите шаги обработки одного блока изображения в правильном порядке:

1. Запросить блок у менеджера
2. Применить фильтр/функцию к блоку

3. Отправить результат обратно менеджеру
4. Получить подтверждение, что блок принят

Подсказка: Как происходит взаимодействие до и после обработки?

Шаг 10:

Миссия 10

Расположите этапы запуска параллельной обработки всего изображения:

1. Главный процесс разбивает изображение на блоки
2. Запуск воркеров с назначением им очередей
3. Сбор результатов от воркеров
4. Сборка финального полного изображения
5. Передача блоков воркерам по запросу

Подсказка: Убедитесь, что каждый этап логически следует из предыдущего.

Шаг 11:

Миссия 11

Расположите виды ускорения в порядке сложности реализации (от простого к сложному):

1. Симплификация/оптимизация кода
2. Многопроцессная обработка
3. Использование GPU/векторизации
4. Профилирование и оптимизация узких мест

Подсказка: Что проще применить сразу без глубокого анализа?

Шаг 12:

Миссия 12

Расположите стадии отладки параллельной системы:

1. Запустить локально на одном процессе
2. Добавить логирование внутри воркеров
3. Запустить с малым числом процессов
4. Проанализировать время ожиданий и узкие места
5. Увеличить число процессов до целевого

Подсказка: Сначала минимизируйте точки отказа, затем масштабируйте.

Шаг 13:

Миссия 13

Расположите шаги профилирования параллельной обработки:

1. Собрать статистику времени на каждом этапе
2. Определить узкие места (bottlenecks)
3. Предложить альтернативные алгоритмы или перераспределение
4. Повторить эксперимент с изменениями
5. Оценить выигрыш производительности

Подсказка: Профилирование — циклический процесс анализа и улучшения.

Шаг 14:

Миссия 14

Вы протестировали на 4 ядрах и получили ускорение $\times 2$. Расположите возможные причины недостаточного ускорения в порядке вероятности:

1. Накладные расходы на IPC слишком велики
2. Некоторые блоки слишком небольшие, и затраты перевешивают работу
3. Оценка сложности блоков не сбалансирована
4. Система ограничена скоростью диска или памяти
5. Процессы ожидают синхронизации слишком часто

Подсказка: Подумайте сначала о узких местах в коммуникации и дисковых операциях.

Шаг 15:

Миссия 15

Перед вами список подходов к параллельной обработке. Расположите их по возрастанию степени контроля, который вы получаете:

1. `multiprocessing.Pool.map`
2. Запуск явных процессов с `Queue`
3. `work stealing` вручную (с динамическим перераспределением)
4. Писать собственный планировщик и IPC вручную

Подсказка: Чем меньше абстракции — тем больше контроля (и ответственности).

Шаг 16:

Миссия 16

Прикидываем рабочий план: вот что вы собираетесь делать — упорядочьте шаги от старта до завершения:

1. Запустить профилировщик и собрать базовую метрику
2. Разбить изображение и запустить воркеры
3. Проанализировать лог и время ожиданий
4. Сбалансировать планирование задач и перераспределить
5. Запустить итоговый эксперимент и сравнить ускорение

Подсказка: Направление: измерить → реализовать → улучшить → измерить заново.

Шаг 17:

Миссия 17

Финальная миссия: вам предстоит оформить инструкцию пользователю, чтобы минимизировать ошибки использования. Расположите пункты инструкции в логическом порядке:

1. Объяснить, что изображение разбивается на блоки
2. Указать, как выбрать параметр количества процессов
3. Проинструктировать по запуску обработки через команду
4. Пояснить возможные ошибки при запуске (IPC, синхронизация)
5. Объяснить, как собрать итоговый файл из блоков

Подсказка: Думайте как пользователь: сначала концепции, затем команды, затем ошибки, затем итог.

Модуль 4 - Модуль 4. Асинхронность время не ждёт

Урок 1 - Что такое event loop

Шаг 01:

Миссия 1

Анна разрабатывает сервер на asyncio и хочет понять, что такое event loop. Что из ниже перечисленного — правильное определение?

Подсказка: Подумайте, кто управляет задачами, когда вы пишете async/await.

Шаг 02:

Миссия 2

▲ Борис запустил несколько корутин, но не видит, что они выполняются. Какое действие запускает event loop?

Подсказка: Подумайте, какую функцию вы вызываете, чтобы «запустить» асинхронность.

Шаг 03:

Миссия 3

🔊 Катя строит веб-сервер и хочет, чтобы он мог обслуживать множество соединений. Что event loop делает для этого?

Подсказка: Подумайте, как одна нить может обслуживать множество задач.

Шаг 04:

Миссия 4

🔗 Сопоставьте термин с его ролью в контексте event loop.

Термины:

Ответ:

- Task
- Coroutine
- Callback
- Future

Роли:

1. Объект, который представляет запланированную корутину и её результат.
2. Функция, выполненная синхронно при наступлении события.
3. Асинхронная функция, которую можно приостановить и возобновить.
4. Обёртка над coroutine, управляемая loop.

Подсказка: Task порождается из coroutine, Future — ожидание результата.

Шаг 05:

Миссия 5

📅 Сопоставьте события с тем, как event loop на них реагирует.

События:

Ответ:

- Ожидание I/O

- Когда корутина делает await
- Когда Future завершилась
- Когда нужно выполнить callback после таймера

Рутины:

1. Возобновление coroutine
2. Отправка задачи в очередь ready
3. Блокировка до I/O завершения
4. Выполнение callback через call_later

Подсказка: Когда Future готов, loop ставит задачу обратно в цикл.

🔮 Шаг 06:

Миссия 6

Сопоставьте методы event loop с их назначением:

Методы:

Ответ:

- run_forever
- call_soon
- run_until_complete
- stop

Назначения:

1. Останавливает loop после завершения текущей итерации.
2. Запускает loop до завершения переданной задачи.
3. Запускает цикл бесконечно.
4. Планирует callback на следующий виток цикла.

Подсказка: Обратите внимание: run_until_complete — мост синхронного и асинхронного.

🔮 Шаг 07:

Миссия 7

🔗 Сопоставьте объекты и роли в реальном сценарии:

Дана ситуация: сервер ждёт входящие соединения, обрабатывает базу данных, запускает таймеры.

Объекты:

Ответ:

- Selector / Epoll
- Task
- Callback (таймер)
- Future (результат DB-запроса)

Роли:

1. Следит за файловыми дескрипторами и готовностью I/O
2. Запланированное действие по истечении времени
3. Асинхронный запрос, который ждёт результата
4. Рабочая единица, возобновляемая корутина

Подсказка: Selector — низкоуровневый механизм; Task — корутина, обернутая loop.

🔗 Шаг 08:

Миссия 8

🕒 У Сергея есть схема работы event loop. Упорядочите шаги, как loop обрабатывает корутины и I/O.

1. Запуск callback, чья задержка истекла
2. Проверка дескрипторов на готовность I/O
3. Возобновление корутины, готовой к выполнению
4. Ожидание готовности I/O или таймера

Подсказка: Loop чередует стадии ожидания и исполнения.

🔗 Шаг 09:

Миссия 9

Упорядочите стадии, которые проходят корутина при await:

1. Снятие корутины с выполнения и передача управления loop
2. Приостановка до завершения awaitable
3. **При завершении** — возврат управления корутине
4. Возобновление её исполнения

Подсказка: await — пауза, затем возобновление.

🔗 Шаг 10:

Миссия 10

📦 Упорядочите этапы превращения coroutine → Task → выполнение:

1. Создание coroutine (например, async def)
2. Передача coroutine в loop.create_task или ensure_future
3. Запуск task через loop
4. Получение результата или исключения

Подсказка: Task — это обёртка над coroutine, которую запускает loop.

🔗 Шаг 11:

Миссия 11

🔗 Алекс заметил, что корутина не начала выполняться, несмотря на использование `async def`. Что он мог забыть?

Подсказка: Просто объявить корутину — недостаточно. Её нужно передать в loop.

🔗 Шаг 12:

Миссия 12

✂ Юлия запускает `loop.run\_forever()`, но её программа не завершалась, пока она не вызвала `loop.stop()`. Почему?

Подсказка: run_forever делает именно то, что говорит его имя. Чтобы остановить, нужен явный сигнал.

🔔 Шаг 13:

Миссия 13

🕒 Что произойдёт, если в coroutine использовать `time.sleep(5)` вместо `await asyncio.sleep(5)`?

Подсказка: Синхронные задержки блокируют поток, асинхронные — нет.

🔔 Шаг 14:

Миссия 14

Сопоставьте типы ожиданий в `asyncio` и как они реализуются:

Ожидания:

Ответ:

- Задержка времени
- Ожидание результата из `Future`
- Ожидание I/O операции
- Ожидание завершения `Task`

Механизмы:

1. `await task`
2. `await future`
3. `await asyncio.sleep()`
4. `await reader.read()`

Подсказка: Все формы ожидания — это `await` чего-то, но каждое — своё.

🔔 Шаг 15:

Миссия 15

🔗 Сопоставьте действия разработчика и реакцию `event loop`:

Действия:

Ответ:

- `asyncio.create_task(my_coro())`
- `loop.call_soon(callback)`
- `await asyncio.sleep(2)`
- `loop.stop()`

Реакции:

1. Запланировать выполнение `callback` на следующем тике
2. Создание и регистрация `Task` в `loop`
3. Приостановка coroutine до истечения времени
4. Остановка цикла после текущей итерации

Подсказка: Все действия взаимодействуют с `loop`, но в разное время.

🔔 Шаг 16:

Миссия 16

🖱 Упорядочите действия, когда пользователь нажимает кнопку в GUI-программе на `asyncio`:

1. Пользователь вызывает действие (например, клик мышью)
2. `loop` фиксирует событие в очереди

3. Вызывается зарегистрированный callback
4. Callback создаёт coroutine и передаёт её в create_task

Подсказка: Сначала событие, потом callback, потом coroutine.

Урок 2 - `asyncawait` и корутины

Шаг 01:

Миссия 1

Вы запускаете простую асинхронную функцию с `await`. Что делает оператор `await` ?

Подсказка: Думайте, что `await` “ждёт” завершения корутины.

Шаг 02:

Миссия 2

Вы вызываете обычную функцию без `async` внутри асинхронного кода. Что произойдёт?

Подсказка: Короче говоря, обычная функция вернёт результат сразу.

Шаг 03:

Миссия 3

Что делает `async def` при объявлении функции?

Подсказка: Подумайте, что возвращает такая функция при вызове.

Шаг 04:

Миссия 4

У вас есть две корутины A и B, и вы пишете `await A`; `await B` . Что означает такой подход?

Подсказка: Они выполняются последовательно.

Шаг 05:

Миссия 5

Как запустить две корутины параллельно и дождаться обеих?

Подсказка: Смотрите в сторону функций библиотеки `asyncio`.

Шаг 06:

Миссия 6

Вы пишете `await asyncio.sleep(1)` . Как это влияет на выполнение?

Подсказка: Эта пауза не блокирует весь цикл событий.

Шаг 07:

Миссия 7

 Сопоставьте конструкции с их ролью (`async def`, `await`, `asyncio.gather`, `asyncio.create_task`):

Ответ:

- `async def`
- `await`
- `asyncio.gather`

- `asyncio.create_task`

1. Запускает корутину в фоне сразу
2. Определяет функцию как `coroutine`
3. Ожидает завершения нескольких корутин
4. Приостанавливает выполнение до результата

Подсказка: Обратите внимание, что `create_task` даёт задачу, `gather` ждёт множество.

🔮 Шаг 08:

Миссия 8

🔗 Сопоставьте стадии исполнения корутины (`suspend`, `resume`, `done`, `schedule`):

Ответ:

- `suspend`
 - `resume`
 - `done`
 - `schedule`
1. Когда `coroutine` приостановлена (например, через `await`)
 2. Когда возобновляется её выполнение
 3. Когда завершилась и вернула результат
 4. Когда она добавлена в очередь на исполнение

Подсказка: Подумайте об этапах: поставить в очередь → ждать → возобновлять → завершить.

🔮 Шаг 09:

Миссия 9

🔗 Сопоставьте функции `asyncio` с их применением (`wait`, `sleep`, `as_completed`, `gather`):

Ответ:

- `wait`
 - `sleep`
 - `as_completed`
 - `gather`
1. Возвращает итератор, выдающий задачи по мере завершения
 2. Ждёт завершения набора задач с опциями (`FIRST_COMPLETED` и др.)
 3. Приостанавливает корутину на время задержки
 4. Запускает все и ждёт завершения всех

Подсказка: `as_completed` выдаёт по одному, `gather` ждёт все.

🔮 Шаг 10:

Миссия 10

🔗 Сопоставьте сценарии с реакцией `event loop`:

Ответ:

- корутина вызвала `await` на блокирующую I/O
- корутина завершилась
- создана новая задача через `create_task`

- корутина вызвала `await asyncio.sleep`

1. переходит в состояние `suspend` и уступает управление
2. убирается из очереди задач
3. добавляется в очередь задач на исполнение
4. возобновление позже, после задержки

Подсказка: Рассмотрите, что делает `loop`, когда корутина ждёт I/O или сна.

Шаг 11:

Миссия 11

📋 Упорядочьте этапы запуска асинхронной программы:

1. `event loop` запускает задачи
2. объявление `async def` функций
3. выполнение корутин через `gather` или `await`
4. закрытие `event loop`

Подсказка: Сначала объявляем, затем запускаем, затем завершаем.

Шаг 12:

Миссия 12

📋 Упорядочите шаги обработки нескольких задач с `gather`:

1. Создание списка корутин
2. Передача списка в `gather`
3. `await gather` на список
4. Получение результатов всех корутин

Подсказка: `gather` сначала запускает, затем ждёт, затем возвращает данные.

Шаг 13:

Миссия 13

📋 Упорядочите обработку ошибок в корутинах с `try/except`:

1. `except` ловит исключение внутри корутины
2. код внутри `try` выполняется
3. внутри `except` можно логировать или подавить
4. при исключении управление передаётся в `except`

Подсказка: `try` → код → исключение → `except` logic.

Шаг 14:

Миссия 14

📋 Упорядочите шаги, когда корутина ждёт внешней I/O:

1. корутина вызывает `await` на I/O
2. переключение на другую задачу
3. I/O завершает работу
4. возобновление корутины

Подсказка: I/O действует вне `loop`: `await` → переключение → завершение → возвращение.

💡 Шаг 15:

Миссия 15

📋 Упорядочите этапы при комбинировании `gather` и `create_task`:

1. создать задачи через `create_task`
2. поместить эти задачи в список
3. `await gather` на задачи
4. обработать полученные результаты

Подсказка: Сначала создаём таски, затем передаём в `gather`, ждём, обрабатываем.

💡 Шаг 16:

Миссия 16

📋 Упорядочите полную последовательность от объявления до завершения асинхронной программы:

1. определение `async def` функций
2. создание задач или вызов `gather/await`
3. запуск `event loop`
4. выполнение корутин
5. обработка результатов и ошибок
6. закрытие `loop`

Подсказка: Сначала объявляем, запускаем `loop`, исполняем, обрабатываем, потом закрываем.

📖 Урок 3 - Функции `gather` и `as_completed`

💡 Шаг 01:

Миссия 1

Вас пригласили протестировать пару асинхронных задач: какие функции Python запускают сразу несколько корутин и возвращают результат в виде списка?

Подсказка: Представьте, что вы хотите запустить три HTTP-запроса и дождаться всех ответов одновременно.

💡 Шаг 02:

Миссия 2

🔗 Представьте, что Пётр запускает три корутины: А, В и С. Соответствующие поведения:

Ответ:

- А возвращает результат через 1 с
- В возвращает через 3 с
- С возвращает через 2 с

Задача: сопоставьте режим запуска с порядком получения результатов.

Режимы:

- `gather`
- `as_completed`
- `wait`

Порядок:

1. результат A, затем C, затем B
2. все вместе, в порядке A, B, C
3. ожидание всех, без гарантии порядка

Подсказка:

`gather` ждёт всех и возвращает в исходном порядке, `as_completed` возвращает по мере готовности.

Шаг 03:

Миссия 3

 Расположите шаги, которые сделает программа, если использовать `asyncio.as_completed` для пяти задач:

1. ожидание первой завершившейся корутины
2. получение результата этой корутины
3. запуск остальных корутин (если ещё не запущены)
4. повторять до тех пор, пока не будут получены все результаты
5. завершение функции, возврат всех результатов

Подсказка:

Подумайте: `as_completed` выдаёт корутины по мере завершения, не дожидаясь всех сразу.

Шаг 04:

Миссия 4

Вас пригласили протестировать пару асинхронных задач: какие функции Python запускают сразу несколько корутин и возвращают результат в виде списка?

Подсказка: Представьте, что вы хотите запустить три HTTP-запроса и дождаться всех ответов одновременно.

Шаг 05:

Миссия 5

 Аня пишет Telegram-бота и использует `gather`. Что произойдёт, если одна из задач внутри `asyncio.gather()` завершится с ошибкой?

Подсказка: Что делает Python, если одна из задач не выполнена успешно?

Шаг 06:

Миссия 6

 Какой вариант использования `asyncio.as_completed()` верный?

Подсказка: Этот метод возвращает итератор, а не список задач.

Шаг 07:

Миссия 7

 Тимур решил ускорить загрузку картинок с серверов. Какой подход быстрее — `gather` или последовательный `await`?

Подсказка: Одновременные запросы — ключ к высокой скорости.

🔮 Шаг 08:

Миссия 8

Какие результаты вернёт `gather` при успешном завершении всех задач?

Подсказка: Он агрегирует ответы от всех задач, а не по одному.

🔮 Шаг 09:

Миссия 9

📦 Какой подход выбрать, если важно получать результаты по мере их готовности?

Подсказка: Представьте, что вы не хотите ждать самой медленной задачи.

🔮 Шаг 10:

Миссия 10

⚙️ Сопоставьте метод с его ключевым поведением:

Методы:

Ответ:

- `asyncio.gather`
- `asyncio.as_completed`
- `await`
- `time.sleep`

Поведения:

1. Выполняет все задачи одновременно и возвращает список результатов
2. Итерирует результаты по мере готовности
3. Блокирует выполнение, не асинхронна
4. Ожидает одну задачу

Подсказка: Асинхронность — про неблокирующие операции и многозадачность!

🔮 Шаг 11:

Миссия 11

Катя использует разные подходы к обработке задач. Сопоставьте метод с ситуацией, где он подходит лучше всего:

Методы:

Ответ:

- `asyncio.gather`
- `asyncio.as_completed`
- `concurrent.futures`
- `await` в цикле

Ситуации:

1. Вы хотите запускать много задач и обрабатывать их по мере завершения
2. Запускать задачи и ждать их результата строго в порядке создания
3. Асинхронно ожидать все задачи, чтобы получить результат одним списком
4. Использовать многопоточность для синхронных функций

Подсказка: Подумай: нужно ли тебе параллельность, последовательность или потоки?

🔧 Шаг 12:

Миссия 12

🔍 Что произойдёт в каждой ситуации? Сопоставьте вызов с его эффектом.

Вызовы:

Ответ:

- `await asyncio.gather(*tasks)`
- `asyncio.as_completed(tasks)`
- `await asyncio.wait(tasks)`
- `await asyncio.gather(*tasks, return_exceptions=True)`

Поведение:

1. Получаем список результатов, включая исключения
2. Итерация по завершённым задачам
3. Ожидание завершения всех задач, без возврата результатов
4. **Получаем результаты всех задач (если ошибки — возникает исключение)**

Подсказка: Поведение зависит от того, обрабатываются ли ошибки и что именно возвращается.

🔧 Шаг 13:

Миссия 13

У Паши пять задач: A, B, C, D, E. Установите соответствие между методом и порядком, в котором он получит результаты.

Методы:

Ответ:

- `asyncio.gather`
- `asyncio.as_completed`
- `await + for`
- `threading.Thread`

Результаты:

1. **Порядок A, B, C, D, E** — независимо от времени выполнения
2. Порядок может быть любым, по мере завершения задач
3. Ждёт каждую задачу последовательно
4. Запускает задачи в отдельных потоках

Подсказка: Все эти методы работают по-разному — одни асинхронные, другие многопоточные.

📖 Урок 4 - Практика асинхронный парсер сайтов

🔧 Шаг 01:

Миссия 1

Молодой разработчик Антон решил написать асинхронный парсер для новостных сайтов. С чего ему стоит начать?

Подсказка: Вспомни: асинхронность требует специального цикла событий и ключевых слов Python.

🔗 Шаг 02:

Миссия 2

🌟 Антон замечает, что его парсер выполняется слишком медленно. Он добавил `await asyncio.sleep(2)` после каждого запроса. Почему это может быть ошибкой?

Подсказка: Иногда ожидание между запросами полезно, но не всегда разумно вставлять искусственные задержки.

🔗 Шаг 03:

Миссия 3

🔗 В проекте Антон использует библиотеку `aiohttp`. Какой синтаксис корректно открывает клиентскую сессию?

Подсказка: Асинхронные контекстные менеджеры — ключ к управлению ресурсами.

🔗 Шаг 04:

Миссия 4

🌐 Антон хочет параллельно загрузить несколько страниц. Какой метод `asyncio` ему поможет?

Подсказка: Вспомни про корутины, которые запускаются одновременно и ждут завершения всех задач.

🔗 Шаг 05:

Миссия 5

🕒 Какое преимущество даёт использование асинхронного парсера перед синхронным при большом количестве сайтов?

Подсказка: Подумай, что именно экономится — процессорное время или время ожидания сети?

🔗 Шаг 06:

Миссия 6

Сопоставьте элементы асинхронного кода с их назначением.

Подсказка: Асинхронность — это про управление задачами и событийным циклом.

🔗 Шаг 07:

Миссия 7

📦 Сопоставьте понятие с его примером в контексте асинхронного парсера.

Подсказка: Пример должен помогать вспомнить назначение понятия.

🔗 Шаг 08:

Миссия 8

🗨 Антон столкнулся с типичными ошибками при работе с асинхронным кодом. Сопоставьте сообщение об ошибке и её причину.

Подсказка: Внимательно посмотри на ключевые слова ошибок — они подсказывают контекст.

🔗 Шаг 09:

Миссия 9

🌐 Разработчик хочет ограничить количество одновременных запросов. Сопоставьте инструмент и его назначение.

Подсказка: Контроль числа одновременных задач — важная оптимизация.

🔔 Шаг 10:

Миссия 10

🔗 Разработчик применяет асинхронность для API-запросов. Сопоставьте библиотеку и её назначение.

Подсказка: Подумай, какая библиотека за что отвечает.

🔔 Шаг 11:

Миссия 11

🔗 Антон хочет логировать ошибки при парсинге. Сопоставьте способ и результат.

Подсказка: Логирование помогает отладить асинхронные цепочки.

🔔 Шаг 12:

Миссия 12

Расположите шаги создания асинхронного парсера в правильном порядке.

Подсказка: Сначала проектная структура, потом функции, потом запуск.

🔔 Шаг 13:

Миссия 13

🔗 Расположите шаги асинхронного запроса через aiohttp.

Подсказка: Вспомни, как открывается сессия и где происходит получение данных.

🔔 Шаг 14:

Миссия 14

🔗 Упорядочите этапы оптимизации асинхронного парсера.

Подсказка: Оптимизация — это цикл улучшений после тестов.

🔔 Шаг 15:

Миссия 15

Расположите шаги при работе с несколькими сайтами через asyncio.gather.

Подсказка: Сначала задачи, потом ожидание, потом результат.

🔔 Шаг 16:

Миссия 16

📖 Упорядочите шаги сохранения результатов парсинга в асинхронную базу.

Подсказка: Сначала подключение, потом запись, потом закрытие.

🔔 Шаг 17:

Миссия 17

🔗 Расположите шаги для отладки асинхронного приложения.

Подсказка: Отладка — это анализ ошибок, логирование и повторный запуск.

Урок 5 - Квест напиши код, который не «ждёт»

Шаг 01:

Миссия 1

Разработчик Илья пишет Telegram-бота, который должен отвечать мгновенно. Что ему нужно использовать, чтобы бот не блокировался при каждом запросе?

Подсказка: Асинхронность позволяет выполнять несколько задач, не дожидаясь завершения предыдущих.

Шаг 02:

Миссия 2

⚡ Когда Илья запускает асинхронную функцию, он забывает поставить `await`. Что произойдет?

Подсказка: Без `await` корутина не выполняется — лишь создается объект, который ждёт запуска.

Шаг 03:

Миссия 3

⬇ Илья решает измерить скорость своего асинхронного кода. Что покажет время выполнения, если все запросы к сайтам выполняются одновременно?

Подсказка: Подумай: если всё выполняется параллельно, общее время примерно равно самому долгому запросу.

Шаг 04:

Миссия 4

🌀 Илья читает про `asyncio.gather()`. Что делает эта функция?

Подсказка: Она позволяет запускать несколько корутин одновременно и ждать их завершения.

Шаг 05:

Миссия 5

🔗 Когда стоит использовать `async with aiohttp.ClientSession()`?

Подсказка: Асинхронный контекстный менеджер нужен, чтобы грамотно закрывать соединения после работы.

Шаг 06:

Миссия 6

Сопоставьте элементы асинхронного кода с их назначением в Python.

Подсказка: Каждый инструмент `asyncio` выполняет свою конкретную задачу.

Шаг 07:

Миссия 7

⚙ Илья дебажит свой код. Сопоставьте тип ошибки с её вероятной причиной.

Подсказка: Ошибки `asyncio` часто говорят о неправильном контексте запуска или неверном использовании `await`.

🔗 Шаг 08:

Миссия 8

📌 Сопоставьте библиотеку с её назначением в асинхронных программах.

Подсказка: Каждая библиотека расширяет `asyncio` в своей области — сеть, файлы, базы данных.

🔗 Шаг 09:

Миссия 9

🔗 Илья добавляет ограничения на число одновременных запросов. Сопоставьте инструмент и действие.

Подсказка: Семафоры — отличный способ регулировать количество активных задач.

🔗 Шаг 10:

Миссия 10

📌 Сопоставьте понятие асинхронности с его реальным примером.

Подсказка: Асинхронность похожа на многозадачность в реальной жизни.

🔗 Шаг 11:

Миссия 11

🔗 Илья хочет обрабатывать ошибки асинхронно. Сопоставьте конструкцию и её цель.

Подсказка: Исключения в асинхронных задачах тоже нужно ловить правильно.

🔗 Шаг 12:

Миссия 12

Расположите шаги написания простого асинхронного скрипта.

Подсказка: Вспомни порядок: определяем функцию, вызываем внутри `event loop`.

🔗 Шаг 13:

Миссия 13

📌 Расположите шаги создания асинхронного HTTP-запроса через `aiohttp`.

Подсказка: Важно закрывать сессию после использования.

🔗 Шаг 14:

Миссия 14

🔗 Расположите шаги параллельной загрузки страниц с помощью `asyncio.gather`.

Подсказка: Сначала создаются корутины, потом объединяются.

🔗 Шаг 15:

Миссия 15

Расположите шаги при работе с семафором для ограничения запросов.

Подсказка: Используйте контекстный менеджер для корректного освобождения.

🔗 Шаг 16:

Миссия 16

📌 Упорядочите действия при обработке ошибок в асинхронном коде.

Подсказка: Ошибки ловятся в try/except, чтобы не падал весь event loop.

🔔 Шаг 17:

Миссия 17

🔧 Расположите шаги запуска асинхронного приложения целиком.

Подсказка: Начни с написания корутин и закончи запуском главного цикла.

📖 Модуль 5 - Модуль 5. concurrent.futures Python-способ параллелить

📖 Урок 1 - ThreadPoolExecutor и ProcessPoolExecutor

🔔 Шаг 01:

Миссия 1

💻 Программист Аня заметила, что её скрипт парсинга сайтов работает слишком медленно. Она решила попробовать ThreadPoolExecutor. Какова основная цель его использования?

Подсказка: Подумай, что помогает ускорить работу с большим количеством сетевых запросов.

🔔 Шаг 02:

Миссия 2

Разработчик Дима запустил CPU-интенсивные вычисления через ThreadPoolExecutor, но ускорения не получил. Почему?

Подсказка: Вспомни, что в Python есть GIL, а потоки делят один интерпретатор.

🔔 Шаг 03:

Миссия 3

⚙️ Алексей пишет парсер картинок и хочет обрабатывать каждую ссылку в отдельном потоке. Как он может отправить задачу в ThreadPoolExecutor?

Подсказка: Вспомни метод, который ставит задачу в очередь для выполнения.

🔔 Шаг 04:

Миссия 4

👤 Тимур решил использовать ProcessPoolExecutor для вычисления числа π методом Монте-Карло. Почему это хорошая идея?

Подсказка: Подумай, как процессы обходят ограничения GIL.

🔔 Шаг 05:

Миссия 5

👤 Разработчица София создала пул из 5 потоков и подала туда 100 задач. Что произойдет?

Подсказка: Потоки не создаются для каждой задачи — они переиспользуются.

🔔 Шаг 06:

Миссия 6

Когда стоит предпочесть ThreadPoolExecutor вместо ProcessPoolExecutor?

Подсказка: Вспомни, какие задачи связаны с I/O (ввод-вывод) и не упираются в CPU.

🔧 Шаг 07:

Миссия 7

Сопоставьте методы `ThreadPoolExecutor` и их описание.

Подсказка: Один метод ставит задачу, другой ждёт завершения всех, третий возвращает результаты итератором.

🔧 Шаг 08:

Миссия 8

⚙ Сопоставьте элементы архитектуры `concurrent.futures` с их назначением.

Подсказка: Один объект управляет, другой представляет задачу, третий возвращает результат.

🔧 Шаг 09:

Миссия 9

Сопоставьте термины с их значением в контексте `Future`.

Подсказка: Вспомни, что `future` можно проверять, ожидать и получать результат.

🔧 Шаг 10:

Миссия 10

⚡ Сопоставьте ошибки с их причиной при работе с пулом исполнителей.

Подсказка: Иногда пул уже закрыт, а иногда задача не может быть сериализована.

🔧 Шаг 11:

Миссия 11

Расположите шаги работы с `ThreadPoolExecutor` в правильном порядке.

Подсказка: Сначала создаём пул, потом добавляем задачи, ждём завершения, закрываем.

🔧 Шаг 12:

Миссия 12

⚙ Расположите шаги работы программы, использующей `ProcessPoolExecutor` для параллельных вычислений.

Подсказка: Последовательность похожа, но добавь получение результатов.

🔧 Шаг 13:

Миссия 13

🕒 Расположите этапы анализа эффективности между потоками и процессами.

Подсказка: Сначала тестируй простое, потом усложняй.

🔧 Шаг 14:

Миссия 14

🔧 Расположите этапы оптимизации программы с использованием `concurrent.futures`.

Подсказка: Сначала определяем проблему, потом применяем оптимизацию, потом проверяем.

🔧 Шаг 15:

Миссия 15

❏ Расположите действия программиста, когда программа зависает при работе с `ThreadPoolExecutor`.

Подсказка: Важно проверить, что передаются только сериализуемые объекты и пул корректно закрывается.

📖 Урок 2 - Future и метод `result()`

🔮 Шаг 01:

Миссия 1

Ирина запустила задачу через `executor.submit()`, получив объект `Future`. Как она может получить итоговый результат?

Подсказка: Подумай, какой метод у `Future` возвращает значение.

🔮 Шаг 02:

Миссия 2

⬇ Павел вызывает `future.result(timeout=1)`, но задача ещё не завершилась. Что произойдёт?

Подсказка: Подумай, как обрабатывается превышение времени ожидания.

🔮 Шаг 03:

Миссия 3

✧ Лена запускает сразу несколько задач через `submit`, сохраняя `Futures` в список. Что будет, если вызвать `future.result()` на первом `Future` до завершения остальных?

Подсказка: `result()` блокирует текущий поток до завершения этого `Future`.

🔮 Шаг 04:

Миссия 4

🔍 Сергей вызвал `future.result()` дважды на одном и том же `Future`. Что произойдёт при втором вызове?

Подсказка: После первого вызова результат уже известен.

🔮 Шаг 05:

Миссия 5

⚙ Ольга использует `executor.map()`, но хочет обрабатывать частичные результаты сразу. Как ей поступить?

Подсказка: `map` возвращает итератор, но можно работать и с `Future` напрямую.

🔮 Шаг 06:

Миссия 6

🌐 Илья вызывает `future.result()` внутри основного потока GUI, и интерфейс подвисает. Как лучше поступить?

Подсказка: Подумай, можно ли не блокировать главный поток.

🔮 Шаг 07:

Миссия 7

Сопоставьте методы `Future` с их поведением:

Подсказка: Некоторые методы проверяют состояние, некоторые возвращают результат.

🔧 Шаг 08:

Миссия 8

🔍 Сопоставьте параметры метода `result()` с эффектом:

Подсказка: У `result` может быть таймаут или задержка.

🔧 Шаг 09:

Миссия 9

🔧 Сопоставьте сценарии с поведением `future.result()` :

Подсказка: В разных ситуациях `result` может возвращать или исключать что-то.

🔧 Шаг 10:

Миссия 10

Сопоставьте объекты с их роли в процессе получения результата:

Подсказка: Один объект создаёт задачу, другой — хранит результат.

🔧 Шаг 11:

Миссия 11

✳ Сопоставьте реалистичные примеры с тем, что произойдёт при вызове `result()`:

Подсказка: Представь, что задача вычисляет факториал, или загружает файл.

🔧 Шаг 12:

Миссия 12

🕒 Сопоставьте стратегии ожидания с их поведением:

Подсказка: Можно ждать конкретный `future`, или все сразу.

🔧 Шаг 13:

Миссия 13

🗑 Расположите шаги, которые делает программист, чтобы безопасно получить результаты от нескольких `Future`:

Подсказка: Сначала запуск задач, потом ожидание, потом получение результата.

🔧 Шаг 14:

Миссия 14

Расположите этапы обработки исключений при вызове `result()`:

Подсказка: Сначала проверяем, есть ли исключение, потом ловим конкретные.

🔧 Шаг 15:

Миссия 15

⚙ Расположите этапы работы с `callback`, чтобы не блокировать основной поток:

Подсказка: Сначала назначаем `callback`, затем в `callback` вызываем `result()`.

🔧 Шаг 16:

Миссия 16

🌐 Расположите шаги анализа производительности при использовании `result()` с разными вариантами:

Подсказка: Сравнение блокирующего вызова и обработки по готовности.

🔗 Шаг 17:

Миссия 17

🔗 Расположите шаги отладочного сценария, если программа «виснет» на `result()`:

Подсказка: Проверь, где блокировка, добавь таймаут, разнеси вызовы.

📖 Урок 3 - Практика ускорям API-запросы

🔗 Шаг 01:

Миссия 1

Разработчица Лена пишет бота, который отправляет десятки API-запросов к погодному сервису. Что поможет ей ускорить процесс без перехода на асинхронный фреймворк?

Подсказка: Python имеет встроенный модуль, позволяющий распараллелить выполнение с минимальными изменениями кода.

🔗 Шаг 02:

Миссия 2

🔗 Алексей хочет параллельно опрашивать 50 разных API. Какой тип `Executor` ему подойдёт лучше, если запросы в основном ждут сеть?

Подсказка: Подумай, какой пул эффективнее при I/O-зависимых задачах.

🔗 Шаг 03:

Миссия 3

🌐 Маша реализует сбор данных из 10 API через `ThreadPoolExecutor`. Что она получит от метода `executor.map(fetch_url, urls)` ?

Подсказка: Вспомни, что делает `map` в параллельном контексте — похожа на обычную `map`, но многопоточна.

🔗 Шаг 04:

Миссия 4

👤 Петя вызывает `executor.submit(fetch, url)` для каждого сайта. Как получить результат всех вызовов?

Подсказка: Каждый `submit` возвращает объект `Future`.

🔗 Шаг 05:

Миссия 5

🔗 Света замечает, что API иногда отвечает с ошибкой 500. Как лучше безопасно обработать это при использовании `as_completed()` ?

Подсказка: Важно не прерывать все задачи из-за одной неудачи.

🔗 Шаг 06:

Миссия 6

⚙ Команда DataScience запрашивает данные у разных API с лимитом в 5 потоков. Как задать это ограничение?

Подсказка: Подумай про параметр конструктора Executor.

🔗 Шаг 07:

Миссия 7

📊 Андрей сравнивает время выполнения запросов в обычном цикле и через ThreadPoolExecutor. Что он, скорее всего, заметит при I/O операциях?

Подсказка: Блокирующие сетевые вызовы — отличный пример для многопоточности.

🔗 Шаг 08:

Миссия 8

🔗 Сопоставьте типы Executor с их назначением:

Подсказка: Один для задач с вычислениями, другой — для задач с сетью.

🔗 Шаг 09:

Миссия 9

Сопоставьте элементы кода с их назначением при работе с API:

Подсказка: Подумай, что делает каждая функция в цепочке запросов.

🔗 Шаг 10:

Миссия 10

Сопоставьте возможные ошибки с их причинами:

Подсказка: При параллельных API-запросах важно учитывать ограничения и таймауты.

🔗 Шаг 11:

Миссия 11

🔍 Сопоставьте сценарии и оптимальные инструменты concurrent.futures:

Подсказка: CPU-загрузка и I/O-загрузка требуют разных подходов.

🔗 Шаг 12:

Миссия 12

⚡ Сопоставьте вызовы с их результатом при работе с Future:

Подсказка: Future хранит результат или исключение.

🔗 Шаг 13:

Миссия 13

🔗 Расположите шаги для ускорения API-запросов через ThreadPoolExecutor:

Подсказка: Сначала нужно определить список URL, потом — создать пул и получить результаты.

🔗 Шаг 14:

Миссия 14

Расположите этапы обработки ошибок при параллельных запросах:

Подсказка: Важно поймать исключения, чтобы программа не упала при сбое одного запроса.

🔮 Шаг 15:

Миссия 15

📦 Расположите шаги для измерения ускорения API-запросов:

Подсказка: Измерения лучше проводить до и после внедрения ThreadPoolExecutor.

🔮 Шаг 16:

Миссия 16

⚙️ Расположите действия при использовании `as_completed` для потоковой обработки данных:

Подсказка: `as_completed` возвращает Future по мере готовности.

🔮 Шаг 17:

Миссия 17

🌐 Расположите шаги оптимизации при массовых API-запросах:

Подсказка: Балансируйте количество потоков и частоту запросов.

📖 Урок 4 - Квест подбери лучший пул для задачи

🔮 Шаг 01:

Миссия 1

Разработчик Лена пишет скрипт, который скачивает тысячи мелких файлов. Какой пул лучше выбрать, чтобы не ждать вечность?

Подсказка: Задача связана с сетью, а не с вычислениями процессора.

🔮 Шаг 02:

Миссия 2

⚙️ Андрей обрабатывает изображения: на каждое требуется сложная CPU-операция. Какой пул выбрать?

Подсказка: Если нагрузка на процессор, стоит подумать о процессе, а не о потоке.

🔮 Шаг 03:

Миссия 3

🕸 Данила пишет программу для скачивания веб-страниц и последующего парсинга текста. Что выбрать, если парсинг лёгкий, а загрузка долгая?

Подсказка: Подумай, где программа простаивает — в сети или в CPU?

🔮 Шаг 04:

Миссия 4

🌀 Олег запускает 8 задач CPU-интенсивных расчётов, но его ноутбук имеет 4 ядра. Сколько воркеров ему стоит использовать в ProcessPoolExecutor для максимальной эффективности?

Подсказка: Не всегда больше — значит быстрее. Учти количество физических ядер.

🔮 Шаг 05:

Миссия 5

📬 Даша создаёт телеграм-бота, который параллельно обрабатывает входящие сообщения. Какой пул ей подойдёт?

Подсказка: Если задача не нагружает процессор, а ждёт IO, решение очевидно.

🔧 Шаг 06:

Миссия 6

🔥 Никита хочет ускорить обработку видеофайлов. Задачи CPU-зависимые и тяжёлые. Что выбрать?

Подсказка: Потоки не ускоряют CPU-тяжёлые задачи из-за GIL.

🔧 Шаг 07:

Миссия 7

🔗 Соедини аргумент с его влиянием на пул.

Подсказка: `max_workers` регулирует количество параллельных задач.

🔧 Шаг 08:

Миссия 8

🕒 Сопоставьте тип ошибки с вероятной причиной при работе пула.

Подсказка: `Timeout` — не успел, `BrokenProcessPool` — сломался процесс.

🔧 Шаг 09:

Миссия 9

⚡ Сопоставьте метод с его назначением в объекте `Future`.

Подсказка: `Future` — это обещание результата.

🔧 Шаг 10:

Миссия 10

🔗 Расположите шаги для запуска параллельных задач с использованием `ThreadPoolExecutor`.

Подсказка: Сначала создаём пул, потом подаём задачи, потом ждём результат.

🔧 Шаг 11:

Миссия 11

Расположите этапы оптимизации при выборе пула.

Подсказка: Сначала анализ, потом выбор, потом тест.

🔧 Шаг 12:

Миссия 12

🕒 Расположите шаги от старта к завершению работы с `ProcessPoolExecutor`.

Подсказка: В конце всегда должен быть `shutdown()`.

🔧 Шаг 13:

Миссия 13

⚙️ Расположите шаги при использовании `as_completed` для мониторинга выполнения задач.

Подсказка: Сначала задачи, потом ожидание, потом обработка.

🔧 Шаг 14:

Миссия 14

🔧 Расположите действия при выборе пула под конкретный проект.

Подсказка: Сначала понять задачу, потом выбрать и протестировать.

📖 Модуль 6 - Модуль 6. Война подходов Thread vs Process vs Async

📖 Урок 1 - Сравнение по скорости и памяти

🔧 Шаг 01:

Миссия 1

Лена тестирует скорость обработки запросов через потоки и процессы. Потоки обрабатывают быстрее, но используют меньше памяти. Почему?

Подсказка: Потоки работают в одном процессе и разделяют память.

🔧 Шаг 02:

Миссия 2

⚙ Андрей сравнивает ThreadPoolExecutor и ProcessPoolExecutor для CPU-интенсивных вычислений. Потоки оказались медленнее. Почему?

Подсказка: Вспомни про GIL — он мешает потокам работать параллельно на CPU.

🔧 Шаг 03:

Миссия 3

🔧 Данила измеряет скорость асинхронного запроса к API против ThreadPoolExecutor. Async оказался быстрее. Почему?

Подсказка: Async не создаёт потоки, а переключается во время ожидания IO.

🔧 Шаг 04:

Миссия 4

🔧 Олег запускает ProcessPoolExecutor с 8 процессами на 4-ядерной машине. Память закончилась! Почему?

Подсказка: Каждый процесс создаёт собственную копию данных и интерпретатора Python.

🔧 Шаг 05:

Миссия 5

🔧 Даша запускает 100 асинхронных HTTP-запросов. Память почти не изменилась. Почему async такой экономичный?

Подсказка: Async использует один поток и не создаёт новые стеки вызовов.

🔧 Шаг 06:

Миссия 6

Никита запускает одинаковый код в ThreadPool и ProcessPool. Потоки быстрее, но процессы масштабируются лучше на 8 ядрах. Почему?

Подсказка: GIL не мешает процессам использовать несколько ядер.

🔮 Шаг 07:

Миссия 7

⚡ Артём сравнивает производительность Async и ProcessPool. Async быстрее, но не использует все ядра. Почему?

Подсказка: Async — про IO, а процессы — про CPU.

🔮 Шаг 08:

Миссия 8

🧠 Сопоставьте подход с его характеристикой по памяти.

Подсказка: Поток экономит память, процессы — прожорливы, async — минималист.

🔮 Шаг 09:

Миссия 9

⚙️ Сопоставьте подход с его типичным преимуществом.

Подсказка: Async — скорость ожидания, поток — простота, процессы — мощь CPU.

🔮 Шаг 10:

Миссия 10

Сопоставьте сценарий с его наиболее вероятным узким местом.

Подсказка: Иногда проблема — не в коде, а в типе задачи.

🔮 Шаг 11:

Миссия 11

🧠 Расположите шаги при сравнении скорости разных подходов.

Подсказка: Сначала эксперимент, потом анализ.

🔮 Шаг 12:

Миссия 12

⚙️ Расположите шаги оптимизации при высоком потреблении памяти ProcessPool.

Подсказка: Иногда нужно не больше процессов, а меньше данных.

🔮 Шаг 13:

Миссия 13

🔮 Расположите шаги при измерении эффективности async-подхода.

Подсказка: Не путай IO и CPU.

🔮 Шаг 14:

Миссия 14

🔮 Расположите шаги тестирования CPU-bound задачи на всех подходах.

Подсказка: Процессы тут покажут себя лучше.

🔮 Шаг 15:

Миссия 15

💻 Программист Алексей решил написать скрипт для обработки фотографий. Он задумался: какой подход быстрее справится с обработкой 1000 изображений на многоядерном процессоре? Подсказка: Подумайте, какой метод лучше использует ресурсы процессора.

🔗 Шаг 16:

Миссия 16

Мария пишет парсер сайтов. Она сравнивает: что будет быстрее для 1000 запросов к API? Подсказка: Когда задачи связаны с ожиданием ответа, лучше использовать подход, который умеет не простаивать.

🔗 Шаг 17:

Миссия 17

🐞 Иван запустил вычисления с помощью threading и multiprocessing. Он заметил, что один подход съедает гораздо больше памяти. Какой именно? Подсказка: Подумайте, где каждый процесс дублирует память.

🔗 Шаг 18:

Миссия 18

📖 Вадим проводит тесты: одна задача — вычислить факториалы, другая — скачать веб-страницы. Какой метод быстрее для скачивания страниц? Подсказка: Когда задача связана с сетью, выигрывает метод, который умеет эффективно работать в ожидании ответа.

🔗 Шаг 19:

Миссия 19

⚙️ Ольга считает использование памяти. Какой подход обычно экономнее всего по памяти? Подсказка: Подумайте, где нет копирования процессов и памяти.

📖 Урок 2 - Кейс веб-запросы и CPU-вычисления

🔗 Шаг 01:

Миссия 1

Разработчик Дима пишет сервис для сбора данных с сайтов. Он хочет ускорить выполнение HTTP-запросов. Что ему выбрать? Подсказка: Подумайте, что лучше работает, когда основная нагрузка связана с ожиданием ответа от сервера, а не с процессором.

🔗 Шаг 02:

Миссия 2

⚡ Настя анализирует данные с помощью сложных вычислений (матрицы, статистика). Что ей выгоднее использовать? Подсказка: Вспомните, что ограничивает GIL в Python, и какой подход позволяет эффективно распределять нагрузку по ядрам CPU.

🔗 Шаг 03:

Миссия 3

📖 Программист Паша делает парсер для 5000 URL. Какой подход чаще всего окажется быстрее и экономичнее?

Подсказка: Здесь важнее не вычисления, а количество одновременных запросов.

🔗 Шаг 04:

Миссия 4

💡 Саша пишет сервис обработки изображений: нужно применять фильтры и преобразования. Что будет эффективнее?

Подсказка: Это CPU-bound задача, важно распределение нагрузки на ядра.

🔗 Шаг 05:

Миссия 5

🌐 Петя делает API-клиент для получения данных с десятков сервисов. Какой вариант принесёт наибольшую выгоду?

Подсказка: Сеть → значит подход без долгой блокировки CPU.

🔗 Шаг 06:

Миссия 6

🔄 Сопоставьте подход и его сильную сторону.

Подсказка: Вспомните, что асунс хорош при IO, процессы при CPU, а потоки при задачах с лёгкими блокировками.

🔗 Шаг 07:

Миссия 7

✂ Соотнесите подход и типичную область применения.

Подсказка: Сеть, CPU и простая многозадачность — три разных мира.

🔗 Шаг 08:

Миссия 8

📌 Расположите шаги написания асинхронного клиента.

Подсказка: Сначала создаём цикл событий, потом подключаем задачи, запускаем.

🔗 Шаг 09:

Миссия 9

⚡ Упорядочите шаги при запуске процессов для CPU-задачи.

Подсказка: Сначала создаём функцию, потом процессы, запускаем и ждём.

🔗 Шаг 10:

Миссия 10

📋 Упорядочите шаги при работе с потоками для GUI.

Подсказка: Поток = задача, запуск, ожидание.

🔗 Шаг 11:

Миссия 11

🔗 Расположите этапы асинхронного запроса к API.

Подсказка: Сначала импорт, затем сессия, потом запрос, потом результат.

Шаг 12:

Миссия 12

✳ Упорядочите действия при смешанном подходе (async + processes).

Подсказка: Async качает данные, процессы считают.

Шаг 13:

Миссия 13

✳ Расположите этапы сравнения производительности Thread vs Process vs Async.

Подсказка: Сначала готовим сценарии, потом запускаем, потом замеряем и сравниваем.

Урок 3 - Практика профилирование разных стратегий

Шаг 01:

Миссия 1

Разработчик Артём решил замерить время выполнения скрипта с потоками. Какой инструмент Python он должен использовать для базового профилирования времени?

Подсказка: Вспомни стандартную библиотеку, где есть модуль для измерения CPU и wall-clock времени.

Шаг 02:

Миссия 2

⚡ Лена сравнивает процессы и потоки. Она заметила, что профилирование показывает разное использование CPU. Что чаще всего ограничивает потоки в Python?

Подсказка: Вспомни, что связано с GIL.

Шаг 03:

Миссия 3

📖 Иван запустил CPU-интенсивную задачу через процессы и получил рост загрузки всех ядер. Почему профилирование показало такой результат?

Подсказка: Подумай, кто может реально обойти GIL.

Шаг 04:

Миссия 4

📖 Настя использует async для профилирования запросов к API. Что async показывает лучше всего?

Подсказка: Подумай о том, что связано с I/O.

Шаг 05:

Миссия 5

Программист Дима хочет понять узкое место программы. Какой модуль Python чаще всего применяют для профилирования кода?

Подсказка: Стандартная библиотека, три буквы: c__.

🔔 Шаг 06:

Миссия 6

⚡ Профилирование показало, что при потоках CPU загружен слабо, но запросы выполняются быстрее. Почему?

Подсказка: Подумай, где есть ожидание данных.

🔔 Шаг 07:

Миссия 7

Сопоставь подход и его лучшую область применения:

Подсказка: Думай про CPU-bound и I/O-bound.

🔔 Шаг 08:

Миссия 8

📊 Сопоставьте метрику профилирования и её назначение:

Подсказка: Подумай: время, CPU, память, вызовы функций.

🔔 Шаг 09:

Миссия 9

📖 Сопоставьте инструмент и его роль:

Подсказка: cProfile, line_profiler, memory_profiler...

🔔 Шаг 10:

Миссия 10

⚡ Сопоставь результат профилирования и причину:

Подсказка: Смотри: GIL, блокировка I/O, параллельность процессов.

🔔 Шаг 11:

Миссия 11

Сопоставь программиста и его задачу:

Подсказка: CPU-bound vs I/O-bound.

🔔 Шаг 12:

Миссия 12

📊 Сопоставь формат отчёта профилирования и его назначение:

Подсказка: Текстовые, графические, детализированные.

🔔 Шаг 13:

Миссия 13

Расположи шаги профилирования кода в правильном порядке:

Подсказка: Сначала запускаем замеры, потом анализируем, потом исправляем.

🔔 Шаг 14:

Миссия 14

⚡ Расставь шаги async-профилирования:

Подсказка: event loop, запуск, сбор статистики.

Шаг 15:

Миссия 15

 Расположи этапы оптимизации CPU-bound через процессы:

Подсказка: сначала создаём процессы, потом распределяем задачи.

Шаг 16:

Миссия 16

 Расставь шаги анализа профилирования потоков:

Подсказка: Смотри: запуск, профилирование, анализ, вывод.

Шаг 17:

Миссия 17

Сравнение стратегий: Расставь от самой эффективной для I/O-bound до самой слабой:

Подсказка: Async → Threads → Processes.

Урок 4 - Квест выбери оптимальную стратегию для стартапа

Шаг 01:

Миссия 1

Команда стартапа обсуждает архитектуру: приложение будет часто делать сетевые запросы к API. Что выбрать?

Подсказка: Если запросы в основном ждут ответа от сети, это влияет на выбор.

Шаг 02:

Миссия 2

У стартапа есть модуль машинного обучения, который сильно грузит процессор. Какую стратегию выбрать?

Подсказка: Здесь важна возможность использовать несколько ядер CPU.

Шаг 03:

Миссия 3

Фронтендеры жалуются: API отвечает медленно. Нужно обрабатывать сотни одновременных HTTP-запросов. Что выбрать?

Подсказка: Сеть ≠ CPU. Подумайте, что масштабируется легче.

Шаг 04:

Миссия 4

СТО говорит: «Мы будем хранить много изображений и их обрабатывать фильтрами». Что лучше?

Подсказка: Важно учесть: фильтры обрабатываются на CPU, а не ждут сеть.

Шаг 05:

Миссия 5

Стартап добавляет чат в реальном времени. Как обрабатывать тысячи подключений?

Подсказка: Чат = постоянные сетевые соединения.

🔔 Шаг 06:

Миссия 6

Команда запускает рекомендательную систему: много математики, матричных вычислений. Что выбрать?

Подсказка: Подумайте про загрузку всех ядер процессора.

🔔 Шаг 07:

Миссия 7

Сопоставьте ошибочный выбор стратегии и его последствие.

Подсказка: Подумайте: что будет, если Asyncio использовать не там, где нужно?

🔔 Шаг 08:

Миссия 8

Сопоставьте стартап-сценарий и показатель эффективности.

Подсказка: Подумайте, где важна память, а где — время отклика.

🔔 Шаг 09:

Миссия 9

Сопоставьте ключевое преимущество стратегии с её названием.

Подсказка: Вспомните: Asyncio = сеть, Processes = CPU, Threads = простые задачи.

🔔 Шаг 10:

Миссия 10

Расположите шаги выбора стратегии в стартапе:

Подсказка: Сначала определите задачу, потом подумайте про ресурсы.

🔔 Шаг 11:

Миссия 11

Расположите по возрастанию сложности внедрения стратегий:

Подсказка: Начните с самого простого способа, завершите самым сложным.

🔔 Шаг 12:

Миссия 12

Расположите действия стартапа для оптимизации по времени:

Подсказка: Важен порядок: сначала анализ, потом оптимизация.

🔔 Шаг 13:

Миссия 13

Расположите этапы внедрения чата:

Подсказка: Думайте, как строится сервер: сначала сеть, потом логика.

Шаг 14:

Миссия 14

Расположите шаги для внедрения ML-модуля:

Подсказка: Сначала математика, потом процессы.

Шаг 15:

Миссия 15

Стартап выбирает финальную стратегию. Расположите шаги анализа:

Подсказка: Начинайте с простого теста, завершите комбинированной архитектурой.

Модуль 7 - Модуль 7. GIL — страж Python

Урок 1 - Что делает Global Interpreter Lock

Шаг 01:

Миссия 1

Разработчик Оля услышала о GIL и задумалась: что же он делает в Python?

Подсказка: Подумай о том, кто может держать ключи к интерпретатору, чтобы одновременно не ворвались все потоки.

Шаг 02:

Миссия 2

Тимур написал многопоточный код для скачивания файлов и удивился: загрузка идёт не быстрее, чем в одном потоке. Почему?

Подсказка: Вспомни: GIL удерживает право на выполнение байткода в руках только одного потока.

Шаг 03:

Миссия 3

Команда стартапа обсуждает: какие задачи всё-таки могут выигрывать от многопоточности в Python, несмотря на GIL?

Подсказка: Подумай о том, где Python поток ждёт — и в этот момент может работать другой поток.

Шаг 04:

Миссия 4

Дима пишет числовые вычисления и жалуется: многопоточность не ускоряет расчёты. Что лучше выбрать для обхода GIL?

Подсказка: Иногда вместо потоков нужны тяжёлые процессы, каждый со своим интерпретатором.

Шаг 05:

Миссия 5

Катя использует библиотеку NumPy и замечает: многопоточность там работает быстрее, чем ожидала. Почему?

Подсказка: Некоторые библиотеки умеют временно отпустить GIL.

Шаг 06:

Миссия 6

Максим хочет обойти GIL. Что из списка точно НЕ поможет?

Подсказка: Подумай: GIL встроен в сам Python интерпретатор.

Шаг 07:

Миссия 7

Сопоставьте понятия о GIL с их правильным объяснением.

Подсказка: Вспомни, что блокировка всегда связана с управлением ресурсами.

Шаг 08:

Миссия 8

Сопоставьте участников истории с их реакцией на GIL.

Подсказка: Обрати внимание: кто столкнулся с вычислениями, а кто — с сетевыми вызовами.

Шаг 09:

Миссия 9

Сопоставьте технологию с её отношением к GIL.

Подсказка: Подумай, какие из них используют процессы, а какие — потоки.

Шаг 10:

Миссия 10

Сопоставьте высказывание разработчика с правильным объяснением роли GIL.

Подсказка: Каждое утверждение связано либо с ограничением, либо с обходным решением.

Шаг 11:

Миссия 11

Сопоставьте описание с тем, как оно связано с GIL.

Подсказка: Найди пары: что блокирует, что ускоряет, что обходит.

Шаг 12:

Миссия 12

Расположите шаги понимания, как GIL влияет на программу.

Подсказка: Начинаем с изучения теории и заканчиваем практикой.

Шаг 13:

Миссия 13

Расположите в правильном порядке стратегии для ускорения программы с учётом GIL.

Подсказка: Подумай: сначала определить тип задачи, потом выбрать метод обхода GIL.

Шаг 14:

Миссия 14

Расположите шаги разработки стартапа, столкнувшегося с GIL.

Подсказка: Сначала шок, потом изучение, затем решение.

🔗 Шаг 15:

Миссия 15

Расположите действия разработчика, который хочет проверить влияние GIL.

Подсказка: Сначала код, потом запуск, потом измерения.

🔗 Шаг 16:

Миссия 16

Расположите шаги от знакомства с GIL до его обхода.

Подсказка: Учимся, пробуем, находим решения.

📖 Урок 2 - Почему потоки не ускоряют вычисления

🔗 Шаг 01:

Миссия 1

Программист Анна решила ускорить свои вычисления в Python с помощью потоков. Она удивилась, что программа работает не быстрее. Почему так произошло?

Подсказка: Вспомните про то, что Python защищает доступ к объектам интерпретатора.

🔗 Шаг 02:

Миссия 2

Иван написал код для обработки картинок и запустил его в двух потоках. Но загрузка процессора осталась на одном ядре. Почему?

Подсказка: Потоки в Python делят общий замок при вычислениях.

🔗 Шаг 03:

Миссия 3

Настя читает статьи и встречается термин «GIL». Что именно означает Global Interpreter Lock?

Подсказка: Думайте про замок, который держит интерпретатор.

🔗 Шаг 04:

Миссия 4

Сергей пробует одновременно запускать несколько потоков для вычисления чисел Фибоначчи. Почему время работы не сокращается?

Подсказка: Вспомните про различие между CPU-bound и IO-bound задачами.

🔗 Шаг 05:

Миссия 5

Вова заметил, что при работе с сетью в потоках программа реально ускоряется. Почему здесь GIL не мешает?

Подсказка: Подумайте, когда потоки ждут ответа от внешних источников.

🔗 Шаг 06:

Миссия 6

Катя сравнила выполнение задачи в потоках и процессах. Почему процессы реально ускоряют CPU-bound задачи?

Подсказка: Каждый процесс имеет свой собственный интерпретатор.

🔮 Шаг 07:

Миссия 7

Андрей написал игру и хотел использовать потоки для обработки физики. Но столкнулся с лагами. Что лучше использовать?

Подсказка: Для вычислений используйте механизм, который не ограничен GIL.

🔮 Шаг 08:

Миссия 8

Сопоставьте тип задач с их эффективностью при использовании потоков в Python.

Подсказка: CPU-bound $\neq$ IO-bound. Важно различать вычисления и ожидания.

🔮 Шаг 09:

Миссия 9

Сопоставьте библиотеку или технологию с её отношением к GIL.

Подсказка: Думайте о том, кто использует процессы, а кто потоки.

🔮 Шаг 10:

Миссия 10

Сопоставьте утверждение с правильным объяснением о GIL.

Подсказка: Подумайте, что делает GIL безопасным, но не быстрым.

🔮 Шаг 11:

Миссия 11

Сопоставьте разработчиков с их реакцией на GIL.

Подсказка: Кто-то злится на медленные вычисления, кто-то доволен безопасностью.

🔮 Шаг 12:

Миссия 12

Сопоставьте задачу с подходящим инструментом.

Подсказка: Подумайте: CPU-bound лучше в процессах, IO-bound в потоках.

🔮 Шаг 13:

Миссия 13

Расположите шаги работы GIL при переключении потоков.

Подсказка: Думайте о том, что поток должен получить замок, выполнить код и освободить замок.

🔮 Шаг 14:

Миссия 14

Расположите подходы для ускорения кода Python по сложности реализации.

Подсказка: От простого к сложному: готовые библиотеки → процессы → асинхронщина → C-расширения.

Шаг 15:

Миссия 15

Расположите типы задач по убыванию пользы от потоков в Python.

Подсказка: IO-bound задачи выигрывают больше всего.

Урок 3 - Как обойти GIL

Шаг 01:

Миссия 1

Разработчик Дима хочет ускорить вычисления в Python с помощью потоков. Но он слышал о GIL. Что это означает?

Подсказка: Подумай, кто управляет доступом к интерпретатору.

Шаг 02:

Миссия 2

 Программистка Аня запустила два CPU-интенсивных потока в Python. Результат оказался медленнее, чем ожидалось. Почему?

Подсказка: GIL не даёт им работать параллельно на разных ядрах.

Шаг 03:

Миссия 3

Ваня хочет обойти GIL для задач, требующих много вычислений. Что ему лучше использовать?

Подсказка: Подумай про независимые процессы, а не потоки.

Шаг 04:

Миссия 4

 Backend-разработчик решил ускорить запросы к API с помощью потоков. Сработает ли это при I/O-задачах?

Подсказка: I/O освобождает GIL на ожидании.

Шаг 05:

Миссия 5

 Тимур хочет ускорить Python, используя NumPy. Почему NumPy может обходить GIL?

Подсказка: Подумай о вычислениях вне интерпретатора Python.

Шаг 06:

Миссия 6

Сопоставь подходы к обходу GIL с их особенностями.

Подсказка: Думай, когда важен CPU, а когда I/O.

Шаг 07:

Миссия 7

 У программиста есть разные задачи. Сопоставь задачу с оптимальным способом обхода GIL.

Подсказка: Учитывай тип нагрузки: CPU или I/O.

🔧 Шаг 08:

Миссия 8

📦 Сопоставь библиотеку с её ролью в обходе GIL.

Подсказка: Обрати внимание, какие библиотеки используют процессы, а какие C.

🔧 Шаг 09:

Миссия 9

🔗 Сопоставь описание проблемы с подходящим способом обхода GIL.

Подсказка: Не все задачи требуют процессов — иногда хватит потоков или async.

🔧 Шаг 10:

Миссия 10

⚡ Сопоставь ключевое преимущество способа обхода GIL.

Подсказка: Подумай: CPU-интенсивные задачи требуют одного подхода, а I/O — другого.

🔧 Шаг 11:

Миссия 11

Расположи шаги запуска multiprocessing в правильном порядке.

Подсказка: Сначала создаём процесс, потом запускаем и ждём его завершения.

🔧 Шаг 12:

Миссия 12

🏠 Расположи этапы использования ThreadPoolExecutor для обхода GIL при I/O задачах.

Подсказка: Не забудь про контекстный менеджер.

🔧 Шаг 13:

Миссия 13

⚙️ Расположи этапы написания программы на asyncio.

Подсказка: Обрати внимание: сначала определяются корутины, а потом запускается цикл.

🔧 Шаг 14:

Миссия 14

⚡ Расположи этапы использования NumPy для обхода GIL.

Подсказка: NumPy позволяет работать с массивами напрямую.

🔧 Шаг 15:

Миссия 15

📁 Расположи шаги параллельной загрузки файлов с помощью потоков.

Подсказка: Потоки хороши, если основная часть времени — ожидание сети.

🔧 Шаг 16:

Миссия 16

🔗 Расположи шаги создания процессов для обработки больших CSV.

Подсказка: Подумай про разбиение данных и объединение результата.

Шаг 17:

Миссия 17

Расположи шаги выбора подхода для обхода GIL.

Подсказка: Начни с анализа задачи.

Урок 4 - Практика симуляция GIL

Шаг 01:

Миссия 1

 Программист Артём запустил многопоточный код и удивился: загрузка CPU только на одно ядро! Почему?

Подсказка: Подумайте, какой механизм внутри Python управляет доступом потоков к интерпретатору.

Шаг 02:

Миссия 2

Разработчица Лена запускает CPU-bound задачу в нескольких потоках. Результат — медленнее, чем в одном потоке! Почему так?

Подсказка: Вспомните, что потоки борются за один глобальный ключ.

Шаг 03:

Миссия 3

 Алексей хочет ускорить вычисления численных матриц. Он выбирает: использовать потоки или процессы?

Подсказка: Подумайте, что обходит GIL, потому что запускает отдельный интерпретатор.

Шаг 04:

Миссия 4

 Разработчик Игорь пробует обойти GIL. Какой инструмент из стандартной библиотеки Python чаще всего используют для этого?

Подсказка: Этот модуль создаёт новые процессы, а не потоки.

Шаг 05:

Миссия 5

 Маша пишет приложение, где нужно много сетевых запросов. Ей советуют использовать asyncio. Почему?

Подсказка: Вспомните: GIL мешает только CPU-bound задачам, но не IO-bound.

Шаг 06:

Миссия 6

Никита использует библиотеку NumPy. Почему его вычисления могут обходить GIL?

Подсказка: Подумайте, на каком языке написаны тяжёлые части NumPy.

🔧 Шаг 07:

Миссия 7

📦 Петя хочет передавать данные между процессами. Какой объект из модуля multiprocessing ему подойдёт?

Подсказка: Вспомните структуры, похожие на очереди.

🔧 Шаг 08:

Миссия 8

📄 Сопоставьте способ обхода GIL с его объяснением:

Подсказка: Подумайте, где используются процессы, а где освобождают GIL внутри кода C.

🔧 Шаг 09:

Миссия 9

🔗 Сопоставьте библиотеку с её типичной областью применения в обходе GIL:

Подсказка: Сравните CPU-bound и IO-bound задачи.

🔧 Шаг 10:

Миссия 10

Сопоставьте задачу с правильным инструментом:

Подсказка: Не все задачи требуют процессов — иногда достаточно событийного цикла.

🔧 Шаг 11:

Миссия 11

Сопоставьте способ и его ключевое преимущество:

Подсказка: Некоторые обходят GIL через процессы, другие — через компиляцию в C.

🔧 Шаг 12:

Миссия 12

⚡ Сопоставьте технологию и пример реальной ситуации:

Подсказка: Обратите внимание, где нужен расчёт, а где — быстрый отклик.

🔧 Шаг 13:

Миссия 13

Расположите шаги при использовании multiprocessing для обхода GIL:

Подсказка: Сначала создаём процессы, потом запускаем, ждём и собираем результат.

🔧 Шаг 14:

Миссия 14

Упорядочите действия при написании асинхронного кода:

Подсказка: Сначала пишем async-функции, потом используем await, затем запускаем через event loop.

🔧 Шаг 15:

Миссия 15

📦 Упорядочите шаги при компиляции Python-кода с помощью Cython для обхода GIL:

Подсказка: Сначала пишем .pyx, затем компилируем, потом используем в Python.

🔮 Шаг 16:

Миссия 16

🖥️ Упорядочите этапы при использовании NumPy для вычислений без блокировок GIL:

Подсказка: Сначала создаём массивы, потом вызываем функции, потом получаем результат.

📖 Урок 5 - Квест обмани интерпретатор

🔮 Шаг 01:

Миссия 1

Программист Игорь заметил, что его потоковый код в Python работает медленнее, чем ожидалось. Что чаще всего мешает потокам исполняться параллельно в Python?

Подсказка: Подумай о механизме, который контролирует доступ к объектам интерпретатора.

🔮 Шаг 02:

Миссия 2

Анна пишет скрипт, который работает с многопоточностью. Она заметила, что CPU загружен на 100%, но ускорения почти нет. Какое объяснение верное?

Подсказка: Вспомни, что в Python только один поток одновременно может выполнять байт-код.

🔮 Шаг 03:

Миссия 3

Дима запускает несколько потоков для вычислений. Он подозревает, что в некоторых задачах GIL не мешает. В каком случае GIL меньше всего влияет?

Подсказка: Подумай о задачах, где поток чаще ждёт, чем считает.

🔮 Шаг 04:

Миссия 4

Катя использует multiprocessing вместо threading. Почему её код стал работать быстрее?

Подсказка: Вспомни, какие сущности не зависят от GIL.

🔮 Шаг 05:

Миссия 5

Витя пишет игру и хочет ускорить обработку численных расчётов. Что ему поможет обойти GIL?

Подсказка: Подумай о сторонних библиотеках и их реализации.

🔮 Шаг 06:

Миссия 6

Сопоставьте понятия с их описаниями, чтобы помочь Оле разобраться, как работает GIL.

Подсказка: Вспомни, что GIL контролирует и какие задачи от него страдают.

🔮 Шаг 07:

Миссия 7

Петя путает threading и multiprocessing. Помоги ему: сопоставь термин и его объяснение.
Подсказка: Подумай, что общее и различное у процессов и потоков.

🔮 Шаг 08:

Миссия 8

Ваня изучает библиотеки. Сопоставь библиотеку с её особенностью.
Подсказка: Некоторые библиотеки написаны на C и умеют обходить GIL.

🔮 Шаг 09:

Миссия 9

Маша думает, что GIL мешает всем языкам. Сопоставь язык и его ситуацию.
Подсказка: GIL есть не во всех языках программирования.

🔮 Шаг 10:

Миссия 10

Сопоставьте 🧠 задачу и способ её оптимизации в Python.
Подсказка: Подумай, когда лучше потоки, а когда процессы.

🔮 Шаг 11:

Миссия 11

Сопоставьте стратегию и её описание, чтобы помочь Лизе выбрать правильный инструмент.
Подсказка: Каждая стратегия решает свою проблему.

🔮 Шаг 12:

Миссия 12

Расположи шаги, как программист Саша оптимизировал свой код, чтобы обойти GIL.
Подсказка: Подумай о порядке действий: диагностика → выбор инструмента → реализация → тестирование.

🔮 Шаг 13:

Миссия 13

Расположи действия программиста Коли, который решил ускорить сервер с помощью asyncio.
Подсказка: Сначала он заметил проблему, затем изучил инструменты и реализовал решение.

🔮 Шаг 14:

Миссия 14

Расположи этапы, как программистка Юля использовала NumPy для обхода GIL.
Подсказка: Сначала определяют проблему, потом заменяют код библиотекой, и тестируют.

🔮 Шаг 15:

Миссия 15

Расположи шаги, как программист Федя проверял гипотезу о вреде GIL.

Подсказка: Сначала строится гипотеза, потом эксперимент, затем анализ.

Модуль 8 - Модуль 8. Синхронизация порядок из хаоса

Урок 1 - Lock, RLock, Semaphore, Event, Condition

Шаг 01:

Миссия 1

 Вася пишет многопоточный парсер и замечает, что потоки обращаются к одной и той же базе данных одновременно. Какой инструмент Python лучше всего использовать, чтобы защитить критическую секцию?

Подсказка: Представь дверь, которую может открыть только один разработчик за раз .

Шаг 02:

Миссия 2

Лена использует RLock в проекте. Зачем ей нужен именно RLock, а не обычный Lock?

Подсказка: Иногда программисту нужно открыть одну и ту же дверь несколько раз подряд .

Шаг 03:

Миссия 3

 Петя создает пул потоков для загрузки файлов. Ему нужно ограничить количество потоков, работающих одновременно. Какой инструмент лучше использовать?

Подсказка: Это как пропускать только определённое количество человек в лифт .

Шаг 04:

Миссия 4

 В проекте Маши есть поток, который должен подождать, пока другой поток закончит обработку данных, и только потом продолжить. Какой инструмент идеально подходит?

Подсказка: Представь зелёный свет светофора .

Шаг 05:

Миссия 5

 Артём использует Condition для координации потоков. Когда именно поток должен вызвать notify() ?

Подсказка: Это как сказать друзьям: «Всё готово, можно начинать!» .

Шаг 06:

Миссия 6

Сопоставьте инструменты синхронизации Python с их реальной аналогией:

Подсказка: Представь разные способы организации очереди в кафе .

Шаг 07:

Миссия 7

 Соотнесите ситуацию программиста и инструмент синхронизации:

Подсказка: Каждая проблема требует своего решения.

🔗 Шаг 08:

Миссия 8

✳ Найдите правильные пары: инструмент и его Python-класс.

Подсказка: Вспомни модуль threading .

🔗 Шаг 09:

Миссия 9

📋 Соотнесите инструмент синхронизации с ключевым методом.

Подсказка: Вспомни, что делает каждый объект.

🔗 Шаг 10:

Миссия 10

🔗 Расположите шаги работы с Lock в правильном порядке:

Подсказка: Вспомни правило: захват — работа — освобождение.

🔗 Шаг 11:

Миссия 11

📋 Расположите шаги использования Event:

Подсказка: Сначала создаём объект, потом ждём, потом сигналим.

🔗 Шаг 12:

Миссия 12

✳ Упорядочите шаги работы с Semaphore:

Подсказка: Это как турникет, который пропускает ограниченное число людей.

🔗 Шаг 13:

Миссия 13

🔗 Расположите шаги работы с Condition:

Подсказка: Один поток ждёт, другой сообщает.

🔗 Шаг 14:

Миссия 14

🔗 Определи порядок действий при использовании RLock:

Подсказка: Один поток может зайти несколько раз подряд.

🔗 Шаг 15:

Миссия 15

🔗 Расположите шаги координации потоков при обработке данных: один готовит данные, другой ждёт сигнал.

Подсказка: Это похоже на кухню ресторана 🍳 и официанта.

🔗 Шаг 16:

Миссия 16

🔍 **Финальная задача:** расположите шаги использования всех инструментов в логике многопоточного сервиса:

Подсказка: Подумай о комплексном применении: защита ресурса, ожидание событий, ограничение доступа.

📖 Урок 2 - Очереди для потоков и процессов

💡 Шаг 01:

Миссия 1

Разработчик Иван решил использовать `queue.Queue()` в многопоточном приложении для загрузки изображений. Зачем вообще использовать очередь между потоками?

Подсказка: Очередь помогает безопасно обмениваться данными между потоками без гонок и блокировок вручную.

💡 Шаг 02:

Миссия 2

👤 Программистка Аня запустила 5 потоков, которые читают задачи из очереди. Что произойдет, если очередь пуста, а поток вызовет `q.get()` без параметров?

Подсказка: По умолчанию метод `get()` блокирует поток до появления элемента.

💡 Шаг 03:

Миссия 3

🔗 Тимур использует `Queue(maxsize=3)`. Он добавил 3 элемента и вызывает `put()` снова без параметров. Что произойдет?

Подсказка: Очередь с ограничением размера ведет себя как переполненный буфер — поток будет ждать освобождения места.

💡 Шаг 04:

Миссия 4

🏢 Команда DevOps создает систему обработки логов с использованием `multiprocessing.Queue`. Почему нельзя просто использовать `queue.Queue` из модуля `threading`?

Подсказка: Потоки делят память, а процессы — нет.

💡 Шаг 05:

Миссия 5

⚙️ Настя решила использовать очередь для передачи результатов от рабочих потоков главному потоку. Как лучше организовать завершение потоков после выполнения всех задач?

Подсказка: Потоки не знают, когда очередь пуста, если просто вызвать `get()`.

💡 Шаг 06:

Миссия 6

Сопоставьте методы класса `queue.Queue` с их назначением.

Подсказка: Методы `put` и `get` управляют элементами, а `task_done` и `join` — процессом выполнения.

💡 Шаг 07:

Миссия 7

Сопоставьте ситуации с типом очереди, который лучше использовать.

Подсказка: Разные очереди подходят для разных сценариев: потоки, процессы, приоритеты.

🔗 Шаг 08:

Миссия 8

Сопоставьте понятие с его описанием в контексте многопроцессорной очереди.

Подсказка: Вспомните, что multiprocessing.Queue основана на механизмах IPC — межпроцессного взаимодействия.

🔗 Шаг 09:

Миссия 9

👤 Разработчик Петя путает `get_nowait()` и `get(block=False)`. Помогите ему понять разницу.

Подсказка: Они делают одно и то же, но называются по-разному.

🔗 Шаг 10:

Миссия 10

Сопоставьте элементы сценария с их описанием при использовании очереди задач в продюсер-консьюмер модели.

Подсказка: Продюсер кладет задачи, консьюмеры их берут.

🔗 Шаг 11:

Миссия 11

⬇ Сопоставьте состояния очереди с их значением для мониторинга в реальном времени.

Подсказка: Методы `empty()`, `full()` и `qsize()` помогают понять текущее состояние, но их данные не всегда точны.

🔗 Шаг 12:

Миссия 12

Расположите шаги создания продюсер-консьюмер модели с использованием очереди в правильном порядке:

Подсказка: Сначала создаются объекты, потом потоки, потом начинается работа.

🔗 Шаг 13:

Миссия 13

⚙ Расположите действия в правильной последовательности при работе с `task_done()` и `join()`.

Подсказка: Каждый элемент, извлеченный через `get()`, должен быть отмечен как выполненный.

🔗 Шаг 14:

Миссия 14

Расположите шаги настройки multiprocessing.Queue для обмена данными между процессами.

Подсказка: Все начинается с создания очереди и запуска процессов.

🔗 Шаг 15:

Миссия 15

Расположите этапы обработки задач в многопоточной системе с использованием очереди.

Подсказка: Вспомните принцип FIFO: первый пришёл — первый ушёл.

🔔 Шаг 16:

Миссия 16

🔗 Расположите действия при реализации паттерна «pipeline» с очередями между этапами обработки.

Подсказка: Данные проходят через несколько очередей, как по конвейеру.

📖 Урок 3 - Практика синхронизируем счётчик

🔔 Шаг 01:

Миссия 1

Программист Лена пишет скрипт для подсчёта заказов. Каждый поток увеличивает общий счётчик заказов. Что случится, если не использовать синхронизацию?

Подсказка: Подумай о том, что потоки могут одновременно читать и изменять одно и то же значение.

🔔 Шаг 02:

Миссия 2

Алекс тестирует параллельное обновление переменной. Он замечает, что значения иногда "пропускаются". Что является лучшим способом предотвратить это?

Подсказка: Используй инструмент, который управляет доступом потоков к общему ресурсу.

🔔 Шаг 03:

Миссия 3

Тимур написал функцию, где каждый поток увеличивает глобальную переменную. Но он добавил Lock. Что произойдёт, если забыть вызвать lock.release()?

Подсказка: Подумай, что произойдёт с другими потоками, когда они захотят получить доступ к заблокированному ресурсу.

🔔 Шаг 04:

Миссия 4

Программист Анна использует with lock: в своём коде. Почему это считается хорошей практикой?

Подсказка: Контекстный менеджер — это гарантированный способ безопасного освобождения ресурсов.

🔔 Шаг 05:

Миссия 5

Даниил хочет подсчитать количество скачиваний из разных потоков. Какой подход правильный?

Подсказка: Каждый поток должен увеличивать общий счётчик безопасным способом.

🔔 Шаг 06:

Миссия 6

Юля проверяет скорость программы с Lock. Она замечает замедление. Почему это нормально?

Подсказка: Синхронизация всегда влечёт компромисс между скоростью и безопасностью данных.

🔗 Шаг 07:

Миссия 7

Команда разработчиков использует RLock вместо Lock. Когда это необходимо?

Подсказка: Иногда одна и та же функция вызывает другую, которая также блокирует тот же Lock.

🔗 Шаг 08:

Миссия 8

Сопоставь типы синхронизации с их описанием.

Подсказка: Некоторые блокировки позволяют нескольким потокам входить, другие — только одному.

🔗 Шаг 09:

Миссия 9

Соедини понятие с примером из жизни программиста.

Подсказка: Представь, что потоки — это люди, которые делят ресурсы.

🔗 Шаг 10:

Миссия 10

Соедини ошибку с её вероятной причиной.

Подсказка: Иногда проблема не в коде, а в порядке блокировок.

🔗 Шаг 11:

Миссия 11

Сопоставь код с его результатом.

Подсказка: Обрати внимание, используется ли Lock или нет.

🔗 Шаг 12:

Миссия 12

Расположи шаги создания потокобезопасного счётчика в правильном порядке.

Подсказка: Сначала создаётся Lock, потом потоки, затем синхронизация и подсчёт.

🔗 Шаг 13:

Миссия 13

Расставь порядок действий при использовании Event для синхронизации.

Подсказка: Сначала создаётся событие, потом потоки ждут сигнала, а затем он подаётся.

🔗 Шаг 14:

Миссия 14

Расположи действия для избежания deadlock при нескольких Lock.

Подсказка: Важно соблюдать одинаковый порядок блокировок в каждом потоке.

🔧 Шаг 15:

Миссия 15

Расставь действия при использовании Queue для синхронизации потоков-производителей и потребителей.

Подсказка: Сначала данные кладутся, потом извлекаются, и всё завершается закрытием очереди.

🔧 Шаг 16:

Миссия 16

Расставь этапы отладки синхронизированного кода.

Подсказка: Сначала нужно воспроизвести проблему, затем добавить логирование и проверить результат.

📖 Урок 4 - Квест поймай баг гонки

🔧 Шаг 01:

Миссия 1

Программист Вика запустила два потока, которые одновременно увеличивают общий счётчик. После нескольких запусков результат всегда разный. Что происходит?

Подсказка: Когда потоки обращаются к одной переменной без защиты, начинается хаос.

🔧 Шаг 02:

Миссия 2

🔒 Андрей пытается защитить общий список от одновременной записи потоками. Какой инструмент Python ему подойдёт?

Подсказка: Этот объект можно использовать с контекстным менеджером with .

🔧 Шаг 03:

Миссия 3

🔑 Никита использует Lock , но забывает вызывать release() . Что произойдет?

Подсказка: Подумай, что происходит с другими потоками, если замок никто не открывает.

🔧 Шаг 04:

Миссия 4

🔗 Программист Илья решил использовать RLock вместо Lock . Зачем?

Подсказка: Подумай, когда один и тот же поток может несколько раз захватить блокировку.

🔧 Шаг 05:

Миссия 5

⬇ Команда запускает параллельную обработку данных и использует Semaphore . Зачем он нужен?

Подсказка: Это ограничитель: пропускает только несколько потоков одновременно.

🔧 Шаг 06:

Миссия 6

🔗 Маша использует Event для управления потоками. Какова основная роль этого объекта?

Подсказка: Он работает как флаг, который можно поднять или опустить.

🔗 Шаг 07:

Миссия 7

⚡ Сопоставьте тип синхронизации с его назначением.

Подсказка: Один управляет доступом, другой уведомляет, третий считает.

🔗 Шаг 08:

Миссия 8

🧠 Сопоставьте ситуацию и правильное решение.

Подсказка: Если результат прыгает — значит, нет синхронизации.

🔗 Шаг 09:

Миссия 9

💬 Подберите правильное объяснение к понятию.

Подсказка: Все эти слова встречаются в мире многопоточности.

🔗 Шаг 10:

Миссия 10

Определи, какой инструмент лучше подходит.

Подсказка: Разные задачи — разные замки.

🔗 Шаг 11:

Миссия 11

🔗 Сопоставь ошибки и причины.

Подсказка: Чаще всего виноваты забытые блокировки или неправильный порядок их захвата.

🔗 Шаг 12:

Миссия 12

🔗 Сопоставьте примеры использования и подходящие инструменты.

Подсказка: Подумай, где нужно ограничение, а где сигнал.

🔗 Шаг 13:

Миссия 13

Расположи шаги поиска и устранения гонки потоков в правильном порядке.

Подсказка: Сначала наблюдаем, потом локируем, потом тестируем.

🔗 Шаг 14:

Миссия 14

Упорядочь действия при работе с объектом Semaphore .

Подсказка: Всегда начинай с инициализации, а заканчивай освобождением.

🔗 Шаг 15:

Миссия 15

🔗 Упорядочи действия при использовании Event для запуска потоков после подготовки данных.

Подсказка: Сначала готовим данные, потом сигналим потокам.

🔗 Шаг 16:

Миссия 16

🔗 Расставь шаги защиты доступа к файлу несколькими потоками.

Подсказка: Не забудь про освобождение замка!

📁 Модуль 9 - Модуль 9. Асинхронные структуры данных

📖 Урок 1 - Async Queue, Semaphore, Event

🔗 Шаг 01:

Миссия 1

Разработчик Артём тестирует асинхронный код с `asyncio.Queue`. Что делает метод `put()` ?

Подсказка: Этот метод используется для добавления элемента в очередь, но не возвращает его обратно.

🔗 Шаг 02:

Миссия 2

Разработчица Ника использует `asyncio.Queue.get()`. Что произойдёт, если очередь пуста?

Подсказка: Асинхронный код не выбрасывает ошибку, он просто ждёт появления элемента.

🔗 Шаг 03:

Миссия 3

Тимофей создал очередь `queue = asyncio.Queue(maxsize=2)`. Что произойдёт, если он попытается добавить третий элемент без извлечения предыдущих?

Подсказка: Очередь с ограниченным размером умеет ждать, пока появится место.

🔗 Шаг 04:

Миссия 4

Инженер Алексей хочет синхронизировать несколько корутин, ограничив одновременный доступ к ресурсу. Какой объект он выберет?

Подсказка: Этот инструмент асинхронности похож на классический замок, но позволяет ограничить не до одного, а до нескольких потоков.

🔗 Шаг 05:

Миссия 5

Разработчица Юлия использует `asyncio.Event`. Что произойдёт, если корутина вызовет `await event.wait()` до того, как событие установлено?

Подсказка: Корутина должна дожидаться сигнала от другого участка кода.

🔗 Шаг 06:

Миссия 6

Антон использует `asyncio.Semaphore(3)` . Что это означает?

Подсказка: Подумайте, сколько задач одновременно смогут выполнять защищённый участок кода.

🔗 Шаг 07:

Миссия 7

Программистка Лера тестирует `asyncio.Queue` . Какой метод позволяет узнать текущее количество элементов в очереди?

Подсказка: Этот метод не ждёт и не блокирует выполнение — просто возвращает число.

🔗 Шаг 08:

Миссия 8

Разработчик Паша изучает ключевые методы `asyncio.Queue` . Сопоставьте методы с их действиями.

Подсказка: Вспомните, какие из них добавляют элементы, какие извлекают, а какие только проверяют состояние.

🔗 Шаг 09:

Миссия 9

Команда разработчиков работает с `asyncio.Semaphore` . Сопоставьте действия с их описанием.

Подсказка: Вспомните, что семафор может быть захвачен и освобождён.

🔗 Шаг 10:

Миссия 10

Разработчица Маша отлаживает систему сигналов через `asyncio.Event` . Сопоставьте методы события с их назначением.

Подсказка: Один метод устанавливает флаг, другой сбрасывает, а третий — проверяет.

🔗 Шаг 11:

Миссия 11

Инженер Вика строит пайплайн обработки данных. Сопоставьте структуру данных с её типичной задачей.

Подсказка: Очередь — для передачи данных, событие — для синхронизации, семафор — для ограничения доступа.

🔗 Шаг 12:

Миссия 12

Расположите шаги работы с `asyncio.Queue` в правильном порядке.

Подсказка: Сначала создаётся очередь, потом помещаем элемент, затем извлекаем и завершаем обработку.

🔗 Шаг 13:

Миссия 13

Упорядочьте действия при использовании `asyncio.Semaphore` для защиты критического участка.

Подсказка: Сначала создаём объект, потом захватываем, выполняем работу, и наконец освобождаем.

Шаг 14:

Миссия 14

Упорядочьте события при использовании `asyncio.Event`.

Подсказка: Сначала событие создаётся, потом кто-то его ждёт, затем другой устанавливает, и ожидание завершается.

Урок 2 - Практика асинхронная очередь задач

Шаг 01:

Миссия 1

Программист Артём решил написать асинхронный планировщик задач. Какой объект лучше всего подойдёт для хранения заданий, ожидающих обработки?

Подсказка: Этот объект умеет работать с элементами в порядке их поступления и безопасен при работе из разных корутин.

Шаг 02:

Миссия 2

 Инна реализует асинхронную очередь задач для веб-краулера. Что произойдёт, если вызвать `await queue.get()` на пустой очереди?

Подсказка: Асинхронные очереди не выбрасывают исключения при пустоте, но умеют ждать события.

Шаг 03:

Миссия 3

 Данила создал очередь задач с ограничением размера: `queue = asyncio.Queue(maxsize=3)`. Что произойдёт при попытке `await queue.put(item)`, если очередь уже заполнена?

Подсказка: Асинхронность спасает от ошибок, даже если очередь переполнена — просто приходится подождать.

Шаг 04:

Миссия 4

 Катя пишет обработчик задач, где каждое задание нужно выполнить и затем пометить как завершённое. Какой метод очереди используется для отметки о завершении?

Подсказка: После получения элемента из очереди и выполнения задачи обязательно нужно вызвать этот метод, иначе `join()` будет ждать вечно.

Шаг 05:

Миссия 5

 Когда следует использовать метод `await queue.join()`?

Подсказка: Этот метод нужен, когда важно убедиться, что все задачи из очереди действительно обработаны.

🔗 Шаг 06:

Миссия 6

Тимур делает систему логирования через асинхронную очередь. Почему использование обычного списка (`list`) вместо `asyncio.Queue` может привести к ошибкам?

Подсказка: Подумайте о конкурентном доступе к структуре данных из разных корутин.

🔗 Шаг 07:

Миссия 7

Маша хочет добавить тайм-аут при получении элемента из очереди. Как правильно это сделать?

Подсказка: Иногда элемент так и не появляется — стоит ограничить ожидание.

🔗 Шаг 08:

Миссия 8

Сопоставьте методы `asyncio.Queue` с их назначением.

Подсказка: Подумайте, какие методы добавляют, какие извлекают и какие ждут завершения работы.

🔗 Шаг 09:

Миссия 9

👤 Разработчик Стас создал несколько асинхронных очередей для разных типов данных. Сопоставьте очередь с типом обрабатываемых задач.

Подсказка: Смотрите на контекст: картинки, логи, пользователи...

🔗 Шаг 10:

Миссия 10

🔍 Сопоставьте тип проблемы с возможным решением при работе с асинхронной очередью.

Подсказка: Подумайте, как избежать зависаний и переполнения.

🔗 Шаг 11:

Миссия 11

🔗 Сопоставьте концепцию асинхронного программирования с её описанием.

Подсказка: Асинхронность — это про ожидание без блокировки.

🔗 Шаг 12:

Миссия 12

📦 Сопоставьте тип асинхронной структуры данных с её особенностью.

Подсказка: Все они решают разные задачи: очередь, стек, буфер...

🔗 Шаг 13:

Миссия 13

Расположите шаги создания асинхронной очереди задач в правильном порядке.

Подсказка: Сначала создаём очередь, затем добавляем задачи, потом обрабатываем и завершаем.

🔗 Шаг 14:

Миссия 14

🔗 Расположите этапы работы воркера, который извлекает задачи из очереди и выполняет их.

Подсказка: Не забудьте про отметку завершения каждой задачи!

🔗 Шаг 15:

Миссия 15

📌 Расположите этапы завершения асинхронной программы с очередью задач.

Подсказка: После добавления задач всегда нужно подождать их выполнения.

🔗 Шаг 16:

Миссия 16

🔗 Расположите действия, необходимые для реализации ограниченного пула воркеров с асинхронной очередью.

Подсказка: Важно сначала ограничить параллельность, затем дождаться завершения всех задач.

📖 Урок 3 - Квест создай асинхронный конвейер

🔗 Шаг 01:

Миссия 1

Алиса пишет асинхронный конвейер и хочет выбрать между очередями `asyncio.Queue` и обычным списком + `await loop.run_in_executor`. Что правильнее?

Подсказка: подумайте, зачем нужна асинхронность: блокировки, неблокирующий ввод-вывод.

🔗 Шаг 02:

Миссия 2

✂ Боб хочет задать максимальный размер очереди (buffer) в конвейере — какую из опций он выберет:

Подсказка: ограниченная очередь помогает контролировать давление на потребителя.

🔗 Шаг 03:

Миссия 3

🔗 Водитель конвейера Каролина хочет, чтобы элемент обрабатывался поочерёдно: шаг А → шаг В → шаг С. Какой механизм выбрать между этапами?

Подсказка: используйте объект, по которому разные корутины могут передавать данные.

🔗 Шаг 04:

Миссия 4

🔗 Эмили хочет запустить несколько параллельных потребителей, чтобы ускорить конвейер. Какое решение она выберет?

Подсказка: несколько корутин могут читать из одной очереди одновременно.

🔗 Шаг 05:

Миссия 5

🔗 Сопоставьте этап конвейера и его назначение:

Этап:

Ответ:

- Прочитать входной канал
- Преобразовать данные (например, парсинг)
- Фильтровать ненужные элементы
- Записать результат в следующий этап

Назначение:

1. Удалить лишние данные, которые не надо обрабатывать дальше
2. Передать элемент дальше в конвейере
3. Привести данные к нужному формату
4. Получить сырой элемент из внешнего источника

Подсказка: анализируйте роль каждого шага как «ввод → обработка → фильтрация → вывод».

🔗 Шаг 06:

Миссия 6

🔗 Сопоставьте ситуации с подходящим поведением конвейера:

Ситуация:

Ответ:

- Очередь переполняется
- Потребитель тормозит
- Новая задача появилась слишком рано
- Ошибка на одном из этапов

Решение:

1. Использовать ``try/except`` внутри этапа и логировать ошибку
2. Ограничить ``maxsize`` и дожждаться освобождения места
3. Запускать дополнительного потребителя или балансировка нагрузки
4. Буферизовать задачу во вспомогательной очереди

Подсказка: подумайте, как система может устойчиво реагировать на сбои и пиковые нагрузки.

🔗 Шаг 07:

Миссия 7

🔗 Сопоставьте типы очередей и их поведение в `async`:

Тип:

Ответ:

- Очередь с ограничением ``maxsize``
- Неограниченная очередь
- Очередь с приоритетом
- Кольцевая очередь (`ring buffer`)

Поведение:

1. Элементы обрабатываются в порядке приоритета
2. Размер неограничен, риск переполнения памяти

3. Запретить добавление если заполнена до предела ``maxsize``
4. Переиспользует фиксированный буфер, перезаписывает старые элементы

Подсказка: вспомните разницу между приоритетами и кольцевым буфером.

Шаг 08:

Миссия 8

 Сопоставьте рекомендации по дизайну конвейера с их эффектом:

Рекомендация:

Ответ:

- Минимизировать связанность этапов
- Использовать back-pressure
- Ограничивать внутренние очереди
- Обрабатывать ошибки локально

Эффект:

1. Система устойчивее к сбоям
2. Предотвращение переполнения
3. Этапы можно изменять независимо
4. Уменьшение накопления задач

Подсказка: для каждой рекомендации подумайте о контроле нагрузки и гибкости архитектуры.

Шаг 09:

Миссия 9

 Сопоставьте события и те, которые должны их логировать:

Событие:

Ответ:

- Элемент вошёл в очередь
- Этап начал обработку
- Этап завершил обработку успешно
- Этап упал с исключением

Что логировать:

1. Время входа и идентификатор элемента
2. Продолжительность обработки и этап
3. Стек ошибки и этап
4. Результат обработки (например, статус, размер)

Подсказка: подумайте, что важно для отладки и мониторинга.

Шаг 10:

Миссия 10

 Сопоставьте части кода из шаблона конвейера и их роль:

Фрагмент:

Ответ:

- ``while True: item = await in_q.get()``

- ``try: result = await process(item)``
- ``await out_q.put(result)``
- ``finally: in_q.task_done()``

Роль:

1. Сигнализировать, что элемент завершён на входной очереди
2. Получить элемент из входной очереди
3. Передать результат дальше
4. Обработать элемент и получить результат

Подсказка: вспомните, как работают очереди и завершение задач.

Шаг 11:

Миссия 11

 Сопоставьте метрики конвейера и то, что они измеряют:

Метрика:

Ответ:

- Средняя задержка элемента
- Пропускная способность (throughput)
- Утилизация потребителей
- Длина очереди

Измерение:

1. Сколько элементов обрабатываются в секунду
2. Сколько времени занимает прохождение через конвейер
3. Среднее число ожидающих элементов в очереди
4. Процент времени, когда этап занят обработкой

Подсказка: метрики бывают «сквозные» и «локальные».

Шаг 12:

Миссия 12

 Сопоставьте компоненты конвейера и их цели:

Компонент:

Ответ:

- Продюсер
- Этап обработки
- Потребитель
- Мониторинг / логирование

Цель:

1. Запускать работу с результатами
2. Записывать статистику, отслеживать сбои
3. Поставлять элементы в очередь
4. Преобразовывать или фильтровать элементы

Подсказка: подумайте о жизненном цикле элемента в системе.

Шаг 13:

Миссия 13

 Упорядочите шаги конфигурации асинхронного конвейера:

1. Создать очереди между этапами
2. Запустить корутины-этапы обработки
3. Запустить продюсера для ввода элементов
4. Запустить контроллер, мониторящий очереди и метрики
5. Запустить потребителя, получающего финальный результат

Подсказка: конвейер настраивается от входа к выходу, но контроллер можно запустить после ключевых этапов.

Шаг 14:

Миссия 14

 Расположите в правильном порядке события при обработке одного элемента в конвейере:

1. Поставить элемент в входную очередь
2. Взять элемент из очереди этапом
3. Обработать элемент
4. Поставить обработанный результат в выходной очередь
5. Потребитель получает финальный результат

Подсказка: элемент проходит путь «продюсер → этапы → потребитель».

Шаг 15:

Миссия 15

 Упорядочите действия продюсера, контроллера и потребителя при завершении работы конвейера:

1. Оповестить этапы о завершении (например, послать `None`)
2. Ждать, пока очереди будут опустошены
3. Завершить контроллер сбора метрик
4. Потребители и этапы завершают свои задачи
5. Произвести финальный отчёт и лог

Подсказка: завершение должно идти от краёв в центр и потом собрать статистику.

Шаг 16:

Миссия 16

 Упорядочите шаги анализа конвейера после запуска:

1. Собрать лог-файлы всех этапов
2. Посчитать статистики (латентность, загрузка, очередь)
3. Найти узкие места (bottlenecks)
4. Внести улучшения (балансировка, изменение buffer, параллелизм)
5. Переотладить и перезапустить

Подсказка: анализ — это цикл: измерить → найти слабые места → улучшить → проверить снова.

Модуль 10 - Модуль 10. Асинхронные веб-запросы

Урок 1 - aiohttp клиент и сервер

Шаг 01:

Миссия 1

 Разработчик Ирина хочет написать простой HTTP-клиент на aiohttp. С чего ей начать?

Подсказка: Надо сначала создать объект сессии.

Шаг 02:

Миссия 2

 Денис делает асинхронный GET запрос с aiohttp. Какой метод используется у объекта сессии?

Подсказка: Метод называется так же, как HTTP-глагол.

Шаг 03:

Миссия 3

 Когда Вика получает ответ от `await session.get(...)`, что она может вызвать, чтобы прочитать тело в виде текста?

Подсказка: Это асинхронный метод ответа.

Шаг 04:

Миссия 4

 Борис написал сервер на aiohttp и хочет запустить приложение. Какой вызов запускает серверное приложение?

Подсказка: В aiohttp есть функция `web.run_app`.

Шаг 05:

Миссия 5

 Вера хочет установить маршрут на сервере aiohttp. Какой декоратор она применит к функции-обработчику?

Подсказка: Он импортируется из `aiohttp.web`.

Шаг 06:

Миссия 6

 Когда сервер aiohttp обрабатывает запрос, что обычно делает обработчик?

Подсказка: Он принимает объект запроса и возвращает объект ответа.

Шаг 07:

Миссия 7

 Сопоставьте методы aiohttp от клиента с их назначением.

Подсказка: методы называются по HTTP-глаголам и возвращают объекты Response.

Шаг 08:

Миссия 8

🌐 Сопоставьте объекты `aiohttp` сервера с их ролью.

Подсказка: Один — приложение, другой — маршрутизация запросов.

🔗 Шаг 09:

Миссия 9

🔗 Сопоставьте ошибки `aiohttp` с их описаниями.

Подсказка: Часто встречаются ``ClientError``, ``ServerDisconnectedError`` и таймауты.

🔗 Шаг 10:

Миссия 10

➡ Расположите шаги написания простого клиента `aiohttp`.

Подсказка: Сначала открывается сессия, потом запрос, потом чтение ответа.

🔗 Шаг 11:

Миссия 11

✂ Расположите шаги создания HTTP-сервера с `aiohttp`.

Подсказка: Сначала приложение и маршруты, потом запуск.

🔗 Шаг 12:

Миссия 12

⌘ Расположите шаги обработки POST-запроса на сервере `aiohttp`.

Подсказка: Чтение тела, обработка, ответ.

🔗 Шаг 13:

Миссия 13

🔄 Расположите шаги параллельных клиентских запросов с `aiohttp`.

Подсказка: Используйте `gather` или создавай задачи.

🔗 Шаг 14:

Миссия 14

⌚ Расположите шаги таймаутной обработки запросов в `aiohttp`.

Подсказка: Используйте ``ClientTimeout`` и обрабатывайте исключения.

🔗 Шаг 15:

Миссия 15

🔍 Расположите шаги дебага и логирования `aiohttp`-запросов.

Подсказка: Включи логирование, поймай ошибки, проследи поток.

🔗 Шаг 16:

Миссия 16

🔄 Расположите полный сценарий: клиент → сервер → клиент (round-trip) на `aiohttp`.

Подсказка: Клиент отправляет, сервер принимает и отвечает, клиент обрабатывает.

Урок 2 - Асинхронный API

Шаг 01:

Миссия 1

 Алиса хочет создать асинхронный API-клиент, который будет обращаться к внешнему сервису. Что ей нужно сделать первым шагом?

Подсказка: Прежде чем делать запрос, нужна сессия.

Шаг 02:

Миссия 2

 Борис делает `await session.get(url)` в своём клиенте API. Какой метод он должен вызвать, чтобы получить JSON-ответ?`

Подсказка: Ответ можно получить как текст или как JSON.

Шаг 03:

Миссия 3

 Денис хочет делать множество API запросов параллельно. Какой подход будет правильным?

Подсказка: В `asynсio` есть способ запустить сразу несколько задач.

Шаг 04:

Миссия 4

 Влад хочет, чтобы клиентский API возвращал ошибки, если сервер вернёт код 400+. Какое поведение ему стоит реализовать?

Подсказка: Обычно проверяют атрибут ``status`` в ответе.

Шаг 05:

Миссия 5

 Вика хочет закрывать сессии после использования. Какой конструкцией удобно это делать?

Подсказка: Есть асинхронный контекстный менеджер.

Шаг 06:

Миссия 6

 Сопоставьте части HTTP-API с их назначением в асинхронном клиенте.

Подсказка: Один отвечает за адрес, другой — за тело, третий — за заголовки.

Шаг 07:

Миссия 7

Сопоставьте исключения `aiоhttp` с ситуациями, когда они могут возникнуть.

Подсказка: Подумайте о сетевых ошибках и таймаутах.

Шаг 08:

Миссия 8

 Сопоставьте режимы сериализации данных в API с их применением.

Подсказка: JSON часто используется, но есть и другие форматы.

🔧 Шаг 09:

Миссия 9

🔗 Сопоставьте методы HTTP и типичные сценарии API.

Подсказка: GET — для чтения, POST — для создания/отправки.

🔧 Шаг 10:

Миссия 10

➡ Расположите шаги реализации API-клиента с aiohttp.

Подсказка: Сначала открывается сессия, потом идут запросы, затем закрывается.

🔧 Шаг 11:

Миссия 11

⚡ Расположите шаги обработки API-запроса на сервере FastAPI / aiohttp-сервисе (гибрид):

Подсказка: Войдёт приём запроса, получение данных, формирование ответа.

🔧 Шаг 12:

Миссия 12

📦 Расположите шаги, когда клиент делает несколько асинхронных API вызовов параллельно:

Подсказка: Определить корутины → собрать в задачи → ждать → обработать.

🔧 Шаг 13:

Миссия 13

⌚ Расположите шаги использования таймаута при запросах через aiohttp:

Подсказка: Таймаут задаётся заранее и обрабатывается исключениями.

🔧 Шаг 14:

Миссия 14

🔑 Расположите шаги авторизации API через заголовки (например, токен):

Подсказка: Нужно подготовить заголовок, передать в запрос, сервер проверит.

🔧 Шаг 15:

Миссия 15

Расположите шаги логирования и обработки ошибок в API-клиенте:

Подсказка: Оберни запрос, логируй статус, поймай исключения, закрой session.

🔧 Шаг 16:

Миссия 16

🔄 Расположите полный сценарий работы клиента → API → клиент (round-trip):

Подсказка: Клиент запускает запрос, сервер обрабатывает, возвращает, клиент парсит.

📖 Урок 3 - Практика собираем данные с 100 сайтов

🔧 Шаг 01:

Миссия 1

Программист Антон решил ускорить свой парсер цен с 100 сайтов. Он слышал, что можно использовать асинхронность в Python. Какой модуль поможет ему отправлять десятки запросов одновременно?

Подсказка: Вспомните модуль, который позволяет использовать ключевые слова `async` и `await`.

💡 Шаг 02:

Миссия 2

🌐 Разработчица Мария хочет асинхронно скачивать данные с десятков API. Какой HTTP-клиент подойдёт лучше всего для работы с `asyncio`?

Подсказка: Этот клиент является асинхронным аналогом `requests`.

💡 Шаг 03:

Миссия 3

⚙️ Стажёр Дима пишет парсер и случайно забыл слово `await` перед вызовом асинхронной функции. Что произойдёт?

Подсказка: Асинхронная функция возвращает объект, который нужно дождаться.

💡 Шаг 04:

Миссия 4

🕒 Тимур запустил 100 асинхронных запросов, но программа выполняется медленно. Он подозревает, что неправильно создал сессию `aiohttp`. Что является лучшей практикой при использовании `aiohttp.ClientSession`?

Подсказка: Сессии стоит создавать один раз и переиспользовать.

💡 Шаг 05:

Миссия 5

💥 Разработчик Олег получил ошибку `RuntimeError: Event loop is closed`. Что чаще всего становится причиной такой ошибки?

Подсказка: Это связано с неправильным управлением циклом событий `asyncio`.

💡 Шаг 06:

Миссия 6

🌀 Программист Ира хочет запустить множество корутин одновременно. Какая функция `asyncio` поможет ей дождаться выполнения всех сразу?

Подсказка: Эта функция принимает список корутин и возвращает результаты всех.

💡 Шаг 07:

Миссия 7

🔑 Антон изучает ключевые слова `async` и `await`. Сопоставьте понятия с их назначением.

Подсказка: Одно ключевое слово создаёт асинхронную функцию, другое — ожидает её выполнение.

💡 Шаг 08:

Миссия 8

🔗 Сопоставьте функции `aiohttp` с их описанием.

Подсказка: Вспомните, какие методы используются для HTTP-запросов.

🔧 Шаг 09:

Миссия 9

🔧 Программисты обсуждают обработку ответов сервера. Сопоставьте методы с их назначением.
Подсказка: Один метод возвращает текст, другой — JSON.

🔧 Шаг 10:

Миссия 10

🕒 Сопоставьте термины асинхронности с их определениями.
Подсказка: Подумайте, что означает конкурентность и параллелизм.

🔧 Шаг 11:

Миссия 11

🕒 Тимур тестирует асинхронный парсер. Сопоставьте типы ошибок с причинами их возникновения.
Подсказка: Подумайте о типичных ошибках при работе с event loop.

🔧 Шаг 12:

Миссия 12

🌐 Сопоставьте типы задач асинхронных запросов с их подходящими инструментами.
Подсказка: Вспомните, какой инструмент используют для массового сбора API-данных.

🔧 Шаг 13:

Миссия 13

🔧 Расположите шаги создания асинхронного HTTP-клиента в правильном порядке.
Подсказка: Сначала создаётся сессия, затем выполняются запросы, и в конце — закрытие.

🔧 Шаг 14:

Миссия 14

📋 Расположите этапы выполнения asyncio программы в правильном порядке.
Подсказка: Программа должна сначала определить корутину, потом запустить цикл событий.

🔧 Шаг 15:

Миссия 15

🕒 Расположите шаги отладки асинхронного парсера в правильном порядке.
Подсказка: Начните с проверки кода, а завершите анализом логов.

🔧 Шаг 16:

Миссия 16

📋 Расположите шаги сбора данных с 100 сайтов в правильной последовательности.
Подсказка: Сначала получаем список ссылок, а в конце сохраняем результат.

🔧 Шаг 17:

Миссия 17

🔧 Финальная миссия! Расположите шаги оптимизации асинхронного скрипта.

Подсказка: Оптимизация — это не только код, но и логика запросов.

Урок 4 - Квест сделай быстрее, чем конкурент

Шаг 01:

Миссия 1

Девопс-разработчик Лена хочет параллельно запустить 5 HTTP-запросов к разным микросервисам. Какой подход быстрее всего начать?

Подсказка: Используйте готовые библиотеки, не писать всё вручную.

Шаг 02:

Миссия 2

✦ Стартап-команда Бориса хочет опросить 3 внешних API и взять только первый успешный ответ. Что выбрать?

Подсказка: Смотрите функции, которые возвращают первый завершённый корутин.

Шаг 03:

Миссия 3

🔗 При большом числе одновременных запросов к серверу, что может стать узким местом?

Подсказка: Подумайте о ресурсах сети и ограничениях ОС.

Шаг 04:

Миссия 4

🧠 Сопоставьте термин с его значением в контексте асинхронных запросов.

Термины:

Ответ:

- event loop
- coroutine
- future / task
- gather

Определения:

1. Объект, представляющий собой отложенное вычисление, которое будет выполнено позже.
2. Функция или метод, обозначенный как асупс, который можно приостановить.
3. Запуск нескольких корутин и ожидание всех их результатов.
4. Центральный цикл, который планирует и выполняет задачи.

Подсказка: Начните с того, что делает цикл событий, затем корутина, затем её упаковка, и как собрать.

Шаг 05:

Миссия 5

🔗 Сопоставьте ошибку и её признак при асинхронных запросах:

Ошибки:

Ответ:

- `TimeoutError`
- `ConnectionError`
- `CancelledError`
- `TooManyRedirects`

Признаки:

1. Корутина прерывается вручную или автоматически.
2. Сервер не отвечает долго.
3. Слишком много перенаправлений.
4. Не удаётся установить TCP-связь.

Подсказка: Подумайте, какая ошибка вызывается, если клиент ждёт слишком долго, какая — если соединение не установить.

 Шаг 06:**Миссия 6**

💡 Сопоставьте подход оптимизации с его эффектом:

Подходы:

Ответ:

- HTTP Keep-Alive
- Сжатие ответов (gzip)
- Connection pooling
- Ограничение concurrency (семафор)

Эффекты:

1. Снижение одновременной нагрузки и предотвращение перегруза.
2. Повторное использование TCP-соединения между запросами.
3. Уменьшение объёма передаваемых данных.
4. Поддержание пула открытых соединений.

Подсказка: Какие техники относятся к сетевым соединениям, а какие — к контролю конкуренции?

 Шаг 07:**Миссия 7**

🔧 Расположите этапы обработки результатов параллельных запросов:

1. Фильтрация успешных ответов и отбрасывание ошибок
2. Запуск запросов параллельно
3. Анализ данных из ответов
4. Ожидание завершения всех запросов

Подсказка: Сначала запускаем, затем ждём, затем фильтруем, затем анализируем.

 Шаг 08:**Миссия 8**

Расположите шаги по обработке таймаутов:

1. Установка тайм-аута на уровне запроса
2. Проверка, что запрос превысил лимит
3. Повторный запрос или fallback
4. Обработка исключения и логирование

Подсказка: Что должно быть сначала — установка лимита или проверка?

Шаг 09:

Миссия 9

📌 Расположите методы оптимизации по возрастанию сложности внедрения:

1. Сжатие ответов
2. Ограничение concurrency
3. Connection pooling
4. Реализация backoff-стратегии

Подсказка: Какой метод проще всего вставить в существующий код?

Шаг 10:

Миссия 10

🌐 Расположите стратегические шаги, чтобы опередить конкурента в скорости API:

1. Измерить текущую скорость ответов
2. Определить «узкие места» (сеть, CPU, DNS)
3. Внедрить параллелизм и оптимизации
4. Повторно измерить и сравнить

Подсказка: Без измерений не поймёшь, что улучшать.

Шаг 11:

Миссия 11

🧩 Вы — программист в команде, где оппонент успел получить результат API за 200 мс. Какой метод стоит попробовать сначала?

Подсказка: Ускорение начинается с самого простого вмешательства.

Шаг 12:

Миссия 12

🎯 Команда хочет ввести ограничение на число одновременно выполняемых корутин, чтобы не перегрузить систему. Что выбрать?

Подсказка: Ищите конструкции семафоров или пулов в asyncio.

Шаг 13:

Миссия 13

Сопоставьте метрику и её роль при оценке производительности запросов:

Метрики:

Ответ:

- Latency

- Throughput
- Error rate
- 99-й перцентиль (p99)

Роли:

1. Процент запросов, которые заняли больше времени, чем почти все остальные.
2. Среднее количество обработанных запросов в единицу времени.
3. Доля неуспешных запросов к API.
4. Время ответа одного запроса.

Подсказка: Latency — это задержка одного запроса, throughput — пропускная способность, error rate — ошибки.

Шаг 14:

Миссия 14

 Расположите стадии оптимизации по циклу улучшений:

1. Анализ метрик и логов
2. Корректировка кода или конфигурации
3. Мониторинг изменений
4. Запуск кода в производство

Подсказка: Помните «Plan → Do → Check → Act».

Шаг 15:

Миссия 15

Расположите действия, которые предпринимает система retry с backoff:

1. Логирование ошибки
2. Ожидание увеличивающегося интервала
3. Повторить запрос
4. Прекращение попыток после предела

Подсказка: Что делать сразу после ошибки, прежде чем повторить?

Шаг 16:

Миссия 16

 Вы заметили, что большинство медленных запросов связаны с DNS-разрешением. Что попробовать?

Подсказка: Используйте кэширование или асинхронный DNS.

Модуль 11 - Модуль 11. Реактивные потоки

Урок 1 - Потоки событий (RxPy)

Шаг 01:

Миссия 1

Аня пишет реактивное приложение с RxPy: у неё есть поток кликов кнопки, и она хочет фильтровать только те клики, где кнопка «Сохранить».

Какой оператор RxPy лучше всего подходит, чтобы оставить только события с нужным атрибутом?

Подсказка: Выбираем по условию.

🔮 Шаг 02:

Миссия 2

Боб объединяет два потока данных (например, из вебсокета и таймера) и хочет получить единый поток смешанных событий.

Какой оператор RxPy соединит два потока?

Подсказка: Объединение потоков — «mix».

🔮 Шаг 03:

Миссия 3

Света хочет преобразовывать каждое событие клика в асинхронную операцию сохранения, и затем возвращать результат как поток.

Какой оператор поможет применить функцию, возвращающую Observable, ко всем входным событиям?

Подсказка: «flat» + «map».

🔮 Шаг 04:

Миссия 4

Игорь хочет взять первые три события и затем автоматически завершить поток.

Какой оператор используется, чтобы ограничить количество элементов?

Подсказка: «take».

🔮 Шаг 05:

Миссия 5

Женя хочет игнорировать повторы подряд и получать только изменения значений в потоке.

Какой оператор убирает подряд идущие дубликаты?

Подсказка: «сжатие» одинаковых.

🔮 Шаг 06:

Миссия 6

Сопоставьте оператор RxPy и его описание.

Подсказка: Подумайте, что делает «map» и что делает «filter».

Операторы:

Ответ:

- map
- filter
- merge
- take

Описание:

1. Оставляет только те значения, которые удовлетворяют условию
2. Применяет функцию к каждому значению
3. Соединяет два потока в один

4. Берёт первые N значений из потока

Шаг 07:

Миссия 7

 Сопоставьте оператор и пример использования в жизни программиста.

Подсказка: Пример подскажет, что оператор делает.

Операторы:

Ответ:

- flat_map
- distinct_until_changed
- skip
- zip

Примеры:

1. Получать только изменения статуса (без повторов подряд)
2. Применить асинхронную функцию и развёрнуть вложенные потоки
3. Пропустить первые несколько загрузок перед началом логики
4. Синхронизировать два потока элементов попарно

Шаг 08:

Миссия 8

 Сопоставьте проблему реактивного потока и правильное решение с RxPy оператором.

Подсказка: Проблемы часто решаются через операторы фильтрации или ограничения.

Проблемы:

Ответ:

- слишком много повторений подряд
- комбинирование разных источников
- конвертация события в вложенный поток
- ограничение объёма входа

Решения (операторы):

1. distinct_until_changed
2. merge
3. flat_map
4. take

Шаг 09:

Миссия 9

 Расположите этапы создания реактивной цепочки RxPy от подписки до завершения:

Подсказка: Подумайте, что сначала определяют источники, затем трансформации, затем подписку.

1. subscribe (подписаться на поток)
2. определение источника Observable (например, from_)
3. применение операторов (map, filter и др.)
4. обработка ошибок или завершение потока

🔗 Шаг 10:

Миссия 10

📖 У Алексея есть такая последовательность шагов: filter → map → merge → take. Нужно упорядочить корректный поток от источников до конечной подписки.

Подсказка: Источники → трансформации → комбинирование → ограничение → подписка.

1. map
2. merge
3. take
4. filter

🔗 Шаг 11:

Миссия 11

Расположите шаги обработки ошибок в реактивном потоке:

Подсказка: Сначала ловим, затем обрабатываем, затем продолжаем или завершить.

1. catch (перехват ошибки)
2. retry или возвращение запасного потока
3. subscribe с блоком on_error

🔗 Шаг 12:

Миссия 12

Олег хочет каждые 2 секунды эмиттить «heartbeat»-событие независимо от внешних источников.

Какой оператор создаст регулярно повторяющийся поток?

Подсказка: Регулярный «тик».

🔗 Шаг 13:

Миссия 13

🔗 Сопоставьте источник Observable и реальный жизненный сценарий программиста.

Подсказка: Подумайте, что может бесконечно генерировать события.

Источники:

Ответ:

- from_iterable
- from_future
- interval
- create

Сценарии:

1. преобразовать список команд в поток
2. реакция на завершение асинхронной задачи
3. сердцебиение (heartbeat)
4. настраиваемый источник, читает события с вебсокета вручную

🔗 Шаг 14:

Миссия 14

Сопоставьте RxPy оператор и эффект на поток:

Подсказка: Некоторые операторы изменяют структуру, другие фильтруют или ограничивают.

Операторы:

Ответ:

- buffer
- scan
- take_until
- merge_map

Эффекты:

1. накопление элементов в списки по времени или размеру
2. применение аккумулятора к элементам потока
3. прекращение при событии другого потока
4. плоское отображение вложенных потоков, подобно flat_map

Шаг 15:

Миссия 15

У Глеба есть цепочка: from_iterable → filter → flat_map → merge → take_until → subscribe. Нужно расположить её в корректном порядке от начала до конца.

Подсказка: Источник → фильтрация → разворачивание вложенных → слияние → ограничение → подписка.

1. flat_map
2. take_until
3. merge
4. filter
5. from_iterable
6. subscribe

Шаг 16:

Миссия 16

🔍 Финальная миссия: расположите этапы сложной обработки событий от источников до завершения:

Подсказка: Начинаем с источника, затем трансформации, комбинации, фильтрации, ограничения и подписки с ошибками.

1. flat_map
2. catch / обработка ошибок
3. filter
4. merge
5. subscribe
6. take
7. from_future

Урок 2 - Реактивное программирование

Шаг 01:

Миссия 1

Алиса запускает поток данных об измерениях температуры и хочет сразу реагировать на аномалии. Какой подход наиболее характерен для реактивного программирования?

Подсказка: В реактивном стиле важна реакция на поступающие события, а не запрашивание состояния.

Шаг 02:

Миссия 2

✳ Борис строит поток телеметрии с датчиков скорости машины. Как назвать элемент, который выступает источником данных в реактивном конвейере?

Подсказка: Это отправитель событий, который генерирует поток значений.

Шаг 03:

Миссия 3

🕒 Катя объединяет два потока — данных из GPS и данных о состоянии двигателя. Какой оператор поможет объединить два потока в один?

Подсказка: Ищите термин, который означает “смешивание” или “слияние” потоков.

Шаг 04:

Миссия 4

Дмитрий хочет преобразовать каждое событие о температуре в строку «Temp: X °C». Какой оператор реактивного потока подходит?

Подсказка: Вы применяете функцию ко всем элементам потока.

Шаг 05:

Миссия 5

🔍 Елена хочет отфильтровать поток, пропуская только значения выше порога. Какой оператор ей нужен?

Подсказка: Выбирайте только те элементы, что удовлетворяют условию.

Шаг 06:

Миссия 6

🔗 Сопоставьте названия операторов реактивного потока с их ролью:

Операторы:

Ответ:

- map
- filter
- merge
- scan

Роли:

1. Преобразует каждое событие
2. Объединяет несколько потоков в один
3. Выбирает события по условию
4. Агрегирует с накоплением

Подсказка: map — преобразование, filter — фильтрация, scan похож на reduce, merge — слияние потоков.

🔔 Шаг 07:

Миссия 7

🐞 Федор хочет запустить поток, который будет выдавать значения через каждые 5 секунд независимо от источника. Какой оператор может дать ему «периодический тик»?

Подсказка: Часто используются таймеры или интервалы.

🔔 Шаг 08:

Миссия 8

👤 Группа потоков данных с разной частотой. Сопоставьте стратегии объединения с описанием: Стратегии:

Ответ:

- zip
- concat
- switchLatest (или switch)
- combineLatest

Описание:

1. Берёт последний элемент из каждого потока при любом новом событии
2. Приходит переключение на новый внутренний поток, старые отменяются
3. Склеивает потоки один за другим
4. Ждёт событие из каждого, затем соединяет попарно

Подсказка: Проанализируйте, как каждый оператор синхронизирует потоки по времени.

🔔 Шаг 09:

Миссия 9

🐞 Галина хочет накопить сумму значений потока, как в аккумуляторе. Какой оператор использовать?

Подсказка: Нужно хранение промежуточного результата и его обновление.

🔔 Шаг 10:

Миссия 10

🏢 В облачном приложении приходят два потока: поток входящих запросов (RPS) и поток ошибок. Сопоставьте стратегию реакции с описанием:

Стратегии:

Ответ:

- backpressure
- throttle</> >
- debounce
- buffer

Описание:

1. Ограничивает частоту событий (например, не более N в секунду)
2. Если поток слишком быстрый, накапливает события в буфер
3. Игнорирует события, которые идут слишком часто, держит паузу между ними
4. Управляет давлением, чтобы предотвратить перегрузку подписчиков

Подсказка: Подумайте: есть ли механизм пропуска, накопления или замедления.

🔔 Шаг 11:

Миссия 11

🔍 В системе мониторинга три потока: нагрузка, ошибки и время ответа. Сопоставьте методы реагирования с практическим действием:

Методы:

Ответ:

- filter + scan
- combineLatest
- throttle
- buffer

Действия:

1. держать усреднённую скорость ошибок за последнюю минуту
2. объединять нагрузку и время ответа, чтобы реагировать при росте обоих сразу
3. пропускать лишние обновления времени ответа, если они идут слишком часто
4. собирать все ошибки за короткий промежуток для пакетной обработки

Подсказка: Каждая стратегия даёт разный способ обработки потока — фильтрация + накопление, объединение, ограничение частоты, накопление.

🔔 Шаг 12:

Миссия 12

⚙ Сергей хочет отменять старые внутренние потоки, как только приходит новое событие с параметрами. Какой оператор ему нужен?

Подсказка: Подумайте про переключения и игнорирование старых подписок.

🔔 Шаг 13:

Миссия 13

📊 В стартапе поступают метрики по CPU, памяти и задержкам. Сопоставьте архитектуру реактивного решения с её компонентами:

Компоненты:

Ответ:

- Source / Publisher
- Operator / Processor
- Subscriber / Sink
- Backpressure / Flow control

Роли:

1. Устройство, получающее готовые события и реагирующее
2. Модуль, который применяет map, filter, scan и др.
3. Механизм, регулирующий скорость передачи событий
4. **Начало потока** — датчики, логирование и генераторы событий

Подсказка: Подумайте, кто генерирует, кто обрабатывает и кто контролирует поток.

🔧 Шаг 14:

Миссия 14

📌 Расположите этапы обработки реактивного потока от источника до подписчика:

1. Subscriber получает и обрабатывает события
2. Source генерирует события
3. Operator применяет трансформации (map, filter и др.)
4. Backpressure или flow control регулирует поток при перегрузке

Подсказка: Сначала создаётся поток, затем он преобразуется, затем контролируется и, наконец, потребляется.

🔧 Шаг 15:

Миссия 15

🔗 Разработчица решает построить pipeline из трёх операторов: фильтрация, трансформация и сумма накопления. Расположите их в порядке применения:

1. Агрегация накопления
2. Трансформация значений
3. Фильтрация по порогу

Подсказка: Сначала отсеиваем ненужное, затем преобразуем, затем агрегируем накопление.

🔧 Шаг 16:

Миссия 16

🌐 В распределённой системе: один поток с клиента, затем трансформация, объединение с другим потоком, регулирование скорости, и вывод в базу. Расположите модули в правильном порядке:

1. Backpressure / flow control
2. Слияние потоков (merge / combineLatest)
3. Source / клиентский ввод
4. Operator / трансформация
5. Subscriber / запись в базу данных

Подсказка: Подумайте: сначала источник, потом трансформеры, потом объединение, регулировка и, наконец, запись.

📖 Урок 3 - Практика обработка потоков данных

🔧 Шаг 01:

Миссия 1

Вы обрабатываете поток событий от датчиков температуры. Какой подход наиболее прямолинейный для фильтрации значений выше порога?

Подсказка: фильтрация — это просто проверка условия на каждом элементе.

🔧 Шаг 02:

Миссия 2

Поток поступающих логов: вы хотите преобразовать каждую строку в объект Python. Что использовать?

Подсказка: map-преобразование удобно для примитивных маппингов.

🔗 Шаг 03:

Миссия 3

Вы хотите объединить два потока событий (например, клики и просмотры). Какой базовый оператор подходит?

Подсказка: объединение часто называют merge или zip, подумайте, нужен ли параллелизм.

🔗 Шаг 04:

Миссия 4

Сопоставьте оператор реактивного потока с его описанием:

Операторы:

Ответ:

- filter
- map
- merge
- flat_map

Описание:

1. преобразует один элемент в один другой
2. отбрасывает элементы, не проходящие проверку
3. объединяет два потока в один
4. разворачивает вложенные потоки

Подсказка: подумайте, что делает flat_map, если map возвращает список.

🔗 Шаг 05:

Миссия 5

Сопоставьте типы окон (windowing) со сценариями:

Тип окна:

Ответ:

- tumbling
- sliding
- session

Сценарии:

1. фиксированные интервалы без перекрытия
2. окна с перекрытием, каждое окно сдвигается
3. окна по сессиям пользователя

Подсказка: подумайте, когда окно открывается и закрывается.

🔗 Шаг 06:

Миссия 6

Сопоставьте понятия потоковой обработки:

Понятие:

Ответ:

- latency

- throughput
- backpressure
- watermark

Определения:

1. обратное давление при перегрузке
2. пропускная способность, количество элементов в единицу времени
3. метка, сигнал прогресса по времени в потоке
4. задержка от входа к выходу

Подсказка: latency — задержка, throughput — скорость, backpressure — реакция системы.

Шаг 07:

Миссия 7

Расположите шаги обработки окна (window) по времени:

1. закрытие окна и эмиссия результатов
2. открытие нового окна
3. сбор элементов в окно
4. инициализация состояния окна

Подсказка: сначала создаётся окно, затем накопление, затем завершение.

Шаг 08:

Миссия 8

У вас есть трансформация: filter → map → merge. Расположите эти операторы в порядке применения к потокам A и B:

1. map на поток A
2. filter на поток B
3. merge потоков после трансформаций

Подсказка: трансформации применяются перед объединением.

Шаг 09:

Миссия 9

Заказчик просит упорядочить этапы построения pipeline:

1. запуск исполнения pipeline
2. определение операторов и связей
3. оптимизация плана исполнения
4. подготовка источников и приемников

Подсказка: сначала этап проектирования, затем подготовка, оптимизация и запуск.

Шаг 10:

Миссия 10

Реактивный pipeline пропускает значительную часть событий из-за медленной обработки. Какой механизм поможет справиться?

Подсказка: подумайте о распределении нагрузки или буферизации.

🔗 Шаг 11:

Миссия 11

Вам нужно периодически сохранять состояние агрегатора даже если поток временно пуст. Что выбрать?

Подсказка: состояние — это накопленные данные, подумайте о таймере.

🔗 Шаг 12:

Миссия 12

Сопоставьте подход к агрегированию в потоках с его сценарием:

Метод:

Ответ:

- reduce
- scan
- fold

Сценарий:

1. накапливать результат и эмитить промежуточно
2. накапливать и выдавать итог в конце
3. аналог reduce, но с начальным значением

Подсказка: scan даёт частичные результаты, fold требует начального состояния.

🔗 Шаг 13:

Миссия 13

Сопоставьте режимы обработки времени с поведением:

Режим:

Ответ:

- event time
- ingestion time
- processing time

Поведение:

1. время по событию, может быть старое
2. время поступления в систему
3. текущее реальное время выполнения

Подсказка: event time — от события, processing — когда код реально исполняется.

🔗 Шаг 14:

Миссия 14

Вам нужно упорядочить фазы recovery и checkpointing:

1. **при запуске** — восстановление состояния
2. выполнение бизнес-логики
3. создание контрольной точки (snapshot)
4. запись состояния на диск

Подсказка: порядок: восстановление → бизнес → snapshot → запись.

Шаг 15:

Миссия 15

У вас есть pipeline: source → filter → window → aggregate → sink. Упорядочите шаги:

1. source
2. filter
3. window + grouping
4. aggregate
5. sink

Подсказка: порядок следования операторов фиксирован по логике потока.

Шаг 16:

Миссия 16

При разрешении временных сдвигов в event time наблюдаете late events. Что стоит делать?

Подсказка: подумайте о allowed lateness или водяных метках.

Шаг 17:

Миссия 17

В конце курса: какой основной критерий успешной обработки потока?

Подсказка: подумайте о корректности, производительности и отказоустойчивости.

Урок 4 - Квест автоматический мониторинг событий

Шаг 01:

Миссия 1

Вы пишете мониторинг логов, и система обнаруживает всплеск ошибок. Что вы сделаете первым?

Подсказка: Сначала — оценка, затем — действия.

Шаг 02:

Миссия 2

Вы получили уведомление: утечка памяти растёт в сервисе. Как вы реагируете?

Подсказка: Не сразу перезапуск — сначала диагностика.

Шаг 03:

Миссия 3

Ваша система реагирует на рост задержек API. Что выберете как первую меру?

Подсказка: Снижение нагрузки и трассировка — разумный старт.

Шаг 04:

Миссия 4

Выстроен реактивный поток уведомлений, но он часто дублирует события. Что исправить?

Подсказка: Логика фильтрации важна.

💡 Шаг 05:

Миссия 5

Поток событий перегружается из-за всплесков. Что примените?

Подсказка: Управление потоком — ключ к устойчивости.

💡 Шаг 06:

Миссия 6

Сопоставьте оператор реактивного потока с его назначением:

Операторы:

Ответ:

- map
- filter
- debounce
- buffer

Назначения:

1. Отбрасывать события, если они происходят слишком часто
2. Преобразовать значение каждого события
3. Собирать несколько событий в группу
4. Пропускать события, не удовлетворяющие условию

Подсказка: map — преобразование, filter — проверка условия.

💡 Шаг 07:

Миссия 7

Сопоставьте сценарий из жизни с оператором реактивного потока:

Сценарии:

Ответ:

- Игнорировать повторения кликов
- Сгруппировать нажатия за секунду
- Отфильтровать лишние события по флагу
- Преобразовать ID события в объект

Операторы:

1. map
2. filter
3. distinctUntilChanged
4. buffer

Подсказка: distinctUntilChanged — для удаления одинаковых подряд.

💡 Шаг 08:

Миссия 8

Сопоставьте стратегию с эффектом на поток:

Стратегии:

Ответ:

- пропускать события
- группировать события
- отсеивать шум
- балансировать нагрузку

Эффекты:

1. уменьшение частоты
2. уменьшение числа обработок
3. снижение нагрузки на подписчика
4. агрегация значений

Подсказка: балансировка — про перераспределение нагрузки.

 Шаг 09:

Миссия 9

Сопоставьте понятие и пример в контексте автоматического мониторинга:

Понятия:

Ответ:

- backpressure
- late data
- hot stream
- cold stream

Примеры:

1. Данные приходят с задержкой, уже поздно для анализа
2. Поток активен независимо от подписчиков
3. Подписчик запускает поток при подключении
4. Регулирование скорости обработки событий

Подсказка: cold vs hot зависит от запуска; backpressure — про нагрузку.

 Шаг 10:

Миссия 10

Расположите шаги построения автоматического мониторинга:

1. Создать правила срабатывания
2. Внедрить сбор метрик и логов
3. Подписаться на потоки событий
4. Проанализировать события и отправить сигнал

Подсказка: Сначала собираем, потом подписываемся, затем обрабатываем.

 Шаг 11:

Миссия 11

Расположите этапы реагирования на инцидент в системе мониторинга:

1. Уведомление инженера
2. Активация playbook-инструкции

3. Сбор контекста и логов
4. Закрытие инцидента после проверки

Подсказка: Сначала сбор, потом — уведомление, далее — инструкции и закрытие.

🔔 Шаг 12:

Миссия 12

Отсортируйте уровни приоритета событий (от самого серьёзного к менее серьёзному):

1. Критическая ошибка, система недоступна
2. Утечка памяти, рост емкости
3. Небольшой рост задержек
4. Информационное лог-сообщение

Подсказка: Критичность — сначала, информативность — в конце.

🔔 Шаг 13:

Миссия 13

Упорядочите операции реактивного конвейера:

1. filter
2. map
3. buffer
4. flatMap

Подсказка: Сначала отсекаем, затем преобразуем, потом группируем или раскрываем.

🔔 Шаг 14:

Миссия 14

Расположите по порядку стадии эскалации инцидента:

1. Уведомление команды поддержки
2. Автоматическое реагирование (ремедиэйшн)
3. Эскалация на более высокий уровень
4. Закрытие и ретроспектива

Подсказка: Авто-реакция до эскалации, затем разбор и закрытие.

🔔 Шаг 15:

Миссия 15

Упорядочите модули реагирования в системе мониторинга:

1. executor (выполнение действий)
2. collector (сбор данных)
3. processor (анализ/фильтрация)
4. notifier (уведомление)

Подсказка: Сбор → анализ → уведомление → действие.

🔔 Шаг 16:

Миссия 16

Последовательность отладки системы мониторинга:

1. Симулировать события
2. Проверить маршрутизацию уведомлений
3. Включить логирование уровня debug
4. Сопоставить срабатывания с правилами

Подсказка: Сначала симуляция, потом логирование, далее — маршруты и проверка правил.

Модуль 12 - Модуль 12. Планировщики и таймеры

Урок 1 - Таймеры в threading и asyncio

Шаг 01:

Миссия 1

Анна пишет тестовый скрипт, который должен завершаться через 5 секунд, если нет ответа. Каким методом threading стоит воспользоваться?

Подсказка: В стандартной библиотеке threading есть способ запустить функцию с задержкой.

Шаг 02:

Миссия 2

 Борис хочет отменить таймер, если пользователь нажал «Отмена» до истечения времени. Какой метод нужно вызвать?

Подсказка: Посмотрите, какой метод есть у объекта Timer для остановки.

Шаг 03:

Миссия 3

 Катя запускает несколько таймеров в одном потоке. Что произойдет, если один таймер блокирует поток долгим вычислением?

Подсказка: threading.Timer выполняется в своём потоке, но основное событие — планировщик ждёт в главном потоке.

Шаг 04:

Миссия 4

 Павел изучает методы таймеров. Сопоставьте объект с действием.

Объект:

Ответ:

- threading.Timer
- asyncio.get_event_loop().call_later
- asyncio.get_event_loop().call_at
- asyncio.create_task + asyncio.sleep

Действие:

1. Запланировать колбэк через заданный интервал (от текущего момента)
2. Запустить функцию через таймер в отдельном потоке
3. Запланировать колбэк к точному времени (по абсолютному времени)
4. Создать задачу с искусственной задержкой внутри асинхронного кода

Подсказка: Внимательно подумайте, какой метод ближе к thread-таймеру, а какие — к asyncio.

🔗 Шаг 05:

Миссия 5

📖 Светлана читает документацию и хочет понять, какие аргументы принимает `call_later` / `call_at`. Сопоставьте методы с формой аргументов.

Метод:

Ответ:

- `call_later`
- `call_at`
- `threading.Timer`

Форма аргументов:

1. `delay, callback, *args`
2. `when, callback, *args`
3. `interval, function, args`

Подсказка: Подумайте, что означает «когда» vs «через сколько», и как `Timer` конструируется.

🔗 Шаг 06:

Миссия 6

🔗 Иван хочет понять жизненный цикл задач с таймерами. Сопоставьте событие с моментом времени.

Событие:

Ответ:

- Запланирован вызов
- Запуск функции таймера
- Отмена таймера
- Завершение функции

Момент:

1. До того, как сработает `delay`
2. Когда истечёт время задержки
3. Если пользователь нажал отмену раньше
4. После исполнения `callback`

Подсказка: Отмена может быть раньше срабатывания, а завершение — после запуска.

🔗 Шаг 07:

Миссия 7

Расположите шаги при использовании `threading.Timer`:

1. Запустить таймер
2. Определить функцию-обработчик
3. Вызывать `cancel` (опционально)
4. Функция вызывается по истечении времени

🔗 **Подсказка:**

Подумайте, что нужно сначала: придумать, что делать, потом запустить таймер, возможно отменить, и только потом функция вызовется.

🔔 Шаг 08:

Миссия 8

Расположите последовательность действий при планировании в `asyncio`:

1. Получить `loop`
2. Запланировать `call_later` / `call_at`
3. Запустить цикл событий (`run_until_complete` или `run_forever`)
4. Колбэк выполняется в будущем

🔔 Подсказка:

Сначала нужен `loop`, потом расписание, затем запуск, и только потом — исполнение `callback`.

🔔 Шаг 09:

Миссия 9

🐞 Ваша игра делает «бонус» через 10 секунд после старта уровня. Что лучше использовать в `asyncio`-коде?

Подсказка: Лучше не использовать `threading` внутри `asyncio`, ищите нативный метод.

🔔 Шаг 10:

Миссия 10

Павел хочет запланировать задачу на точное время, например, на 12:00. Какой метод `asyncio` выбрать?

Подсказка: `call_later` работает относительно, а нужен абсолютный момент.

🔔 Шаг 11:

Миссия 11

🐞 Ольга комбинирует `threading.Timer` и `asyncio`. Сопоставьте сценарий с правильным решением.

Сценарий:

Ответ:

- Нужно запустить `callback` через 2 сек из синхронного кода
- Нужно внутри `async` функции запланировать `callback` через задержку
- Нужно отменить ранее запланированный таймер
- Нужно запланировать абсолютное время в `async`-коде

Решение:

1. `threading.Timer(2, ...).start()`
2. `await asyncio.sleep` и затем вызов
3. `timer.cancel()`
4. `loop.call_at(time, callback)`

Подсказка: Учтите, что внутри `coroutine` нельзя просто вызвать `threading` в качестве расписания.

🔔 Шаг 12:

Миссия 12

Расположите шаги, если вы хотите из synchronous кода запустить callback через asyncio спустя время:

1. Получить ссылку на loop (например, `asyncio.get_event_loop()`)
2. Запланировать callback через `loop.call_later` или `call_at`
3. **Если loop ещё не запущен** — запустить его (`run_until_complete` / `run_forever`)
4. **При наступлении времени** — выполнение callback в loop

🔔 Подсказка:

В синхронном коде нужно сначала подготовить loop, затем расписать задачу, и только затем запустить сам цикл.

🔔 Шаг 13:

Миссия 13

В демонстрационной программе Таймер запускается каждые 3 секунды, но Анна хочет, чтобы callback выполнялся повторно. Как её реализовать?

Подсказка: `threading.Timer` по умолчанию однократный; подумайте, как повторить сам вызов.

🔔 Шаг 14:

Миссия 14

🐍 Василий хочет комбинировать `call_later` с `await`. Что важно учесть?

Подсказка: callback, запущенный via `call_later`, не ждёт `await` внутри.

🔔 Шаг 15:

Миссия 15

🔍 Тимур хочет сравнить подходы повторных задач: самостоятельный `Timer`, recursion в callback, и `asyncio.loop.call_later` внутри callback. Сопоставьте подход с преимуществом / недостатком.

Подход:

Ответ:

- `threading.Timer` рекурсивно в callback
- Внутри callback делать `loop.call_later` (для `asyncio`)
- `asyncio tasks` + `sleep` внутри цикла

Преимущество / недостаток:

1. Работает как постоянный цикл в async-коде
2. Может накопить дрейф времени (смещение) при рекурсивном вызове
3. Высокая нагрузка, если много `Timer`'ов в разных потоках

Подсказка: Подумайте, как каждая конструкция влияет на накладные расходы и временные ошибки.

🔔 Шаг 16:

Миссия 16

Расположите шаги для реализации «heartbeat» (сердцебиение) в `asyncio` каждые N секунд:

1. внутри coroutine: `await asyncio.sleep(N)`
2. вызвать нужную работу / callback
3. помещать цикл обратного вызова в `while True`
4. запустить задачу через `asyncio.create_task`

🔔 **Подсказка: Heartbeat** — это постоянное повторение внутри loop: цикл + sleep + действие.

🔔 **Шаг 17:**

Миссия 17

✨ Финал: вы хотите, чтобы одновременно работали таймеры в разных режимах: threading и asyncio, и они не мешали друг другу. Что выберете?

Подсказка: Можно интегрировать event loop с threading, либо держать их независимыми, если задачи разделены.

Урок 2 - События по расписанию

🔔 **Шаг 01:**

Миссия 1

🕒 Вася хочет запланировать задачу, которая должна выполняться каждый день в 9:00. Какой тип планировщика ему нужен?

Подсказка: Ежедневные события — не одноразовый таймер.

🔔 **Шаг 02:**

Миссия 2

✂ Катя хочет запустить функцию через точно 30 секунд. Какой механизм подходит лучше всего?

Подсказка: Временной отрезок — фиксированная задержка.

🔔 **Шаг 03:**

Миссия 3

🗂 В проекте есть множество задач, которые должны запускаться ежечасно. Что удобнее всего использовать?

Подсказка: Регулярный интервал, одинаковый для всех.

🔔 **Шаг 04:**

Миссия 4

📅 Петя хочет запланировать задачу только по будням (вт, ср, чт). Какой элемент расписания нужно указать?

Подсказка: Не все дни — только конкретные дни недели.

🔔 **Шаг 05:**

Миссия 5

Сопоставьте виды планировщиков с их описаниями.

Типы:

Ответ:

- Одноразовый таймер
- Интервальный планировщик
- Расписание по дате/времени
- Дневной/недельный планировщик

Описания:

1. Запускает задачу через заданную задержку один раз
2. Повторяет задачу через одинаковый интервал (каждые N секунд/минут)
3. Запускает задачу в конкретную дату и время
4. Запускает задачу по расписанию типа «каждый день/неделю»

Подсказка: Подумайте: одноразовые — это единичный запуск, расписные — связаны с календарём.

Шаг 06:

Миссия 6

 Сопоставьте термины с их ролями в модуле таймеров.

Термины:

Ответ:

- delay
- schedule
- cron-выражение
- callback

Роли:

1. Задаёт отложенный запуск через заданный промежуток времени
2. Описывает время запуска в формате «каждый день в ...»
3. Выражение с шаблоном «минуты, часы, дни недели»
4. Функция, которая вызывается по событию планировщика

Подсказка: cron-выражения часто используются в планировщиках cron.

Шаг 07:

Миссия 8

 В системе мониторинга есть события: event_a , event_b , event_c , event_d . Сопоставьте каждый с подходящим описанием:

События:

Ответ:

- event_a
- event_b
- event_c
- event_d

Описание:

1. Запуск еженощно в полночь
2. Разовое событие через 5 минут
3. План ежедневной проверки системы
4. Событие по понедельникам

Подсказка: Подумайте, какие из событий регулярны, а какие — единичны.

Шаг 08:

Миссия 9

 Упорядочите этапы при планировании события в расписании:

1. Определить нужный тип (delay/interval/schedule)
2. Указать параметры (время, интервал, дни недели)
3. Привязать callback-функцию
4. Активировать планировщик

Подсказка: Сначала выбираете тип, затем настраиваете детали, потом назначаете функцию и запускаете.

Шаг 09:

Миссия 10

Упорядочите действия при миграции задач из одного планировщика в другой:

1. Остановить старые задачи
2. Сконвертировать параметры расписания
3. Зарегистрировать задачи в новом планировщике
4. Запустить задачи в новом планировщике

Подсказка: Без остановки старых задач возможны дублирования.

Шаг 10:

Миссия 11

 Расставьте шаги при планировании еженедельного отчёта:

1. Выбрать день недели (например, понедельник)
2. Назначить время отправки (например, 9:00)
3. Назначить функцию генерации отчёта
- Включить задачник с расписанием

Подсказка: Сначала календарные параметры, затем функционал, затем активация.

Шаг 11:

Миссия 12

 Упорядочите логику обработки таймера с повторением:

1. Выполнение callback
2. Расчёт следующего времени запуска
3. Регистрация новой задачи
4. Проверка, не отменён ли таймер

Подсказка: После выполнения обратного вызова нужно решить, запускать ли снова.

Шаг 12:

Миссия 13

 Разместите этапы отладки проблем с расписанием:

1. Проверить логи запуска задач
2. Проверить параметры schedule/interval
3. Проверить, вызывается ли callback
4. Проверить, не происходит ли пересечение задач

Подсказка: Начните с предполагаемых причин — логов и параметров.

🔔 Шаг 13:

Миссия 14

❏ Какова правильная последовательность для запуска ежемесячного бэкапа в последний день месяца?

1. Определить последний день месяца
2. Установить расписание на это число в 23:59
3. Назначить функцию бэкапа
4. Активировать планировщик

Подсказка: Логика календаря — самое начало.

🔔 Шаг 14:

Миссия 15

🔧 Упорядочите шаги по реализации динамического расписания, когда сроки могут измениться:

1. Удалить старое расписание
2. Пересчитать новое время по входным данным
3. Зарегистрировать обновлённое событие
4. Логировать изменения

Подсказка: Обновление требует удаления старой и создание новой версии.

🔔 Шаг 15:

Миссия 16

🔧 Упорядочите фазы тестирования планировщика в большом проекте:

1. Юнит-тесты на schedule / delay / interval
2. Интеграционные тесты с реальным временем
3. Нагрузочное тестирование при множественных задачах
4. Мониторинг живого выполнения событий

Подсказка: Сначала мелкое тестирование, затем интеграция, нагрузка и наконец проверка в продакшене.

🔔 Шаг 16:

Миссия 17

🕒 Задача: Упорядочите полный жизненный цикл планируемой задачи:

1. Составление расписания (время, дни, интервалы)
2. Регистрация задачи в планировщике
3. Запуск задачи по расписанию
4. Обработка обратного вызова (callback)
5. Повтор нового запуска или завершение
6. Отключение или отмена задачи (при необходимости)

Подсказка: Это цикл: после обработки — либо повтор, либо завершение.

Урок 3 - Практика фоновый планировщик

Шаг 01:

Миссия 1

Ваша команда хочет запустить задачу по отправке email каждые 10 минут в фоне. Что из перечисленного лучше всего использовать?

Подсказка: Ищите инструмент, который позволяет планировать повторяющиеся задачи.

Шаг 02:

Миссия 2

Вы хотите, чтобы фоновый планировщик работал только в рабочие часы (с 9:00 до 18:00). Какое условие стоит добавить в задачу?

Подсказка: Проверьте текущее время и сравните с границами рабочего дня.

Шаг 03:

Миссия 3

 Сопоставьте компоненты фонового планировщика и их роли:

Компоненты:

Ответ:

- Таймер
- Задача (job)
- Расписание (schedule)
- Обработчик ошибок

Роли:

1. Запускает задачу в нужное время
2. Делает повторяющуюся работу
3. Узнаёт, когда должна стартовать задача
4. Ловит исключения и перезапускает задачу

Подсказка: Таймер непосредственно ждёт момент запуска; расписание содержит информацию о периодичности.

Шаг 04:

Миссия 4

 Расположите шаги создания фонового планировщика в правильном порядке:

1. Определить задачи, которые нужно выполнить периодически
2. Запустить таймер / цикл планировщика
3. Обработать возможные ошибки внутри задач
4. Сообщить планировщику расписание задач
5. Запустить задачу, когда наступит время

Подсказка: Сначала нужно составить список задач и расписание, потом запустить цикл и уже в нём вызывать задачу, обрабатывая ошибки.

Шаг 05:

Миссия 5

✳ Вам нужно планировать разные задачи: одна — каждую минуту, другая — каждый день в полночь. Что важно при выборе планировщика?

Подсказка: Убедитесь, что планировщик поддерживает разные интервалы и cron-подобные выражения.

🔗 Шаг 06:

Миссия 6

Сопоставьте формат написания расписания с его описанием:

Форматы расписания:

Ответ:

- cron строка
- интервал (каждые N секунд / минут)
- дата и время (однократно)
- подсчет времени до запуска

Описание:

1. «в 03:00 каждый день»
2. «каждые 15 минут»
3. «в 2025-10-10 12:00»
4. «запустить через 5 минут»

Подсказка: Cron подходит для сложных расписаний; интервалы — для регулярных повторов; одноразовый запуск — через дату/время или задержку.

🔗 Шаг 07:

Миссия 7

📋 Расположите проверки, которые нужно сделать перед запуском каждой фоновой задачи:

1. Проверка, что предыдущий запуск завершился
2. Проверка допустимого времени (рабочее время, день недели и т.д.)
3. Обработка исключений
4. Запуск задачи по расписанию
5. Логирование начала и конца задачи

Подсказка: Сначала убедитесь, что условия позволяют запуск, затем сам запуск, и в конце логирование и обработка ошибок.

🔗 Шаг 08:

Миссия 8

🔗 У вас есть долгоживущая задача: сбор статистики каждую минуту. Как избежать накопления задач при сбоях?

Подсказка: Подумайте о механизме дедупликации или контроле конкурентных запусков.

🔗 Шаг 09:

Миссия 9

🔗 Сопоставьте ошибки, которые могут возникнуть в фоне, с подходящим решением:

Ошибки:

Ответ:

- Зависание задачи
- Исключение в задаче
- Потеря расписания
- Неправильная конфигурация времени

Решения:

1. Обёртывать задачу в try/except и логировать
2. Установить таймаут на выполнение задачи
3. Сохранять расписание в конфигурации и проверять на старте планировщика
4. Проверять корректность часового пояса и граничных значений времени

Подсказка: Задача может подвиснуть — установите таймаут; исключения — ловите их; расписание и конфигурация надо проверять.

 Шаг 10:**Миссия 10**

Расположите этапы восстановления после сбоя фонового планировщика:

1. Перезагрузить планировщик / сервис
2. Проверить логи ошибок
Проверить расписания, отсутствующие задания
3. Запустить недостающие задачи вручную
4. Убедиться, что все задачи запускаются по расписанию

Подсказка: Сначала устраните причину (логи), затем восстановите расписания и убедитесь, что всё работает.

 Шаг 11:**Миссия 11**

 Нужно плавно обновить код фоновой задачи без остановки планировщика. Что стоит сделать?

Подсказка: Важно поддерживать совместимость новой версии и старой, пока идет миграция.

 Шаг 12:**Миссия 12**

 Сопоставьте стратегию развертывания обновлений с её описанием:

Стратегии:

Ответ:

- Blue-Green deployment
- Canary deployment
- Обновление in-place
- Rolling update

Описание:

1. Запуск новой версии на части серверов для тестирования
2. Поддержка двух стабильных сред и переключение на новую
3. Поэтапное обновление узлов поочередно
4. Замена версии на месте (без переходов)

Подсказка: Canary — экспозиция ограниченному трафику, blue-green — полный параллельный маршрут, rolling — постепенно.

🔮 Шаг 13:

Миссия 13

📦 Расположите шаги по внедрению фоновой задачи в продакшн:

1. Провести нагрузочное тестирование задачи в staging
2. Настроить логирование и мониторинг
3. Запустить задачу сначала с минимальной нагрузкой
4. Проверить стабильность и отклики
5. Постепенно увеличить нагрузку до реального уровня

Подсказка: Начинайте с низкой нагрузки, затем поднимайте, при этом тщательно отслеживая логи и метрики.

🔮 Шаг 14:

Миссия 14

✂ Пришло уведомление: задача, запланированная на каждый час, не сработала. Где начать поиск причины?

Подсказка: Начните с самого низкого уровня: логи задач, затем конфигурацию планировщика.

🔮 Шаг 15:

Миссия 15

🔍 Сопоставьте возможные причины пропуска фоновой задачи и способ их диагностики:

Причины:

Ответ:

- Исключение внутри задачи
- Расписание не загружено
- Задача блокируется дедупликацией
- Сбой таймера / внутреннего цикла

Диагностика:

1. Проверить конфигурацию расписаний на старте сервиса
2. Проверить лог ошибок задачи
3. Проверить механизм блокировки / флага дедупликации
4. Проверить состояние основного цикла / памяти / нагрузку

Подсказка: Если schedule не загружен — задачи просто никогда не запускаются; блокировки часто мешают повторным запускам.

🔮 Шаг 16:

Миссия 16

📅 Расположите этапы динамического изменения расписания задачи (например, меняется каждые N минут на лету):

1. Остановить текущую задачу или интервал
2. Получить новые параметры расписания
3. Вычислить новый интервал / cron выражение

4. Зарегистрировать новую задачу / расписание
5. Убедиться, что она запускается в новом режиме

Подсказка: Изменения нужно применять последовательно: сначала остановка, потом регистрация новой версии.

Шаг 17:

Миссия 17

🚩 Финальное испытание: ваша система с фоновыми задачами должна работать 24/7. Какой подход обеспечит максимальную надёжность?

Подсказка: Используйте сочетание мониторинга, перезапуска и резервных механизмов.

Урок 4 - Квест построй свой мини-срон

Шаг 01:

Миссия 1

Ваш друг-разработчик Аня хочет запланировать задачу на каждую минуту в мини-срон. Какой подход вам выбрать первым?

Подсказка: Подумайте, какое средство в Python может периодически вызывать функцию.

Шаг 02:

Миссия 2

🚩 Вы решили воспользоваться `threading.Timer`, но задача должна повторяться. Что добавить, чтобы срон-подобный планировщик работал?

Подсказка: Одна вызов — недостаточно, нужна рекурсия или цикл.

Шаг 03:

Миссия 3

🚩 Вы применили `threading.Timer` с рекурсивным перезапуском. Но что, если задача долго выполняется и накладывается на следующий запуск?

Подсказка: Подумайте о следующей задаче, когда предыдущая ещё не завершена.

Шаг 04:

Миссия 4

🔗 Сопоставьте понятие планировщика с его описанием.

Понятия:

Ответ:

- `Timer`
- `Sched`
- Асинхронный `loop` (`asyncio`)
- `ThreadPoolExecutor`

Определения:

1. Запускает функции с задержкой, один раз или рекурсивно.
2. Планирует события с учётом времени, позволяет упорядочивать задачи.
3. Асинхронный цикл с отложенными вызовами.
4. Использует пул потоков для запуска задач параллельно.

Подсказка: Подумайте: что делает Scheduler, что делает пул потоков, а что — асинхронно.

Шаг 05:

Миссия 5

🕒 Сопоставьте тип ошибки с причиной в мини-сроч:

Типы ошибок:

Ответ:

- Перекрытие вызовов
- Задержка drift
- Потеря запуска
- Блокировка основного потока

Причины:

1. Время выполнения функции превышает период.
2. Планировщик «уходит вперед» из-за накопленной ошибки.
3. Главный цикл не отвечает из-за долгой работы.
4. Интервал между задачами оказывается больше, пропуская запуск.

Подсказка: Представьте, что запускаете задачу каждую секунду — что может пойти не так?

Шаг 06:

Миссия 6

📅 Расположите этапы организации мини-сроч:

1. Обработка задачи
2. Расчёт времени следующего запуска
3. Ожидание или sleep до времени запуска
4. Запланировать первый запуск

Подсказка: Что нужно сделать сначала, прежде чем ждать?

Шаг 07:

Миссия 7

👤 Вы хотите позволить пользователю изменять расписание на лету. Какая структура данных поможет хранить задачи с временем запуска?

Подсказка: Нужно быстро выбирать следующую задачу.

Шаг 08:

Миссия 8

🔗 Сопоставьте подход с его преимуществом при планировании:

Подходы:

Ответ:

- Статическое расписание
- Динамическое расписание
- Приоритетная очередь
- Спланировать с оговоркой slack

Преимущества:

1. Можно изменять задачи на лету
2. Минимизирует время ожидания
3. Позволяет вставлять задачи не в конец
4. Уменьшает риск дрейфа при вариабельной нагрузке

Подсказка: Статическое расписание заранее жёсткое, динамическое — гибкое.

🔮 Шаг 09:

Миссия 9

Расположите шаги обработки сбоя задачи (retry) в мини-cron:

1. Увеличить счётчик попыток
2. Запланировать повторный запуск с экспоненциальной задержкой
3. Обработать исключение
4. **Если превышен лимит** — пометить как окончательно проваленную

Подсказка: Что делать сразу после пойманного исключения?

🔮 Шаг 10:

Миссия 10

✧ Вы хотите поддержать отложенный запуск задачи через 5 минут после события. Что правильнее использовать?

Подсказка: Это не регулярное расписание, а однократная задержка.

🔮 Шаг 11:

Миссия 11

📖 Сопоставьте стратегию планирования с ситуацией, когда она полезна:

Стратегии:

Ответ:

- Poll-циклы
- Callback в event loop
- Separate thread loop
- Process forked scheduler

Ситуации:

1. Нужно запускать задачи в фоне без блокировки главного интерфейса
2. Уже используется asyncio в приложении
3. Интервал проверки очень короткий (миллисекунды)
4. Нужно независимое выполнение с изолированной памятью

Подсказка: Подумайте, где лучше poll, где лучше поток, где цикл, где отдельный процесс.

🔮 Шаг 12:

Миссия 12

✧ Расположите этапы интеграции планировщика в веб-приложение:

1. Запустить планировщик в фоновом потоке или процессе
2. Передавать задачи из веб-обработчика в очередь планировщика

3. Обрабатывать результат задачи и возвращать статус

4. Остановить планировщик при завершении сервера

Подсказка: Как сервер запускается — должен запуститься планировщик, и при завершении — корректно завершиться.

Шаг 13:

Миссия 13

⚠ Вы хотите избежать drift (накапливания смещения). Какой подход поможет уменьшить drift?

Подсказка: Часто используйте «точное» время, а не просто sleep(period).

Шаг 14:

Миссия 14

🔗 Сопоставьте термин с техникой компенсации дрейфа:

Термины:

Ответ:

- Fixed-offset scheduling
- Catch-up launches
- Skipping missed runs
- Adaptive delay

Техники:

1. Запуск всех пропущенных задач, если длительность велика
2. Установка следующего запуска как базового времени + $N \times \text{period}$
3. Пропускать запуски, если они просрочены
4. Корректировать интервал в зависимости от нагрузки

Подсказка: Подумайте: что значит catch-up, что значит adaptive?

Шаг 15:

Миссия 15

🔗 Расположите уровни при работе мини-сгрупп в распределённой системе:

1. Локальный планировщик на каждом узле
2. Централизованная очередь задач
3. Балансировщик нагрузки
4. Мониторинг состояния задач и логирование

Подсказка: Подумайте: задачи должны поступать централизованно, распределяться, исполняться и контролироваться.

Шаг 16:

Миссия 16

🔗 Вы хотите поддержать отказоустойчивость: при падении узла задачи должны перезапускаться на другом. Что включить?

Подсказка: Нужна система контроля и переназначения задачи.

Шаг 17:

Миссия 17

✧ Финальная миссия! Расположите шаги полной реализации мини-cron с планировщиком, устойчивым к сбоям:

1. Добавить heartbeat и failover
2. Хранить задачи в приоритетной очереди
3. Запускать задачу в отдельном потоке или процессе
4. Рассчитать next_run как базовое время + $N \times \text{period}$
5. Обработка retry и исключений
6. Мониторинг и логирование успехов и ошибок

Подсказка: Сначала подготовка, потом планирование, выполнение, обработка, мониторинг.

Модуль 13 - Модуль 13. Профилирование и измерение производительности

Урок 1 - Модуль timeit и cProfile

Шаг 01:

Миссия 1

🐭 Вася запускает маленький тест функции и хочет измерить её время. Что он должен использовать?

Подсказка: Модуль timeit — специально для измерения времени выполнения.

Шаг 02:

Миссия 2

Катя хочет сравнить два варианта реализации алгоритма, но внутри Jupyter-ноутбука. Что из этого ей удобно использовать?

Подсказка: В ноутбуке удобно встроенное «магическое» измерение.

Шаг 03:

Миссия 3

🐭 Петя использует cProfile для своего скрипта. Что он получает в выводе?

Подсказка: Профилировщик показывает статистику вызовов функций.

Шаг 04:

Миссия 4

✧ Когда timeit запускает функцию несколько раз, зачем это делается?

Подсказка: Чтобы уменьшить влияние флуктуаций времени.

Шаг 05:

Миссия 5

✧ Лена хочет измерить время одного вызова функции без подготовки кода. Какой вызов she should использовать?

Подсказка: timeit.timeit принимает строку и число повторений.

🔧 Шаг 06:

Миссия 6

👤 Иван профилирует функцию. Сопоставьте колонку «Метрика» с её значением.

Метрики:

Ответ:

- ncalls
- tottime
- percall
- cumtime

Значения:

1. Общее время, потраченное только в теле функции (без вызовов подфункций)
2. Среднее время на один вызов (tottime / ncalls)
3. Суммарное время, включая все вложенные вызовы
4. Количество вызовов функции

Подсказка: Помните, что $\text{cumtime} \geq \text{tottime}$ всегда.

🔧 Шаг 07:

Миссия 7

🌟 Наташа записывает профиль и видит функции: foo, bar, baz. Сопоставьте функцию с её вероятным описанием.

Функции:

Ответ:

- foo
- bar
- baz

Описание:

1. Вызов foo часто, но занимает мало времени
2. bar вызывает другие функции и занимает большую часть времени
3. baz вызывается редко, но тяжелый внутри

Подсказка: Посмотрите на отношение tottime и cumtime и число вызовов.

🔧 Шаг 08:

Миссия 8

🕒 Стас записывает данные timeit и cProfile. Сопоставьте инструмент и его ключевое свойство.

Инструменты:

Ответ:

- timeit
- cProfile
- %timeit (IPython)

Свойства:

1. Запускает много раз, усредняет
2. Показывает статистику по функциям

3. Удобно в ноутбуке, автоповтор

Подсказка: %timeit — это вариация timeit для среды IPython.

Шаг 09:

Миссия 9

 Денис исследует вклад функций. Сопоставьте термин и его описание.

Термины:

Ответ:

- Hot spot
- Bottleneck
- Overhead

Описание:

1. Функция, которая потребляет большую часть времени
2. Узкое место в производительности системы
3. Дополнительные расходы на измерения

Подсказка: Overhead — «накладные расходы».

Шаг 10:

Миссия 10

 Юля сопоставляет термины из timeit и cProfile.

Термины:

Ответ:

- number (timeit)
- repeat (timeit)
- cumtime (cProfile)

Описание:

1. Сколько повторов запустить
2. Сколько серий измерений сделать
3. Общее время с вложенными вызовами

Подсказка: number — число запусков одного сеанса timeit.

Шаг 11:

Миссия 11

 Олесь хочет провести сравнение: сначала подготовка среды, потом измерения, потом анализ. Отсортируйте шаги профилирования.

1. Запуск профилировщика cProfile
2. Анализ статистики (ncalls, tottime и др.)
3. Подготовка кода (выбор функций, инструментов)
4. Запуск тестов и исправление ошибок

Подсказка: Сперва нужно подготовиться, потом запустить измерения, потом проанализировать и в конце убедиться, что всё корректно.

🔗 Шаг 12:

Миссия 12

🧩 Вика хочет упорядочить этапы использования timeit для наглядного измерения.

1. Импорт модуля timeit
2. Написание кода setup
3. Вызов timeit.timeit или repeat
4. Интерпретация полученного времени

Подсказка: порядок логичный: настройка, выполнение, анализ.

🔗 Шаг 13:

Миссия 13

🔗 Павел анализирует профиль и хочет оптимизировать. Расположите этапы оптимизации по порядку:

1. Выявление горячей точки (hot spot)
2. Изменение алгоритма или структуры кода
3. Повторное профилирование
4. Тестирование корректного поведения

Подсказка: Оптимизация — итеративный процесс: найти, исправить, проверить, убедиться.

🔗 Шаг 14:

Миссия 14

🔗 Сергей сравнивает два варианта реализации. Расположите шаги A/B-тестирования с timeit:

1. Написать два варианта функции
2. Запустить timeit для варианта А и В
3. Сравнить средние времена
4. Выбрать лучший вариант и внедрить

Подсказка: Сначала код, потом измерения, потом выбор и внедрение.

🔗 Шаг 15:

Миссия 15

🌐 Мария профилирует веб-приложение. Отсортируйте этапы работы с cProfile и анализом:

1. Инструментирование точек входа (например, функций HTTP)
2. Запуск приложения с профилированием
3. Сбор и экспорт статистики
4. Анализ «узких мест» и оптимизация
5. Повторное измерение после изменений

Подсказка: Порядок: подготовка → запуск → сбор → анализ → проверка.

🔗 Шаг 16:

Миссия 16

4 Дмитрий комбинирует инструменты: сначала timeit, потом cProfile, потом сравнение. Расположите эти шаги:

1. Запустить timeit для микрофункций
2. Запустить полный сценарий под cProfile
3. Сопоставить временные отчёты и статистику
4. Определить участки, требующие оптимизации

Подсказка: Начинайте с мелких измерений, затем общий профиль и сводный анализ.

Шаг 17:

Миссия 17

 Финальная миссия: расставьте полную последовательность процесса анализа производительности с timeit и cProfile:

1. Выбрать подозрительные функции
2. Измерить их с timeit (мини-тесты)
3. Запустить скрипт под cProfile
4. Анализ статистики профиля
5. Оптимизировать код
6. Повторять замеры и профилирование
7. Документировать изменения и выносить в продакшн

Подсказка: Процесс итеративен и завершится внедрением изменений.

Урок 2 - Метрики потоков и процессов

Шаг 01:

Миссия 1

Вы пишете многопоточную программу, и один поток сильно медленнее других. Какие метрики стоит первым делом измерить?

Подсказка: Важно выяснить, тратится ли время на вычисления или ожидание.

Шаг 02:

Миссия 2

Вы запускаете процесс-ферму (multiprocessing), и время выполнения сильно варьируется. Что может быть причиной?

Подсказка: Подумайте о дисбалансе нагрузки между процессами.

Шаг 03:

Миссия 3

Ваша команда хочет измерить масштабируемость многопоточного сервера. Какую метрику логичнее всего построить графиком?

Подсказка: Что показывает «линейность» роста при увеличении числа потоков?

Шаг 04:

Миссия 4

При профилировании потоков вы замечаете, что некоторые потоки простаивают. Какой тип ожидания наиболее вероятен?

Подсказка: Подумайте о блокировках, очередях и внешних ресурсах.

Шаг 05:

Миссия 5

Вы анализируете процесс-пул, где задачи быстрее завершаются. Что показание метрики «время ожидания в очереди» говорит вам?

Подсказка: Если это время велико, то сами задачи выполняются быстро — что-то задерживает их старт.

Шаг 06:

Миссия 6

 Сопоставьте метрику с её описанием:

Ответ:

- CPU-время потока
 - Время ожидания блокировки
 - Время ожидания в очереди задач
 - Контекстные переключения
1. Сумма времени, когда поток активно использовал CPU
 2. Время, которое поток провел, ожидая захватить lock
 3. Время с момента подачи задачи до её начала
 4. Число случаев, когда ОС переключила контекст между потоками

Подсказка: Отдельно подумайте про «активное» время и время ожидания.

Шаг 07:

Миссия 7

 Сопоставьте ситуацию и какую метрику она отражает:

Ответ:

- Поток ожидает доступа к разделяемому ресурсу
 - Задача долго не стартует после постановки в очередь
 - Много переключений потоков при малой нагрузке
 - Процессы обмениваются данными чрез очередь
1. Время ожидания блокировки
 2. Время ожидания в очереди задач
 3. Контекстные переключения
 4. Время IPC / время обмена между процессами

Подсказка: Подумайте, что именно отражает «ожидание» или «обмен».

Шаг 08:

Миссия 8

 Сопоставьте метрику и тип узкого места:

Ответ:

- Высокое время ожидания очереди
- Большое время ожидания блокировки
- Много контекстных переключений

- Высокое время IPC / обмен между процессами

1. Узел очередей / диспетчеризации
2. Блокировки / синхронизация
3. Переусложнённая конкуренция за CPU
4. Межпроцессное взаимодействие может быть узким местом

Подсказка: Когда очередь задерживает — это диспетчер; когда lock — синхронизация; коммуникация — IPC.

🔮 Шаг 09:

Миссия 9

🔧 Сопоставьте инструмент профилирования и что он может измерять:

Ответ:

- cProfile (в CPython)
 - thread-profiler / PyInstrument
 - time.perf_counter()
 - multiprocessing.Queue profiling / custom IPC tracing
1. Время исполнения функций и вызовов Python
 2. Где именно в потоке возникает ожидание (lock, I/O и т.д.)
 3. Общее время между метками начала и конца
 4. Время обмена между процессами через очередь

Подсказка: Некоторые инструменты лучше подходят для потоков, некоторые — для процессов, некоторые — для ручного измерения.

🔮 Шаг 10:

Миссия 10

Сопоставьте этап измерения с действием:

Ответ:

- Добавление таймингов внутри функций
 - Запуск профилирования на реальных данных
 - Сбор статистики по времени ожидания
 - Анализ метрик и построение графиков
1. Инструментальный подход (manual instrumentation)
 2. Экспериментальное тестирование (benchmarking)
 3. Сбор метрик ожидания (locks, очереди, IPC)
 4. Интерпретация результатов и визуализация

Подсказка: Подумайте, что сначала — измерить, затем анализировать.

🔮 Шаг 11:

Миссия 11

📊 Сопоставьте тип графика и метрику, которую логичнее всего на нём показать:

Ответ:

- График throughput vs число потоков/процессов
- Гистограмма распределения задержек задач

- Линейный график ожидания блокировок со временем
- График отношения ожидания к CPU-времени

1. Масштабируемость / производительность
2. Смещение latency / задержки задач
3. Тренд блокировок во времени
4. Накопительный вклад ожидания к общему времени

Подсказка: Подумайте, какая метрика логично выглядит как функция от количества потоков, какая — как распределение и т.д.

🔔 Шаг 12:

Миссия 12

📌 Расположите шаги типичной процедуры измерения производительности многопоточной/многопроцессной задачи:

1. Сбор сырой статистики (CPU, ожидания, IPC и др.)
2. Инструментирование: вставка меток времени
3. Запуск тестов / нагрузочных сценариев
4. Постобработка: агрегация и визуализация метрик
5. Интерпретация результатов и поиск узких мест

Подсказка: Сначала нужно подготовить измерения, затем выполнить, потом обработать и интерпретировать.

🔔 Шаг 13:

Миссия 13

📌 Расположите виды ожиданий в порядке от наиболее локализованного к наиболее глобальному:

1. Ожидание локальной блокировки потока
2. Ожидание очереди задач в процессе
3. Ожидание межпроцессного обмена (IPC)
4. Ожидание сетевого I/O

Подсказка: Локальное ожидание ближе к исполнению, глобальные — через границы процессов или сеть.

🔔 Шаг 14:

Миссия 14

📌 Расставьте стадии оптимизации многопоточной программы после измерения:

1. Устранить синхронизационные конфликты / уменьшить lock contention
2. Перераспределить нагрузку между потоками / балансировка
3. Минимизировать IPC-обмены / уменьшить межпроцессные взаимодействия
4. Повторное измерение после изменений
5. Визуализировать изменения и проверить прирост

Подсказка: Оптимизация — это цикл: изменить — измерить — проверить.

🔔 Шаг 15:

Миссия 15

📊 Расположите показатели, которые следует сравнить при переходе от потоков к процессам:

1. Общее время задачи (время от старта до завершения)
2. Время ожидания IPC / обменов между процессами
3. CPU-время в процессах / потоках
4. Накладные расходы на контекстные переключения / переключения между процессами
5. Процент времени, потраченного на ожидание блокировок

Подсказка: Сначала смотри итог, затем разбивай на части — ожидания, накладные расходы и CPU.

🔗 Шаг 16:

Миссия 16

Расположите этапы анализа после сбора метрик многопроцессной/многопоточной системы:

1. Выявление узких мест по времени ожидания и обменов
2. Сравнение метрик между потоковым и процессным подходом
3. Выбор оптимальных масштабов (число потоков/процессов)
4. Проверка на регрессии и контрольные тесты
5. Документирование сделанных изменений и рекомендаций

Подсказка: Анализ — это шаг за шагом: от поиска узких мест к проверке изменений и записи результатов.

📖 Урок 3 - Практика измеряем ускорение

🔗 Шаг 01:

Миссия 1

Антон запустил два варианта функции: версия А и версия В. Время А = 2,4 с, время В = 1,8 с. Что он может сразу утверждать про ускорение версии В относительно А?

Подсказка: отношение времени — основа ускорения.

🔗 Шаг 02:

Миссия 3

🔗 Марк профилирует функцию, замечает, что 80 % времени тратится на чтение файлов, 20 % — на вычисления. Какой максимум ускорения он может ожидать по закону Амдала при идеальном параллелизме вычислительной части?

Подсказка: Закон Амдала: только часть, пригодная к распараллеливанию, ускоряется.

🔗 Шаг 03:

Миссия 4

✂ Юлия запускает код на 8 ядрах, а ускорение оказалось всего 4х. Что может быть причиной низкого ускорения?

Подсказка: Даже при многих ядрах — есть ограничения: синхронизация, часть последовательно работающая.

🔗 Шаг 04:

Миссия 5

📊 Вася просчитал ускорение = 6х на 6 ядрах и утверждает, что у него сверхлинейное ускорение. Что можно возразить?

Подсказка: Ускорение выше числа ядер может быть артефактом кешей, неточностей измерений или эффекта суперлинейности.

🔗 Шаг 05:

Миссия 6

🔍 Вам нужно сопоставить понятия из профилирования с их описаниями.

Понятия:

Ответ:

- Горячая точка (hot spot)
- Инструментарий профилирования (profiler)
- Сэмплинг-профайлер
- Трассирующий профайлер

Определения:

1. Инструмент, фиксирующий время и вызовы функций.
2. Механизм, который периодически прерывает исполнение, чтобы измерить стек вызовов.
3. Участок кода, потребляющий значительную долю времени.
4. Профайлер, записывающий полную трассировку каждого вызова.

🔗 **Подсказка:**

Подумайте, что значит «горячая точка» и как работают разные методы профилирования.

🔗 Шаг 06:

Миссия 7

🔍 Сопоставьте метрики профилирования с их смыслом:

Метрики:

Ответ:

- Wall-clock time
- CPU time
- User time
- System time

Описание:

1. Время, затраченное на системные вызовы операционной системы.
2. Общее фактическое время (реальное время) от начала до конца.
3. Время, потраченное на пользовательский код (не считая ядра ОС).
4. Время ЦП, объединяющее user + system.

🔗 **Подсказка:**

Проверьте разницу между «истинным» временем и временем, проведённым в ЦП.

🔗 Шаг 07:

Миссия 8

🔍 В прогоне параллельной программы мы получили следующие результаты — сопоставьте причину с её эффектом:

Причины:

Ответ:

- Избыточная синхронизация
- Нераспараллеливаемая часть
- Неправильное распределение нагрузки
- Накладные расходы на создание потоков

Эффекты:

1. Потоки простаивают, часть ядер простаивает.
2. Ускорение меньше, чем ожидалось даже на малом числе ядер.
3. На малом числе задач или коротких секциях создание и переключение потоков съедают ресурсы.

 Подсказка:

Сопоставьте каждую причину с наиболее логичным следствием — подумайте, где возникают простои или затраты.

 Шаг 08:

Миссия 9

 В профайлере вы наблюдаете пары (функция, число вызовов). Сопоставьте функции с их возможным поведением:

Функции:

Ответ:

- f()
- g()
- h()
- i()

Описание поведения:

1. Вызывается часто, но быстро выполняется.
2. Редко вызывается, но тяжелая вычислительно.
3. Вызывает другие функции и сам становится «центром».
4. Обертка, делающая над собой минимальную работу.

 Подсказка:

Обратите внимание на соотношение числа вызовов и общего времени — часто маленькие функции встречаются часто.

 Шаг 09:

Миссия 10

Расположите этапы настройки профайлера и сбора данных перед измерением ускорения:

1. Собрать базовый замер последовательного кода
2. Отключить отвлекающие процессы и фоновые задачи
3. Запустить профилирование с нужными параметрами
4. Запустить параллельный вариант и собрать данные
5. Проанализировать результаты ускорения и узкие места

 Подсказка:

Подумайте, что важно сделать перед запуском измерений, и что делать после.

🔔 Шаг 10:

Миссия 11

📖 В правильном порядке расположите шаги, чтобы сравнить две реализации (A и B) по ускорению:

1. Запустить реализацию A и засечь время
2. Запустить реализацию B и засечь время
3. Убедиться, что входные данные одинаковы
4. Вычислить ускорение как отношение времени
5. Повторить прогон несколько раз и усреднить

🔔 Подсказка: Убедитесь, что условия сравнения честные, и результаты стабильны.

🔔 Шаг 11:

Миссия 12

📖 Разместите этапы анализа профиля после сбора данных:

1. Найти места, где время сосредоточено (hot spots)
2. Сопоставить горячие функции с исходным кодом
3. Оценить потенциал распараллеливания этих функций
4. Предложить оптимизации и повторно измерить ускорение

🔔 Подсказка: Сначала — найти горячее, затем — понять, можно ли его распараллелить, и только потом — действовать.

🔔 Шаг 12:

Миссия 13

📖 Представьте, что вы профилировали код параллельно и получили данные: функция A — 40 % времени, B — 30 %, C — 20 %, D — 10 %. Расположите эти функции от наиболее приоритетной к оптимизации до наименее.

1. A
2. B
3. C
4. D

🔔 Подсказка: Оптимизируйте сначала те функции, которые потребляют больше всего времени.

🔔 Шаг 13:

Миссия 14

📖 При комбинировании нескольких оптимизаций: распараллеливание, кэширование, сокращение аллокаций — расположите шаги внедрения в правильном порядке:

1. Сначала минимизировать аллокации и копирования
2. Затем добавить кэширование повторных вычислений
3. После — распараллелить оставшуюся часть
4. Наконец — измерить итоговое ускорение

🔔 Подсказка: Сначала — убрать узкие места, потом — параллелизм, и в конце — проверка.

🔮 Шаг 14:

Миссия 15

📖 Команда Дарьи внедрила три независимых оптимизации (P, Q, R). Расположите порядок тестирования: какой сначала, какой потом:

1. Оптимизация P + Q вместе
2. Оптимизация P одна
3. Оптимизация Q одна
4. Оптимизация P + Q + R все вместе

🔮 **Подсказка: Сначала проверяйте по одному, потом — комбинации, чтобы понять взаимодействия.**

🔮 Шаг 15:

Миссия 16

📖 У вас есть три реализации модуля: α , β , γ . Нужно тестировать их в порядке: сначала базовую, потом улучшения по степени риска. Какой порядок логичнее?

1. α (базовая, проверенная)
2. β (с умеренными оптимизациями)
3. γ (радикальные изменения)

🔮 **Подсказка: Сначала используйте стабильную версию, потом осторожные оптимизации, и в конце смелые эксперименты.**

🔮 Шаг 16:

Миссия 17

📖 Финальная миссия: вы построили пайплайн измерений. Что должно быть последовательно:

1. Написание отчёта с графиками ускорения
2. Повторные прогоны и сбор статистики
3. Профилирование оптимизированного кода
4. Выбор оптимизаций на основе анализа профиля
5. Первичный замер последовательного и параллельного кода

🔮 **Подсказка: Сначала — первичные замеры, затем профилирование, потом оптимизация, повторные прогоны и в конце — отчёт.**

📖 Урок 4 - Квест докажи, что async быстрее

🔮 Шаг 01:

Миссия 1

Анна запускает синхронную функцию, которая делает I/O-операции с сетью. Какой из вариантов наиболее вероятно будет её тормозить?

Подсказка: Подумайте, что блокирует весь поток при ожидании ответа от сети.

🔮 Шаг 02:

Миссия 2

Боб думает: «Если я просто дам всем задачам работать через ``asyncio.sleep``, получится ли ускорить?» Что ответить?

Подсказка: ``asyncio.sleep`` лишь освобождает цикл на время сна.

🔔 Шаг 03:

Миссия 3

Света считает, что ``asyncio`` бесполезен, если все задачи CPU-bound. Верно ли это утверждение?

Подсказка: Подумайте, где именно ``asyncio`` даёт преимущество — в ожидании I/O.

🔔 Шаг 04:

Миссия 4

Игорь хочет профилировать асинхронную программу. Какой инструмент ему подойдёт?

Подсказка: Он должен поддерживать измерение await-точек и тайминги внутри ``async``.

🔔 Шаг 05:

Миссия 5

Мария измерила: одна асинхронная задача делает работу за 2 с, а синхронная — за 2 с. Но при 10 задачах `async` показывает 3 с, синхронно — 20 с. Как объяснить?

Подсказка: Подумайте, как задачи пересекаются по времени ожидания I/O.

🔔 Шаг 06:

Миссия 6

Антон нарисовал таблицу измерений. Нужно сопоставить ситуации с объяснениями.

Ситуации:

Ответ:

- Синхронно, 5 запросов к API
- Async, 5 запросов к API
- Синхронно, 5 задач CPU
- Async, 5 задач CPU

Объяснения:

1. Время почти суммируется, нет перекрытия
2. Задачи конкурируют за CPU, нет выигрыша от `async`
3. I/O-ожидания перекрываются, выигрыш большой
4. Async не помогает, ведь всё время занимает CPU

Подсказка: Проанализируйте, где есть ожидание и где использование CPU.

🔔 Шаг 07:

Миссия 7

Ксения изучает таймлайн выполнения с `async`. Сопоставьте этапы с пояснениями:

Этапы:

Ответ:

- Запуск задач через ``asyncio.gather``
- Выполнение первой `await`
- Контекст переключения на другую задачу
- Возврат к первой задаче

Пояснения:

1. Ожидание I/O внутри задачи
2. Запланирование, что все задачи работают параллельно
3. Цикл переключается, пока ждут I/O
4. Дальнейшее выполнение завершения задачи

Подсказка: Внимательно рассмотрите порядок событий в event loop.

🔮 Шаг 08:

Миссия 8

Павел записал метрики: latency, throughput, concurrency. Сопоставьте понятия с их определениями:

Понятия:

Ответ:

- Latency
- Throughput
- Concurrency

Определения:

1. Количество выполненных задач за единицу времени
2. Одновременное число активных задач
3. Время ответа от начала до конца

Подсказка: Латентность — это задержка, а throughput — пропускная способность.

🔮 Шаг 09:

Миссия 9

Лиза экспериментирует: запускает 3 группы запросов разной сложности. Сопоставьте группы с предполагаемым результатом (отношение async к sync):

Группы:

Ответ:

- Много I/O, мало CPU
- Сбалансированная I/O и CPU
- Почти полностью CPU-bound

Ожидаемые отношения:

1. async намного быстрее
2. async немного быстрее или сравнимо
3. sync немного быстрее или async не даёт прироста

Подсказка: Где меньше ожидания — там минимум выигрыша от async.

🔮 Шаг 10:

Миссия 10

Расположите по возрастанию, какие сценарии получат наибольший выигрыш от async:

1. Чисто CPU-bound задачи
2. I/O + CPU смешанные задачи

3. Чисто I/O-bound задачи
4. Короткие I/O-запросы с минимальным ожиданием

Подсказка: Чем дольше ждёт I/O, тем выше потенциал перекрытия задач.

Шаг 11:

Миссия 11

Ваша задача: упорядочить шаги измерения производительности async-сервиса:

1. Собрать метрики (latency, throughput, concurrency)
2. Запустить нагрузочный тест
3. Проанализировать профили
4. Интерпретировать результаты и сделать вывод

Подсказка: Сначала запускаем тест, потом собираем, потом профилируем, потом выводы.

Шаг 12:

Миссия 12

Упорядочите действия Светы, чтобы доказать, что async быстрее:

1. Запустить sync-версии под нагрузкой
2. Профилировать exec-время и ожидания
3. Написать async-версии тех же задач
4. Сравнить метрики latency и throughput

Подсказка: Сравнить логично после запуска и сбора профилей.

Шаг 13:

Миссия 13

Упорядочите субъекты, которых стоит измерить в программе:

1. await-вызовы к внешним сервисам
2. внутренние CPU-участки
3. очереди задач, задержки переключения
4. общие накладные расходы библиотеки I/O

Подсказка: Измерьте всё от внешних ожиданий до внутренних операций.

Шаг 14:

Миссия 14

Расположите по убыванию ключевые метрики, чтобы показать выигрыш async over sync:

1. Throughput
2. Latency
3. Concurrency
4. CPU utilization

Подсказка: Выигрыш проявляется в пропускной способности и задержке.

Шаг 15:

Миссия 15

Кира записала результаты трёх запусков (sync и async). Упорядочите их по наибольшему относительному выигрышу async:

1. Запуск A: async 100 запросов vs sync
2. Запуск B: async 10 запросов vs sync
3. Запуск C: async + смешанный I/O/CPU vs sync

Подсказка: Чем больше задач и чем больше I/O, тем выше относительный выигрыш.

Шаг 16:

Миссия 16

Упорядочите стадии улучшения производительности async-приложения:

1. Определить узкие места
2. Добавить await-точки или разделить задачи
3. Переписать CPU-интенсивные части в потоки/процессы
4. Сравнить метрики и сделать вывод

Подсказка: Сначала диагностика, потом изменения, потом проверка.

Шаг 17:

Миссия 17

Финальный вызов: расположите полный экспериментальный цикл для доказательства, что async быстрее:

1. Написать синхронную версию
2. Запустить нагрузочное тестирование sync
3. Профилировать sync-версию
4. Написать асинхронную версию аналогичных задач
5. Запустить нагрузочное тестирование async
6. Профилировать async-версию
7. Сравнить метрики и сделать заключение

Подсказка: Это цикл от идеи до доказательства.

Модуль 14 - Модуль 14. Оптимизация и кэширование

Урок 1 - functools.lru_cache

Шаг 01:

Миссия 1

Вы работаете над веб-сервисом, и замечаете, что часто вызываете тяжёлую функцию расчёта, параметры которой повторяются. Что из нижеследующего лучше всего использовать?

Подсказка: Подумайте, как сохранить результаты вызова, чтобы избежать повторной тяжёлой работы.

Шаг 02:

Миссия 2 🤖

Вы уже применили @lru_cache к вашей функции. Что нужно сделать, чтобы кеш работал корректно?

Подсказка: Посмотрите на типы аргументов функций.

🔮 Шаг 03:

Миссия 3 🔍

После внедрения кеша команда предлагает установить максимальный размер кеша равным 1. Как это повлияет на работу?

Подсказка: Подумайте, сколько закешированных элементов будет храниться одновременно.

🔮 Шаг 04:

Миссия 4

Вы хотите, чтобы кеш очищался автоматически при достижении лимита. Какой параметр декоратора `@lru_cache` позволяет это настроить?

Подсказка: В документации есть параметр, обозначающий «максимальный размер кеша».

🔮 Шаг 05:

Миссия 5 ✂

Кажется, кеш переполняется и становится неэффективным. Какой параметр стоит установить в декораторе `@lru_cache` для контроля поведения кеша при переполнении?

Подсказка: Это параметр, отвечающий, можно ли неограниченно увеличивать кеш.

🔮 Шаг 06:

Миссия 6 ↗

Сопоставьте термины, связанные с `@lru_cache`, с их описаниями.

Термины:

Ответ:

- `cache_info`
- `cache_clear`
- `hits`
- `misses`

Описания:

1. Метод для сброса кеша к исходному состоянию
2. Количество попаданий в кеш
3. Количество промахов (когда результат не был в кеше)
4. Информация о состоянии кеша (`hits`, `misses`, `maxsize`, `currsize`)

Подсказка: Методы `cache_info` и `cache_clear` — это методы самого кеша.

🔮 Шаг 07:

Миссия 7

У вас есть функция с декоратором `@lru_cache(maxsize=32)`. Сопоставьте, за что отвечают следующие поля из `cache_info()`.

Поля:

Ответ:

- `maxsize`
- `currsize`
- `hits`
- `misses`

Значения:

1. Текущее количество закешированных записей
2. Максимальный размер кеша
3. Число запросов, когда значение уже было в кеше
4. Число запросов, которые не нашли значения в кеше

Подсказка: Порядок поля и его смысл соответствует их названию.

Шаг 08:

Миссия 8

Вы хотите оценить эффективность кеширования. Сопоставьте формулы с их значениями:

Формулы:

Ответ:

- $\text{hit_rate} = \text{hits} / (\text{hits} + \text{misses})$
- $\text{miss_rate} = \text{misses} / (\text{hits} + \text{misses})$
- $\text{total} = \text{hits} + \text{misses}$
- $\text{currsz} / \text{maxsz}$

Значения:

1. Общее число попыток запросов функции
2. Процент попаданий в кеш
3. Процент промахов кеша
4. Заполненность кеша (доля используемого размера)

Подсказка: Подумайте, что собой представляет каждая формула.

Шаг 09:

Миссия 9

Расположите шаги включения кеширования через `@lru_cache` в вашем проекте.

1. Добавления декоратора `@lru_cache` к функции
2. Проверка, что аргументы функции хешируемы (`immutable`)
3. Запуск тестов, чтобы убедиться, что поведение не изменилось
4. Мониторинг и анализ статистики `cache_info`

Подсказка: Сначала нужно убедиться, что аргументы подходят, затем декорировать, далее тестировать и затем анализировать поведение.

Шаг 10:

Миссия 10

Ваша задача: упорядочить этапы отладки кеша, если он не работает как ожидалось.

1. Проверить, нет ли изменяемых аргументов
2. Изучить статистику `cache_info` (`hits`, `misses`)
3. Добавить логирование входных и выходных значений
4. Сбросить кеш через `cache_clear` и повторить тесты

Подсказка: Логика: сначала проверить данные, затем поведение, затем очистка и повтор.

🔧 Шаг 11:

Миссия 11 ↘

Упорядочите уровни оптимизации, связанные с caching и мемоизацией (от простого к сложному):

1. Вручную хранить кеш в глобальной dict
2. Использовать `@functools.lru_cache` с низким `maxsize`
3. Использовать `@lru_cache` с правильным `maxsize` и анализом статистики
4. Использовать распределённый кеш (например, Redis) для межпроцессного кеширования

Подсказка: От простейшего подхода к более сложным системам.

🔧 Шаг 12:

Миссия 12

Расположите события, которые происходят при первом вызове функции с `@lru_cache`:

1. Происходит «miss», функция вычисляется
2. Результат функции записывается в кеш
3. Возвращается вычисленное значение
4. В `cache_info` увеличивается счётчик `misses`

Подсказка: Внимательно проследите порядок операций при первом вызове.

🔧 Шаг 13:

Миссия 13

Вы зарегистрированы на соревновании по алгоритмам, и функция вычисления числа Фибоначчи используется многократно с одинаковыми аргументами. Какой из вариантов оптимизации наиболее целесообразен?

Подсказка: Фибоначчи — классический пример, выгодный для мемоизации.

🔧 Шаг 14:

Миссия 14 🤖

Допустим, ваши функции вызываются из нескольких потоков. Как `@lru_cache` работает по умолчанию в многопоточном режиме?

Подсказка: Подумайте о безопасности доступа к внутренним структурам кеша.

🔧 Шаг 15:

Миссия 15 🌐

Вы хотите использовать кеширование на уровне веб-сервиса с несколькими процессами. Почему `@lru_cache` может быть недостаточен?

Подсказка: Подумайте, что происходит, если разные процессы держат свой собственный кеш.

🔧 Шаг 16:

Миссия 16 ↗

У вас есть три функции: `f1`, `f2`, `f3`. Они обернуты `@lru_cache` с разными параметрами. Сопоставьте функции с ситуациями:

Функции:

Ответ:

- `f1`: `@lru_cache(maxsize=None)`

- f2: @lru_cache(maxsize=1)
- f3: @lru_cache(maxsize=10)

Ситуации:

1. Очень ограниченное количество повторяющихся параметров, почти всегда одинаковый аргумент.
2. Большое разнообразие аргументов, и не хочется держать бесконечный кеш.
3. Высокое разнообразие аргументов и хочется максимальной гибкости кеша.

Подсказка: maxsize=None означает неограниченный кеш, maxsize=1 — только один элемент.

Шаг 17:

Миссия 17

Расположите сценарии по возрастанию сложности внедрения кеша (от простого к сложному):

1. Одиночный скрипт, один процесс, кеш внутри процесса через @lru_cache
2. Многопоточное приложение, кеш меж потоков через @lru_cache (thread-safe)
3. Многопроцессное приложение, нужно делить кеш между процессами
4. Распределённая система на несколько узлов, кеш через внешнее хранилище (Redis, Memcached)

Подсказка: Подумайте, какие сложности возникают, когда растёт масштаб системы.

Урок 2 - Кэширование результатов задач

Шаг 01:

Миссия 1

Анна написала функцию, вычисляющую факториал большого числа, и запускает её много раз с одинаковыми аргументами. Какой подход поможет ускорить повторные вызовы?

Подсказка: хранение результатов, чтобы не считать заново.

Шаг 02:

Миссия 2

Дмитрий работает над веб-приложением с API. Один и тот же запрос к внешнему сервису часто повторяется. Что логично сделать?

Подсказка: кэшировать ответ, чтобы не делать повторные запросы.

Шаг 03:

Миссия 3

Саша реализует функцию, которая часто вызывается с одинаковыми параметрами в мультипоточном режиме. Что важно учесть при кэшировании?

Подсказка: безопасность доступа к общему хранилищу.

Шаг 04:

Миссия 4

Ирина заметила, что её кэш растёт бесконтрольно. Что разумно добавить?

Подсказка: ограничение размера или срок хранения.

🔮 Шаг 05:

Миссия 5

Василий рассматривает кэш в файловой системе vs in-memory. Когда файловый кэш предпочтительнее?

Подсказка: вспомните: если данных много и память ограничена.

🔮 Шаг 06:

Миссия 6

Марина добавила кэш, но не обновляет его при изменении источника данных. Что может произойти?

Подсказка: устаревшие данные — распространённая проблема.

🔮 Шаг 07:

Миссия 7

Маша описывает методы кэширования, но путается. Сопоставьте название метода с его поведением.

Методы:

Ответ:

- Memoization
- Write-through cache
- Write-back cache
- Read-through cache

Поведение:

1. Сначала записывает в кэш и в базу сразу
2. **При запросе, если нет в кэше** — загружает из базы и возвращает
3. Записывает только в кэш, а изменения позже записывает в хранение
4. Кэширование результатов функций с ключами аргументов

Подсказка: думайте о том, когда происходит запись в базу.

🔮 Шаг 08:

Миссия 8

Пётр рассматривает характеристики кэша. Сопоставьте термин с объяснением.

Термины:

Ответ:

- Cache hit
- Cache miss
- Eviction
- TTL (time to live)

Объяснения:

1. Ключ удаляется из кэша по политике удаления
2. Когда нет значения в кэше и нужно загрузить из оригинального источника
3. Максимальное время, сколько элемент хранится в кэше
4. Когда запрос удалось обслужить из кэша

Подсказка: подумайте, что означает “hit” и “miss”.

🔮 Шаг 09:

Миссия 9

Светлана анализирует кэш-слои. Сопоставьте уровень кэша с примером.

Уровни:

Ответ:

- In-memory cache
- Distributed cache (например, Redis)
- Disk-based cache
- Browser cache

Примеры:

1. Хранение данных в оперативной памяти процесса Python
2. Кэш на уровне клиента в браузере
3. Кеширование в распределённой системе (несколько машин)
4. Запись кэша на диск между запусками

Подсказка: подумайте, какой уровень ближе к пользователю, а какой — к хранилищу.

🔮 Шаг 10:

Миссия 10

Борис вспоминает метрики кэша. Сопоставьте метрику с её значением.

Метрики:

Ответ:

- Hit ratio
- Miss ratio
- Load latency
- Eviction count

Значения:

1. Время на загрузку данных из источника, если нет в кэше
2. Доля обращений, когда кэш не содержал данные
3. Процент обращений, обслуженных кэшем
4. Количество удалённых элементов из кэша

Подсказка: $\text{hit} + \text{miss} = 1$, eviction — это удаление.

🔮 Шаг 11:

Миссия 11

Катя объединяет несколько стратегий кэширования. Сопоставьте стратегию и её преимущество.

Стратегии:

Ответ:

- Cache-aside
- Read-through
- Write-through

- Write-back

Преимущества:

1. Кэш сам загружает данные, если их нет
2. Уменьшенные задержки на чтение при наличии в кэше
3. Запись сразу синхронно в базу и кэш гарантирует консистентность
4. Уменьшение количества операций записи в базу благодаря батчам

Подсказка: подумайте: что выгодно при чтении, что — при записи.

🔮 Шаг 12:

Миссия 12

Олег хочет оценить, как эффективен кэш. Сопоставьте инструмент/приём с тем, что он измеряет.

Инструменты/приёмы:

Ответ:

- Логи hit/miss
- Замер времени ответа API с кэшем и без
- Мониторинг используемой памяти кэша
- График роста числа записей в кэш

Что измеряет:

1. Насколько часто кэш используется вместо источника
2. Влияние кэша на производительность
3. Когда кэш начинает потреблять много памяти
4. Скорость наполнения кэша новыми ключами

Подсказка: думайте, какие данные логически связаны.

🔮 Шаг 13:

Миссия 13

Илья планирует этапы внедрения кэширования в своём модуле. Расположите шаги в логичном порядке:

1. Добавить обращение к кэшу при запросе
2. Разработать стратегию очистки / TTL / LRU
3. Замерить текущие задержки без кэша
4. **Только если нет в кэше** — получить данные из источника и сохранить в кэш
5. **Оценить, как работает кэш** — hit ratio, время

Подсказка: сначала Benchmark, потом код, потом проверка.

🔮 Шаг 14:

Миссия 14

Катя сопровождает систему после интеграции кэша. Расположите действия поддержки в порядке приоритета:

1. Мониторинг hit/miss и времени
2. Очистка или перераспределение кэша при росте
3. Переоценка стратегии записи (write-through / write-back)

4. Логирование аномалий кэша (пропуски, ошибки)
5. Анализ роста памяти кэш-системы

Подсказка: сначала следим, потом действуем.

Шаг 15:

Миссия 15

Александр хочет оптимизировать кэш в многоузловой системе. Какой порядок действий логичен?

1. Выбрать централизованное кэш-хранилище (Redis или др.)
2. Добавить согласованную политику удаления (LRU / TTL)
3. Синхронизировать инвалидацию между узлами
4. Внедрить клиентский слой, использующий распределённый кэш
5. Мониторить метрики распределённого кэша

Подсказка: сначала инфраструктура, потом логика, потом мониторинг.

Шаг 16:

Миссия 16

Ольга тестирует систему с кэшем и без, хочет убедиться, что всё правильно работает. Какой порядок тестов логичен?

1. Проверка корректности без кэша
2. Проверка корректности с кэшем
3. Проверка времени ответа с кэшем vs без
4. Проверка поведения при заполненном кэше и инвалидации
5. Проверка устойчивости при одновременных запросах

Подсказка: сначала базовая функциональность, потом нагрузочные сценарии.

Шаг 17:

Миссия 17

Финальная миссия: как интегрировать кэш в существующий модуль так, чтобы минимизировать риск ошибок? Расставьте шаги:

1. Написать юнит-тесты без учёта кэша
2. Добавить обращения к кэшу в код (read / write)
3. Проверить, что функциональность не сломалась
4. Добавить очистку / TTL / инвалидацию
5. Развернуть на тестовом окружении и мониторить

Подсказка: комбинируйте тест-first и безопасный rollout.

Урок 3 - Практика ускоряем API

Шаг 01:

Миссия 1

У вас API, и вы хотите уменьшить время ответа на часто запрашиваемые данные. Что из нижеперечисленного — самый простой метод кеширования?

Подсказка: Начните с самого базового решения — без внешнего хранилища.

🔧 Шаг 02:

Миссия 2

🔧 Вы решили добавить TTL (время жизни) к кэшу в памяти. Какой параметр выбрать, чтобы не держать старые данные слишком долго?

Подсказка: Слишком большой TTL может привести к устаревшим ответам.

🔧 Шаг 03:

Миссия 3

📦 Ваш API отвечает на запросы, возвращая списки товаров. Какой подход ускорит сборку ответа, если часть данных меняется редко?

Подсказка: Можно кешировать целые фрагменты ответа.

🔧 Шаг 04:

Миссия 4

🔍 Вы заметили, что некоторые ключи кеша пересекаются (разные запросы возвращают похожие данные). Что стоит сделать?

Подсказка: Избегайте дублирования кеша.

🔧 Шаг 05:

Миссия 5

🔥 Вы дошли до горячей точки: API начинает тормозить под нагрузкой — нужно выбрать между кешом и оплатой за дополнительную базу. Что сделать вначале?

Подсказка: Начните с оптимизации до расширения ресурсов.

🔧 Шаг 06:

Миссия 6

Сопоставьте тип кеширования с его характеристикой.

Типы кеша:

Ответ:

- Локальный (in-process) кеш
- Распространённый внешний кеш (Redis, Memcached)
- HTTP-кеш (cache headers, CDN)

Характеристики:

1. Делится между несколькими процессами/серверами
2. Работает внутри текущего процесса, быстрая память
3. Кеш на уровне HTTP, кэширование запросов на краю сети

Подсказка: Подумайте, где хранится кеш и кто его использует.

🔧 Шаг 07:

Миссия 7

Сопоставьте стратегию кеширования с её риском:

Стратегии:

Ответ:

- Неограниченный кеш (без TTL)
- Кеш с небольшим TTL
- Кеш с эвикцией LRU (удаление редко используемых)

Риски:

1. Кеш переполнится или займет много памяти
2. Данные станут устаревшими
3. Удаление нужных данных, если они редко используются, но важны

Подсказка: Стратегия кеша влияет на баланс между свежестью и памятью.

 Шаг 08:**Миссия 8**

Сопоставьте операцию с её кеш-эффектом:

Операции:

Ответ:

- Обновление записи в базе
- Удаление записи
- p>
- Запрос GET со свежими данными

Эффекты:

1. Нужно инвалидировать или обновить кеш
2. Кеш может хранить устаревшие данные, если не обновить
3. Кеш может быть использован, если версия актуальна

Подсказка: Подумайте: запись изменилась, значит кеш надо синхронизировать.

 Шаг 09:**Миссия 9**

Сопоставьте подход к кешированию с его применимой задачей:

Подходы:

Ответ:

- Кеширование на уровне функций (декораторы)
- Промежуточный кеш (middleware, interceptor)
- Кеш на уровне базы данных (materialized views)

Задачи:

1. Кешировать тяжёлые вычисления внутри приложения
2. Кешировать часто читаемые запросы к БД с обновлением по расписанию
3. Кешировать ответы API до обработки основного кода

Подсказка: Где должен располагаться кеш — перед функцией, между слоями или после БД.

 Шаг 10:**Миссия 10**

🔗 Расположите шаги внедрения кеширования в API в правильном порядке:

1. Проверка и мониторинг эффективности кеша
2. Выбор функций или путей API для кеширования
3. Добавление логики кеширования и инвалидирования
4. Запуск тестов и контроль правильности работы

Подсказка: Сначала определите, что кешировать, затем реализуйте и протестируйте.

🔗 Шаг 11:

Миссия 11

📋 Расположите шаги профилирования API до и после кеша:

1. Сбор начальных метрик (время ответа, нагрузка)
2. Внедрение выбранного кеша
3. Сбор новых метрик
4. Сравнение и анализ результатов

Подсказка: Профилирование — это до, внедрение, после, анализ.

🔗 Шаг 12:

Миссия 12

✂ Расположите слои архитектуры, где может быть применён кеш (снаружи внутрь):

1. HTTP / CDN / прокси
2. API-слой (ответы, контроллеры)
3. Бизнес-логика / функции
4. Данные / материализованные представления

Подсказка: Кеширование может работать на каждом уровне от входа до данных.

🔗 Шаг 13:

Миссия 13

⚙ Расположите шаги эвикции кеша по стратегии LRU:

1. Отслеживание доступа к элементам
2. Определение наименее используемого элемента
3. Удаление этого элемента
4. Вставка нового элемента

Подсказка: LRU — это удаление наименее недавно используемого.

🔗 Шаг 14:

Миссия 14

📋 Расположите по порядку стадии отката кеша при ошибке:

1. Возврат данных из базы без кеша
2. Попытка чтения кеша
3. Пропуск кеша и сбор данных с нуля
4. Обновление кеша, если данные получены успешно

Подсказка: Сначала пробуем взять из кеша — если не получилось, то идём дальше.

Шаг 15:

Миссия 15

♥ Расставьте шаги при согласовании кеша и обновлений:

1. Обнаружить изменение данных
2. Инвалидировать соответствующий кеш
3. Обработать новый запрос к базе / обновление записи
4. Пересоздать кеш (если нужно) или оставить пустым

Подсказка: Обновление должно сопровождаться сбросом устаревшего кеша.

Шаг 16:

Миссия 16

🔍 Расположите этапы A/B-тестирования кеширования:

1. Запуск контрольной (без кеша) и экспериментальной группы (с кешем)
2. Сбор метрик (время ответа, ошибки, нагрузка)
3. Анализ статистики и принятие решения
4. Развёртывание победившего варианта на всех пользователей

Подсказка: A/B-тест — как обычный эксперимент: сначала тест, затем анализ, затем масштаб.

Шаг 17:

Миссия 17

🎯 Финальная миссия: Расположите этапы полной оптимизации API с кешированием и откатами:

1. Сбор исходных метрик без кеша
2. Добавление локального кеширования функций
3. Добавление внешнего кеша (Redis) и CDN
4. Инвалидирование при изменении данных
5. Мониторинг, A/B-тест и откат при необходимости
6. Полное развёртывание улучшений

Подсказка: Пройдите от простого к сложному, с проверками и возможностью вернуть назад.

Урок 4 - Квест минимизируй время выполнения

Шаг 01:

Миссия 1

🖥️ Вы профилировали код и обнаружили, что функция чтения данных расходует 90 % времени. Что вы выберете первым?

Подсказка: Не сразу переписывать всё — начните с небольших улучшений.

Шаг 02:

Миссия 2

🔗 Вы хотите ускорить API-метод, который делает дорогой запрос в базу. Какой подход выбрать первым?

Подсказка: Часто внешние запросы — узкое место.

🔮 Шаг 03:

Миссия 3

💡 Хотите минимизировать накладные расходы при многократных вызовах функции с одинаковыми параметрами. Что разумно сделать?

Подсказка: Мемоизация часто помогает.

🔮 Шаг 04:

Миссия 4

⚡ Внезапно вы видите, что десериализация JSON занимает значительное время. Что попробуете первым?

Подсказка: Формат обмена может быть не самый лёгкий.

🔮 Шаг 05:

Миссия 5

📁 Вы заметили, что при параллельной обработке файлов IO очереди блокируются. Что логично предпринять?

Подсказка: IO-bound задачи лучше выполнять иначе, чем CPU-bound.

🔮 Шаг 06:

Миссия 6

🔍 Сопоставьте технику оптимизации с её главной сферой применения:

Метод:

Ответ:

- Кэширование
- Мемоизация
- Пул потоков / потоков
- Асинхронный ввод-вывод

Сфера:

1. Когда функция вызывается много раз с одинаковыми аргументами
2. Когда много мелких сетевых или файловых операций
3. Когда результат дорогой операции сохраняется
4. Когда задача ограничена вводом-выводом

Подсказка:

Подумайте, что делает мемо — хранит результаты для тех же параметров. А кэш — более общий механизм.

🔮 Шаг 07:

Миссия 7

⚙️ Сопоставьте термин с проблемой, которую он решает:

Термины:

Ответ:

- Thundering herd
- Cache stampede
- Lock contention

- IO starvation

Проблемы:

1. Большое количество потоков ждёт один ресурс
2. Большой всплеск запросов к кэшу одновременно
3. Когда IO не получает время из-за других задач CPU
4. Когда много клиентов «пробуждаются» одновременно

🔗 **Подсказка:**

Подумайте, что означает «stampede» и «herd», и к какому ресурсу относятся проблемы конкуренции.

🔗 **Шаг 08:**

Миссия 8

🔗 Сопоставьте библиотеку/инструмент с тем, что она даёт в оптимизации:

Инструмент:

Ответ:

- cProfile
- line_profiler
- memory_profiler
- pytest-benchmark

Результат:

1. Профилирование памяти по строкам
2. Профилирование времени выполнения
3. Бенчмарки тестов (сравнение версий)
4. Профилирование всего приложения

Подсказка: cProfile даёт общий профиль, а line_profiler — построчный.

🔗 **Шаг 09:**

Миссия 9

🔗 Сопоставьте метрику с тем, что она измеряет:

Метрика:

Ответ:

- Latency
- Throughput
- Requests per second (RPS)
- CPU utilization

Что измеряет:

1. Процент времени, загружен ли CPU
2. Количество запросов, обработанных за единицу времени
3. Задержку ответа на один запрос
4. Общее число запросов в секунду

🔗 **Подсказка:**

Latency — это задержка одного запроса, throughput — пропускная способность.

🔮 Шаг 10:

Миссия 10

🔧 Расположите этапы цикла оптимизации в правильном порядке:

1. Измерение начальной производительности
2. Профилирование узких мест
3. Внедрение изменений оптимизации
4. Проверка, что поведение не изменилось
5. Повторение цикла

🔮 Подсказка:

Без измерения не узнаешь, что оптимизировать. А после изменений снова нужно проверить.

🔮 Шаг 11:

Миссия 11

Расположите способы кеширования по росту сложности реализации:

1. Простой словарь в памяти
2. LRU-кэш из стандартной библиотеки
3. Кеширование на диск (файлы)
4. Внешний кеш: Redis / Memcached
5. Гибридный кеш с уровнем памяти + внешний

🔮 Подсказка: Простые подходы — быстрее внедрить, внешние системы требуют настроек.

🔮 Шаг 12:

Миссия 12

⚙️ Расположите шаги борьбы с cache stampede:

1. Установка спин-локов или mutex вокруг заполнения кеша
2. Использование когнитивного задерживания (jitter) на обновление
3. Проверка текущего значения в кеше перед обновлением
4. Обновление кеша только одним из конкурирующих процессов
5. Возврат старого значения пока кеш не обновился

🔮 Подсказка: Нужно минимизировать одновременные обновления и гарантировать корректность.

🔮 Шаг 13:

Миссия 13

🏢 Расположите подходы к параллелизму по характеру задач:

1. Multiprocessing (процессы)
2. ThreadPoolExecutor (потoki)
3. AsyncIO
4. Joblib / concurrent.futures с распараллеливанием
5. Разбиение на микросервисы

🔮 Подсказка: Разные подходы — под разные виды задач: CPU, IO, распределённость.

🔮 Шаг 14:

Миссия 14

📍 Расположите этапы адаптивной оптимизации кеша:

1. Сбор метрик использования ключей
2. Анализ «горячих» ключей
3. Решение, какие ключи кэшировать
4. Очистка или сброс редко используемых ключей
5. Корректировка TTL и политики кеша

🔮 Подсказка: Нужно сначала понять, что часто запрашивают, а что редко — и действовать.

🔮 Шаг 15:

Миссия 15

📍 Расположите виды ожиданий, влияющие на производительность, от наименее к самому тяжёлому:

1. active spinning / busy wait
2. yield / короткий sleep
3. условное ожидание (condition, mutex) блокировки
4. IO wait (длительное ожидание дисков / сети)

🔮 Подсказка: Busy wait жрёт CPU, IO wait — просто ждёт ресурс.

🔮 Шаг 16:

Миссия 16

📍 Перед вами список оптимизаций; расположите их по убыванию эффекта в вашем приложении (гипотетически):

1. Переписать критичный цикл на C / Cython
2. Добавить кэширование часто запрашиваемых данных
3. Удалить лишние проверки и логирование в горячем пути
4. Использовать асинхронный ввод-вывод там, где возможно
5. Оптимизировать алгоритм (уменьшить сложность)

🔮 Подсказка: Оптимизация алгоритма часто даёт самый большой выигрыш, затем кэширование и т.д.

🔮 Шаг 17:

Миссия 17

📍 Расположите подходы к минимизации времени выполнения сервиса в порядке предпочтения:

1. Оптимизация алгоритмов
2. Кэширование / мемоизация
3. Параллелизм / многопроцессность / многопоточность
4. Асинхронные вызовы и неблокирующий IO
5. Расширение инфраструктуры (масштабирование, шардирование)

🔮 Подсказка: Сначала экономьте внутри, затем масштабируйте инфраструктуру.

Модуль 15 - Модуль 15. Очереди задач (Task Queue)

Урок 1 - Celery и Redis

Шаг 01:

Миссия 1

 Анна запускает задачу через Celery с брокером Redis, но задача не стартует. Какое из следующих действий стоит проверить первым?

Подсказка: Ошибки часто связаны с подключением к брокеру.

Шаг 02:

Миссия 2

Иван хочет, чтобы результат задачи записывался в базу, но он не хочет ждать выполнения. Какой тип задачи Celery ему подойдёт?

Подсказка: Подумай, как не блокировать основной поток.

Шаг 03:

Миссия 3

 При деплое серверов, Сергей настраивает совместное использование очередей между машинами. Что он должен настроить в конфигурации Celery?

Подсказка: Все воркеры должны видеть один брокер и одну очередь.

Шаг 04:

Миссия 4

 Мария хочет ограничить одновременное выполнение задач в определённой очереди (rate limiting). Что из этого подходит?

Подсказка: Celery поддерживает лимиты через сигнатуры.

Шаг 05:

Миссия 5

 Пётр наблюдает, что некоторые задачи повторяются. Как в Celery называется механизм, который автоматически запускает задачу по расписанию?

Подсказка: Имя напоминает cron.

Шаг 06:

Миссия 6

 Сопоставьте компоненты Celery с их назначением:

Компоненты:

Ответ:

- Брокер
- Воркер
- Beat (scheduler)
- Результат backend

Назначения:

1. Запускает задачи
2. Хранит очередь сообщений
3. Управляет периодическими задачами
4. Сохраняет результаты выполнения

Подсказка: Подумайте, что делает каждая часть системы.

Шаг 07:

Миссия 7

 Сопоставьте тип ошибки с возможной причиной в Celery + Redis:

Ошибки:

Ответ:

- ConnectionError
- TimeoutError
- TaskRevokedError
- DuplicateTaskError

Причины:

1. Неверные хост/порт Redis
2. Слишком долго выполнение задачи
3. Задача была отозвана вручную
4. Задача с таким ID уже существует

Подсказка: Обратите внимание на содержание ошибок и их смысл.

Шаг 08:

Миссия 8

 Сопоставьте команды Celery CLI с их назначением:

Команды:

Ответ:

- celery -A proj worker
- celery -A proj beat
- celery -A proj purge
- celery -A proj inspect active

Назначения:

1. Запустить обработчик задач
2. Запустить планировщик периодических задач
3. Очистить очередь задач
4. Посмотреть активные задачи

Подсказка: Названия команд подсказывают их действие.

Шаг 09:

Миссия 9

📖 У Ксении есть последовательность этапов для настройки Celery + Redis. Расположите шаги в правильном порядке:

1. Запустить Redis сервер
2. Записать конфигурацию Celery (broker, backend)
3. Запустить воркер Celery
4. Отправить задачу на выполнение

Подсказка: Сначала надо подготовить инфраструктуру.

🔮 Шаг 10:

Миссия 10

🕒 Иван настраивает механизм retry: упорядочьте этапы обработки сбоя задачи:

1. Задача падает с исключением
2. Celery отмечает задачу для повторения
3. Через заданный интервал задача перезапускается
4. Если исчерчены попытки, логируем окончательный сбой

Подсказка: Подумайте, как срабатывают попытки и как фиксировать фатальный исход.

🔮 Шаг 11:

Миссия 11

🔗 Павел хочет установить порядок обработки задач: упорядочите события в lifecycle задачи Celery:

1. Task received
2. Task started
3. Task succeeded (или failed)
4. Task acknowledged

Подсказка: Какие шаги происходят между получением и завершением задачи?

🔮 Шаг 12:

Миссия 12

📦 Ольга хочет, чтобы при раскатывании обновлений, текущие задачи успели завершиться до перезапуска. Упорядочьте шаги:

1. Остановить приём новых задач
2. Дождаться завершения текущих задач
3. Выполнить миграции и обновление кода
4. Запустить воркеры заново

Подсказка: Важно не потерять запущенные задачи.

🔮 Шаг 13:

Миссия 13

⚡ Алексей хочет распараллелить работу над веб-скрапингом, используя Celery. Какой pattern он может выбрать?

Подсказка: Соотнеси вопрос к массовым задачам.

🔮 Шаг 14:

Миссия 14

🔪 Сопоставьте типы цепочек Celery (canvas) с ситуациями:

Типы:

Ответ:

- chain
- group
- chord
- map

Сценарии:

1. Сначала очистка, потом анализ, потом отчет
2. Отправка множества гео-задач параллельно
3. Сбор результатов после параллельных задач и дальнейшая финализация
4. Применение функции к каждому элементу списка

Подсказка: «Chord» — это группа + callback.

🔮 Шаг 15:

Миссия 15

Сопоставьте особенности Redis как брокера с их эффектом на Celery:

Особенности:

Ответ:

- Поддержка списков (lists)
- Pub/Sub модель
- Высокая скорость доступа в памяти
- Поддержка ключей с истечением (TTL)

Эффекты:

1. Позволяет быстро доставлять задачи в очередь
2. Обеспечивает автоматический сброс старых записей
3. Поддержка каналов сообщений
4. Использование списков как структуры очереди

Подсказка: Redis комбинирует структуры данных и модель pub/sub.

🔮 Шаг 16:

Миссия 16

🔪 В сценарии с Celery canvas упорядочите этапы выполнения chord:

1. Запуск группы задач
2. Сбор их результатов
3. Вызов callback после сбора
4. Возврат финального результата

Подсказка: chord = group + callback.

🔧 Шаг 17:

Миссия 17

Финальный практический сценарий: упорядочите этапы работы распределённого приложения с Celery и Redis:

1. Клиент отправляет задачу через broker
2. Воркер получает задачу из очереди Redis
3. Воркер выполняет задачу и записывает в backend
4. Клиент запрашивает результат задачи из backend
5. Backend возвращает результат клиенту

Подсказка: Подумай, как данные движутся от клиента к воркеру и обратно.

📖 Урок 2 - Отложенные задачи и retry

🔧 Шаг 01:

Миссия 1

Программист Алексей использует Celery для фоновых задач. Он хочет, чтобы задача выполнялась не сразу, а через 10 секунд. Какой параметр он должен указать при вызове задачи?

Подсказка: Подумай о параметре, который задаёт задержку перед выполнением задачи.

🔧 Шаг 02:

Миссия 2

🕒 Разработчица Настя отправила задачу с параметром `eta=datetime.utcnow() + timedelta(minutes=5)`. Что произойдёт?

Подсказка: Параметр `eta` задаёт точное время старта задачи.

🔧 Шаг 03:

Миссия 3

🐞 Дима написал задачу, которая может упасть из-за сетевых ошибок. Как ему лучше реализовать повтор выполнения (`retry`)?

Подсказка: В Celery есть специальный метод, вызываемый внутри задачи при ошибке.

🔧 Шаг 04:

Миссия 4

⚙️ Инженер Саша хочет, чтобы при каждой неудаче Celery автоматически делал до 5 попыток. Какой параметр нужно задать?

Подсказка: Этот параметр указывается в декораторе `@task` или в конфигурации.

🔧 Шаг 05:

Миссия 5

🕒 Разработчик Паша хочет, чтобы между повторными попытками была задержка 30 секунд. Какой параметр ему нужен?

Подсказка: Этот параметр часто используется вместе с `max_retries`.

🔮 Шаг 06:

Миссия 6

🔮 У Леры задача выполняется с ошибкой, но она не хочет, чтобы она автоматически повторялась. Как это отключить?

Подсказка: Проверь параметр `retry` в декораторе задачи.

🔮 Шаг 07:

Миссия 7

Сопоставь параметры Celery с их назначением.

Подсказка: Подумай, какие из них управляют временем и какие — повторными попытками.

🔮 Шаг 08:

Миссия 8

🔮 Разработчик Илья анализирует параметры `retry`. Сопоставь параметр с его действием.

Подсказка: Вспомни, какой отвечает за количество, а какой — за интервалы.

🔮 Шаг 09:

Миссия 9

Сопоставь ситуации с подходящими параметрами Celery.

Подсказка: Некоторые задачи важнее других — подумай, где нужны задержки или строгие лимиты.

🔮 Шаг 10:

Миссия 10

⚙️ Сопоставь тип ошибок с действиями Celery при `autoretry_for`.

Подсказка: Подумай, какие ошибки стоит повторять, а какие — нет.

🔮 Шаг 11:

Миссия 11

🔮 Сопоставь реальные ситуации с применением отложенных задач.

Подсказка: Отложенные задачи часто используют для рассылок, отчётов и напоминаний.

🔮 Шаг 12:

Миссия 12

💬 Сопоставь параметр `retry_*` с его поведением в Celery.

Подсказка: Все они связаны с повторными попытками, но управляют разными аспектами — временем, лимитом, экспонентой.

🔮 Шаг 13:

Миссия 13

Расставь шаги создания отложенной задачи в правильном порядке.

Подсказка: Сначала создаётся задача, потом указываются параметры времени, и только потом она отправляется в очередь.

🔧 Шаг 14:

Миссия 14

⚙️ Упорядочи этапы обработки повторных попыток задачи Celery.

Подсказка: Сначала происходит ошибка, потом вычисляется задержка, затем выполняется повтор.

🔧 Шаг 15:

Миссия 15

🕒 Упорядочь шаги настройки автоматического retry в задаче.

Подсказка: Настройка параметров должна предшествовать объявлению задачи.

🔧 Шаг 16:

Миссия 16

📋 Расставь в правильном порядке шаги при выполнении отложенной задачи Celery.

Подсказка: Подумай, что делает Celery до, во время и после исполнения задачи.

📖 Урок 3 - Практика создаём очередь задач

🔧 Шаг 01:

Миссия 1

👩 Разработчица Лена создаёт простую систему очереди задач. Она хочет, чтобы задачи выполнялись асинхронно. Какой инструмент Python подойдёт лучше всего?

Подсказка: Подумай про стандартные библиотеки, поддерживающие конкурентность.

🔧 Шаг 02:

Миссия 2

Программист Игорь тестирует очередь задач. Он замечает, что при добавлении задач из нескольких потоков всё ломается. Что ему нужно использовать для безопасной работы?

Подсказка: Очереди в Python из модуля queue уже потокобезопасны.

🔧 Шаг 03:

Миссия 3

👤 Стас решил измерить, сколько задач осталось в очереди. Какой метод очереди ему поможет?

Подсказка: Этот метод возвращает текущее количество элементов.

🔧 Шаг 04:

Миссия 4

👩 Маша хочет, чтобы поток-воркер ждал, пока появятся новые задачи в очереди. Какой метод следует использовать для извлечения задачи?

Подсказка: Этот метод блокирует поток, пока очередь пуста.

🔧 Шаг 05:

Миссия 5

🕒 Фрилансер Олег пишет обработчик задач. Он боится, что программа зависнет, если очередь пуста. Как можно задать ограничение по времени при ожидании задачи?

Подсказка: В методе `get()` можно указать специальный параметр.

🔊 Шаг 06:

Миссия 6

📖 Алекс добавляет результаты работы обратно в очередь, но она переполняется. Какое решение правильное?

Подсказка: Очередь может иметь ограничение по размеру — параметр `maxsize` .

🔊 Шаг 07:

Миссия 7

⚙️ Соотнесите методы очереди с их назначением.

Подсказка: Один отвечает за добавление, другой — за извлечение.

🔊 Шаг 08:

Миссия 8

Команда стартапа делит роли между потоками. Соотнесите поток и его задачу.

Подсказка: Продюсер создаёт задачи, а потребитель обрабатывает.

🔊 Шаг 09:

Миссия 9

📦 Разные типы очередей используются в разных случаях. Соотнесите тип очереди и её поведение.

Подсказка: FIFO — как очередь в магазин, LIFO — как стопка книг.

🔊 Шаг 10:

Миссия 10

Сопоставьте функцию и её основное назначение при работе с потоками и очередями.

Подсказка: Поток — это объект из `threading` , а очередь — из `queue` .

🔊 Шаг 11:

Миссия 11

Соотнесите ситуацию с правильным методом очереди.

Подсказка: Когда очередь полна — помогает `full()` , когда пуста — `empty()` .

🔊 Шаг 12:

Миссия 12

🧠 Настя проектирует мини-сервис обработки изображений. Соотнеси действие с правильной частью системы очереди.

Подсказка: Очередь связывает продюсеров и потребителей.

🔊 Шаг 13:

Миссия 13

Расставь шаги по созданию простой очереди задач с потоками в правильном порядке.

Подсказка: Сначала создаётся очередь, потом — потоки.

Шаг 14:

Миссия 14

⚙️ Расположи этапы обработки задачи в очереди.

Подсказка: После получения задачи её нужно выполнить и отметить завершение.

Шаг 15:

Миссия 15

Упорядочь действия при использовании PriorityQueue .

Подсказка: Приоритет обычно указывается первым элементом кортежа.

Шаг 16:

Миссия 16

Расположи шаги создания пула потоков с очередью для обработки задач.

Подсказка: Сначала создаём очередь, потом — несколько потоков.

Урок 4 - Квест асинхронная рассылка писем

Шаг 01:

Миссия 1

👥 Команда разработчиков создает сервис для рассылки писем клиентам. Им нужно выбрать подход для выполнения множества задач одновременно. Что им поможет?

Подсказка: Асинхронность — ключ к масштабируемости.

Шаг 02:

Миссия 2

Тимлид объясняет: «Очередь задач — это инструмент для...» Что он имеет в виду?

Подсказка: Очередь помогает управлять задачами, а не просто хранить данные.

Шаг 03:

Миссия 3

👩 Разработчик Кира замечает, что отправка писем блокирует основной поток. Как ей исправить ситуацию?

Подсказка: Вспомните про asyncio и отдельные процессы.

Шаг 04:

Миссия 4

⚙️ Команда внедряет Celery. Что в первую очередь нужно настроить для работы очереди?

Подсказка: Без брокера задач Celery просто не запустится.

Шаг 05:

Миссия 5

📧 В проекте письма иногда не доходят до получателя. Что стоит проверить в первую очередь?

Подсказка: Иногда проблема не в коде, а в брокере.

🔮 Шаг 06:

Миссия 6

👤 Разработчик хочет, чтобы задачи не терялись при сбоях. Что ему стоит использовать?

Подсказка: Думайте о надежности и подтверждении доставки.

🔮 Шаг 07:

Миссия 7

💬 Сервис должен обрабатывать десятки тысяч писем. Что поможет масштабировать систему?

Подсказка: Масштабирование — это не только код, но и архитектура.

🔮 Шаг 08:

Миссия 8

⚡ Сопоставьте основные элементы системы очередей задач с их назначением.

Подсказка: Вспомните, кто отправляет, кто получает и кто управляет задачами.

🔮 Шаг 09:

Миссия 9

Сопоставьте библиотеку с её типичным назначением при работе с очередями.

Подсказка: Celery — не единственная библиотека для задач!

🔮 Шаг 10:

Миссия 10

Сопоставьте тип задачи с подходящим способом выполнения.

Подсказка: Не все задачи требуют асинхронности — думайте про длительность.

🔮 Шаг 11:

Миссия 11

⚡ Сопоставьте команды Celery с их назначением.

Подсказка: Каждая команда выполняет конкретную роль — вспомните их порядок запуска.

🔮 Шаг 12:

Миссия 12

📦 Сопоставьте тип брокера сообщений с его особенностью.

Подсказка: Вспомните, чем Redis отличается от RabbitMQ и Kafka.

🔮 Шаг 13:

Миссия 13

⚡ Расположите шаги создания асинхронной рассылки писем в правильном порядке.

Подсказка: Сначала настройка брокера, потом — код.

🔮 Шаг 14:

Миссия 14

📋 Расположите этапы обработки задачи в очереди Celery.

Подсказка: Вспомните путь: задача — брокер — воркер — результат.

🔔 Шаг 15:

Миссия 15

📖 Расположите шаги настройки периодической рассылки в Celery.

Подсказка: Beat — это планировщик, который знает, когда отправить задачу.

🔔 Шаг 16:

Миссия 16

📖 Расположите действия для обеспечения надежности очереди.

Подсказка: Подумайте о подтверждениях, ретраях и логах.

📖 Модуль 16 - Модуль 16. Распределённые вычисления

📖 Урок 1 - Основы Dask и Ray

🔔 Шаг 01:

Миссия 1

Программист Алекс решил ускорить анализ больших логов на своём ноутбуке. Он услышал про библиотеку Dask . Что делает Dask особенным?

Подсказка: Подумай о том, как Pandas работает с одной машиной, а Dask — с множеством.

🔔 Шаг 02:

Миссия 2

👤 Инженер Марина решила заменить циклы Python на параллельные задачи Dask. Что из этого лучше описывает Dask.delayed ?

Подсказка: Эта функция не выполняет задачу сразу, а лишь создаёт граф вычислений.

🔔 Шаг 03:

Миссия 3

Команда стартапа решила перенести обработку данных на кластер. Они выбирают между Dask и Ray. В чём основное различие между ними?

Подсказка: Один больше про данные, другой — про акторы и распределённые функции.

🔔 Шаг 04:

Миссия 4

⚙️ Артём запустил Dask DataFrame, но не увидел результатов. Почему?

Подсказка: Dask использует ленивые вычисления — нужно явное выполнение.

🔔 Шаг 05:

Миссия 5

🌐 В лаборатории по анализу данных требуется распределённый планировщик для Dask. Какой компонент за это отвечает?

Подсказка: Этот компонент управляет распределением задач между рабочими узлами.

🔗 Шаг 06:

Миссия 6

📄 Разработчик Илья хочет использовать Dask для обработки CSV-файлов размером 200 ГБ. Что он должен сделать в первую очередь?

Подсказка: Dask не загружает весь файл сразу — он делит данные на части.

🔗 Шаг 07:

Миссия 7

🔗 Лена использует Ray и хочет запустить функцию в параллельном режиме. Какой декоратор ей нужен?

Подсказка: Этот декоратор превращает обычную функцию в задачу, выполняемую на кластере.

🔗 Шаг 08:

Миссия 8

👤 Разработчик Денис использует Ray для управления состоянием между задачами. Что ему поможет?

Подсказка: Эта сущность похожа на объект с методами, работающий в другом процессе.

🔗 Шаг 09:

Миссия 9

🔗 Сопоставьте компоненты Dask с их назначением.

Подсказка: Один управляет, другой работает, третий подключается.

🔗 Шаг 10:

Миссия 10

🔗 Сопоставьте объекты Dask с их аналогами в Pandas.

Подсказка: Они похожи по API, но работают по-разному.

🔗 Шаг 11:

Миссия 11

🔗 Сопоставьте основные понятия Ray с их описаниями.

Подсказка: Одни функции — отложенные, другие — постоянные.

🔗 Шаг 12:

Миссия 12

Сопоставьте термин с его значением в контексте распределённых вычислений.

Подсказка: Вспомни: данные, узлы, кластер, вычисления.

🔗 Шаг 13:

Миссия 13

⚡ Расположите шаги запуска Dask-кластера в правильном порядке.

Подсказка: Сначала стартует кластер, затем подключается клиент и выполняется код.

🔧 Шаг 14:

Миссия 14

Расположите этапы работы с Ray в правильной последовательности.

Подсказка: Сначала инициализация, потом задачи, потом сбор результатов.

🔧 Шаг 15:

Миссия 15

✳ Расположите этапы распределённой обработки данных в Dask.

Подсказка: Подумай, как строится граф, распределяются задачи и собираются результаты.

📖 Урок 2 - Распределённые пулы

🔧 Шаг 01:

Миссия 1

Разработчик Артём решил ускорить обработку изображений, используя multiprocessing.Pool . Что делает пул процессов?

Подсказка: Вспомните, как можно параллельно обрабатывать задачи на нескольких ядрах процессора.

🔧 Шаг 02:

Миссия 2

✳ Программистка Лена использует Pool.map() . Что делает этот метод?

Подсказка: Подумайте, как работает обычная функция map() , но теперь — с несколькими процессами.

🔧 Шаг 03:

Миссия 3

📖 Разработчик Тимур запускает задачи с помощью apply_async() . Чем этот метод отличается от map() ?

Подсказка: Один метод запускает всё сразу, другой — управляет задачами индивидуально.

🔧 Шаг 04:

Миссия 4

🔗 Команда стартапа анализирует логи сервера. Зачем вызывать pool.close() перед pool.join() ?

Подсказка: Эти методы управляют завершением работы пула. Один блокирует приём новых задач, другой ждёт завершения старых.

🔧 Шаг 05:

Миссия 5

🕒 Дата-инженер Кира обнаружила, что её пул зависает. Что может быть причиной?

Подсказка: Часто зависания возникают из-за неправильного завершения процессов или блокировки ввода/вывода.

🔧 Шаг 06:

Миссия 6

Программист Саша хочет узнать, как ограничить число одновременных процессов. Какой параметр нужно передать при создании пула?

Подсказка: Этот параметр задаёт количество рабочих процессов, создаваемых пулом.

Шаг 07:

Миссия 7

Сопоставьте методы пула с их назначением:

Подсказка: Один из них возвращает результаты сразу, другой — позже, третий закрывает пул.

Шаг 08:

Миссия 8

 Сопоставьте типы вызовов с их особенностями.

Подсказка: Синхронные блокируют поток, асинхронные — позволяют выполнять другие задачи.

Шаг 09:

Миссия 9

 Команда делит работу между процессами. Сопоставьте термин с его смыслом.

Подсказка: Процесс — это исполнитель, задача — это работа, а очередь — место, где они встречаются.

Шаг 10:

Миссия 10

 Сопоставьте тип пула с областью применения.

Подсказка: Потоки — для ввода-вывода, процессы — для тяжёлых вычислений.

Шаг 11:

Миссия 11

 Сопоставьте методы multiprocessing.Manager() с их типами данных.

Подсказка: Некоторые структуры безопасны для совместного доступа из разных процессов.

Шаг 12:

Миссия 12

 Сопоставьте ситуацию с правильным инструментом multiprocessing.

Подсказка: Если нужно делиться памятью — используйте Manager, если обмениваться сообщениями — Queue.

Шаг 13:

Миссия 13

 Расположите шаги запуска пула процессов в правильном порядке.

Подсказка: Сначала создаём пул, потом запускаем задачи, затем закрываем и ждём завершения.

Шаг 14:

Миссия 14

Расположите действия программиста при работе с распределёнными вычислениями.

Подсказка: Подумайте, что идёт раньше: подготовка данных или сбор результатов?

Шаг 15:

Миссия 15

⚙️ Расположите шаги завершения программы с пулом процессов.

Подсказка: Закрытие, ожидание, вывод — именно в таком порядке.

Шаг 16:

Миссия 16

⚙️ Расположите этапы оптимизации параллельных задач по сложности.

Подсказка: Сначала анализ, потом изменение структуры и только потом настройка ресурсов.

📖 Урок 3 - Практика обработка большого массива

📖 Урок 4 - Квест симуляция кластера

📖 Модуль 17 - Модуль 17. Параллелизм в облаке

📖 Урок 1 - AWS Lambda и Python

📖 Урок 2 - Docker и распределённые процессы

📖 Урок 3 - Практика параллельный запуск контейнеров

📖 Урок 4 - Квест оптимизация пайплайна

📖 Модуль 18 - Модуль 18. Тестирование параллельного кода

📖 Урок 1 - pytest и asyncio

📖 Урок 2 - Проверка многопоточности

📖 Урок 3 - Практика тестируем race condition

📖 Урок 4 - Квест найди непойманный поток

📖 Модуль 19 - Модуль 19. Безопасность и ошибки

📖 Урок 1 - Исключения в потоках и async

📖 Урок 2 - Обработка отмены задач

📖 Урок 3 - Практика устойчивый код

📖 Урок 4 - Квест поймай падение в проде

📖 Модуль 20 - Модуль 20. Архитектуры параллельных систем

📖 Урок 1 - Pipeline, MapReduce, PubSub

📖 Урок 2 - Архитектура задач с брокером

📖 Урок 3 - Практика проектирование пайплайна

📖 Урок 4 - Квест собери архитектуру мечты

📖 Модуль 21 - Модуль 21. Python и GPU

📖 Урок 1 - NumPy, Numba, CuPy

📖 Урок 2 - Практика ускорение вычислений на GPU

📖 Урок 3 - Квест переписи CPU-код под GPU

Модуль 22 - Модуль 22. Параллельный ввод-вывод

 **Урок 1 - Асинхронные файлы**

 **Урок 2 - Параллельная запись логов**

 **Урок 3 - Практика логирование в несколько потоков**

 **Урок 4 - Квест лог без потерь**

Модуль 23 - Модуль 23. Отладка и визуализация потоков

 **Урок 1 - Thread Inspector и визуальные трекеры**

 **Урок 2 - Мониторинг asyncio loop**

 **Урок 3 - Практика визуализатор потоков**

 **Урок 4 - Квест найди медленную задачу**

Модуль 24 - Модуль 24. Финальный проект — Битва за скорость

 **Урок 1 - Постановка задачи API + вычисления**

 **Урок 2 - Планирование параллельной архитектуры**

 **Урок 3 - Реализация и профилирование**

 **Урок 4 - Итоги кто быстрее**

Модуль 25 - Модуль 25. Финальный экзамен Герой Python

 **Урок 1 - Тестовый квест потоки, async, процессы**

 **Урок 2 - Практическое задание оптимизируй код**

 **Урок 3 - Рефлексия и анализ**

 **Урок 4 - Сертификат и бонусы**